

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Transforming 2D Images into 3D Child Avatars – Enhancing Self-Attachment Technique with 3D Reconstruction

Project Report

Author: Raymond Guo

Supervisor: Professor Abbas Edalat

MSc Computing

September 9, 2024

Abstract

3D reconstruction is becoming increasingly popular owing to its ability to provide immersive, realistic and engaging experiences in its applications. These benefits can be extended into the domain of a digital psychotherapy intervention employing a self-administered therapy called the Self-Attachment Technique (SAT), which involves sourcing one’s own emotional comfort by interacting with one’s childhood self. It is proposed that by integrating a realistic and personable 3D-avatar head reconstruction of oneself as a child, that the effectiveness of a digital therapy application can be heightened. In this project, we therefore aim to investigate how to accurately reconstruct a 3D head. We explore a number of different approaches in the literature, showing that methods situated in the 3D Morphable Model (3DMM) field provides an excellent means of capturing a face’s shape, expression, and skin texture. We discover an approach called 3D Face Reconstruction with FFHQ-UV that provides the most robust means of reconstructing a head avatar, combining a reliable skin texture trained from an extensive texture dataset with a popular 3DMM head topology called HIFI3D++. The accuracy of this model is verified through the REALY benchmark, demonstrating a shape accuracy rivalling the top face reconstruction research papers placing third on average. A user feedback evaluation of this model also demonstrated generally positive feedback with an average satisfaction of 4.03 out of 5, with subjects being particularly impressed by the speed of reconstruction. Furthermore, a short application allowing one to dynamically control the expression of the face model according to the six basic emotions (happiness, sadness, fear, anger, surprise, and disgust) was created. This was achieved by assigning semantic meaning to arbitrary expression bases in the 3DMM head topology, representing the first implementation to do so.

Acknowledgements

I would like to thank Professor Abbas Edalat and Xinyan Ye for their advice and guidance during the completion of this project. I would also like to thank my family and friends for their constant support.

Contents

Abstract	1
Acknowledgements	2
1 Introduction	5
2 Background and Related Work	7
2.1 Fundamentals in 3D Reconstruction	8
2.1.1 3D Output Representations	8
2.1.2 Important Deep Learning Architectures	9
2.2 An Introduction to 3D Morphable Models	11
2.2.1 Mathematical Theory Behind the 3D Morphable Model	12
2.2.2 Extending the Traditional 3DMM	13
2.2.3 3D Morphable Model Topology	14
2.3 Combining 3D Morphable Models with Deep-Learning	14
2.3.1 Auto-Encoder 3DMMs	15
2.3.2 GAN-Based 3DMMs	19
2.4 3D Deep-Learning Approaches for Reconstruction	23
3 Methodology	27
3.1 Design Considerations	27
3.2 Model Experimentation	28
3.2.1 3D-VAE-GAN	28
3.2.2 Deep 3D Face Reconstruction	31
3.2.3 A Pipeline for Head Model Reconstruction Using the Cascaded Refinement Network	31
3.2.4 FLAME Reproduction	37
3.3 3D Face Reconstruction with FFHQ-UV	39
3.3.1 Justification for Implementation	39
3.3.2 Reconstruction Algorithm	39
3.3.3 Additions to the Rendering Pipeline	40
3.4 Extension: Trialling an Interactive Application for Dynamic Facial Expression Change	40
3.4.1 Justification	40
3.4.2 Application Concept and Design	41
4 Implementation	42
4.1 3D-VAE-GAN	42

4.2	Deep 3D Face Reconstruction	44
4.3	A Pipeline for Head Model Reconstruction Using the Cascaded Refinement Network	46
4.4	FLAME-Fitting Reproduction	48
4.5	3D Face Reconstruction with FFHQ-UV	50
4.5.1	Extension: Expression Change Application	52
5	Evaluation	55
5.1	A Brief Evaluation of the Experimented Models	55
5.1.1	3D-VAE-GAN	55
5.1.2	Deep 3D Face Reconstruction	56
5.1.3	A Pipeline for Head Model Reconstruction Using the Cascaded Refinement Network	56
5.1.4	FLAME-Fitting	58
5.2	3D Face Reconstruction with FFHQ-UV	59
5.2.1	REALY Benchmark	59
5.2.2	User Feedback Survey	61
5.2.3	Qualitative Evaluation of the Model Framework	63
5.2.4	A Brief Word on the Expression Change Application	67
5.3	Ethical Considerations	67
6	Conclusion	69
	Bibliography	71
	Appendix	80
	A Survey for the 3D Face Reconstruction with FFHQ-UV	81

Chapter 1

Introduction

3D reconstruction is an increasingly relevant field of research owing to its diverse range of applications in areas such as gaming and virtual reality [1], archaeology [2], and medical imaging [3]. With the aim of determining the 3D structure of an object of interest from single or multiple images [4], the positive impact of reconstruction is derived from its ability to provide more realistic settings and personal experiences to those that engage with it. Acknowledging this potential, it has been proposed that the generation of avatars using 3D reconstruction methods could be used as a legitimate form of social interaction designed to facilitate digital psychotherapy interventions. Such digital technologies have justifiably been considered as potential tools for therapy[5], owing to the issues of affordability for many when seeking mental healthcare [6] as well as accessibility as was observed during the COVID-19 pandemic.

The present research therefore aims to engineer a method for reconstructing an accurate and personable avatar head model to complement the Self-Attachment Technique (SAT) [7], a form of self-administered therapy developed by the Algorithmic Human Development Group at Imperial College London. In SAT, it is proposed that individuals possess a logical ‘inner adult’ and an emotional ‘inner child’. The premise of SAT is that instead of relying on a caregiver to provide the emotional security and comfort as posited in Bowlby’s attachment theory [8], one can rely on themselves to source the positive developmental benefits that secure attachment can provide by establishing interactions between the left-side (logical) and right-side (emotional) part of the brain [7]. It is hoped that the development of a high-quality realistic childhood avatar will enhance the affectionate interactions between the inner adult and child [9], and that the gamification of digital therapy by integrating avatars will provide heightened positive impact on the efficacy of therapy, as has been proven in similar VR interventions [10].

Developing the model with this intended use case in mind, it is not simply the pure quality of the rendering output that matters, but also how reliable the model is, the time taken to reconstruct the head, and how dynamic the model can be if used in a virtual-reality intervention requiring some form of interactivity such as changing facial expressions. The task of 3D-avatar head reconstruction is non-trivial however, requiring the learning of intricate facial details accounting for the facial shape, the texture (which refers to all aspects relating to colour such as skin colour or hair colour), and the facial expression [11]. Therefore, despite

numerous methods for 3D reconstruction having been proposed in the literature capable of reconstructing daily-life objects such as cars and chairs [12, 13], the methods required to reconstruct a head require far more consideration and nuance in order to capture the full complexity of an individual’s facial features.

Interestingly, despite a boom in computational processing power in recent decades yielding highly complex state-of-the-art deep-learning architectures capable of achieving great things in a variety of applications, the primary success of face and head modelling has come from a relatively more humble theoretical origin: the 3D Morphable Model (3DMM) [14]. In the 3DMM, a statistical method called Principal Component Analysis (PCA) is used [15], which proposes that all faces can be parameterised in the same statistical space. This means that an individual’s unique face shape, texture, and expression can be modelled as just a linear deviation from some average face’s shape, texture, and expression measured by vector coefficients [11]. Pioneered by Blanz & Vetter [11], the 3DMM has been improved upon and extended over decades to facilitate more accurate face and head reconstruction.

Specifically, success has been derived from embedding the principals of the 3DMM within deep-learning architectures [14], with the Auto-Encoder (AE) and Generative Adversarial Network (GAN) architectures being particularly popular. From the perspective of the AE, the main goal has been to learn the vector coefficients that best support the transformation of a 2D-image to a 3D output representation [16, 17]. This has been tackled from both a linear perspective directly using 3DMMs such as the famous Basel Face Model [18], and also a non-linear perspective in learning the 3DMM parameters without PCA [19]. GANs have been used to tackle other difficult tasks in reconstruction, often involving the capturing of texture [20, 21]. The merging of the world of 3DMMs with deep-learning has consequently led to impressive face and head reconstruction results throughout the literature.

In the present study, multiple experiments are carried out on leading research in the face and head reconstruction domain holding 3DMMs at its core. The choice of these models are justified based on the specific problems that arise from 3D head reconstruction, with each model building on a limitation of the previous model, in accordance with the chronology of the development phase. After extensive experimentation, it was found that the 3D Face Reconstruction with FFHQ-UV pipeline provides the most convincing results [20], with its ability to comprehensively model an entire head with consistent and smooth texture, while doing so with an acceptable rendering speed. This was objectively evaluated by carrying out model benchmarking on the popular head reconstruction dataset *REALY* [22], which placed the FFHQ-UV model at third in-class compared to other leading head reconstruction models. Feedback was also obtained in a survey from study participants who had their head models rendered using the FFHQ-UV framework, reflecting generally positive sentiments, and providing constructive feedback for areas of improvement.

With the 3D Face Reconstruction with FFHQ-UV [20] being established as the model best satisfying the static rendering of the head model, the project turned towards the goal of dynamically changing the facial expression, helping to facilitate a potential interactive digital psychotherapy intervention. This was achieved through using the rendered head using the FFHQ-UV pipeline and applying changes to the coefficients as per the philosophy of 3DMMs [20]. Through trial-and-error, the combinations of the expression coefficients in the 3DMM were mapped to the 6 basic emotions [23] representing the first implementation to achieve this.

Chapter 2

Background and Related Work

In this section, a detailed overview of the current state of research in face and head reconstruction is discussed. Numerous approaches have been proposed in the general field of 3D reconstruction, differing by the type of input data supplied, the desired output representation, the network architecture, and the method of training [24]. Traditional methods in 3D reconstruction originated from theoretical backgrounds in geometry and photogrammetry, requiring a complex and detailed understanding of a 3D object from multiple perspectives [25], using sensitive camera calibration to achieve effective feature matching across views [26]. However, as Wang and Fang [25] suggest, approaches in deep-learning present an alternative yet exciting solution to 3D reconstruction.

In the reconstruction literature, many deep-learning frameworks have been proposed, from more tried-and-tested architectures such as auto-encoders (AEs) [27] and Generative Adversarial Networks (GANs) [28] to more novel but fast-growing techniques such as Diffusion Models [29], Neural-Radiance Fields (NeRFs) [30] and 3D Gaussian Splatting [31]. Currently, the application of the more novel approaches has seen less coverage in face and head reconstruction, whereas AEs and GANs have played and continue to play a large role in this research domain. The popularity of these networks perhaps comes as a result of their ability to learn complex 3D data while maintaining relative simplicity of implementation. However, as will be discussed later and further shown in implementation, the face and head reconstruction domain fields its own specific requirements meaning that AE and GAN architectures designed for reconstruction trained using non-face/head objects do not extrapolate so well when trained using faces and heads.

This therefore necessitates the incorporation of domain-specific research in the name of 3D Morphable Models (3DMMs) [11], which have made significant progress in the study of face and head reconstruction and modelling. At its most basic level, 3DMMs are far simpler, targeting three crucial elements defining the accuracy of facial reconstruction: the geometric shape, the capturing of the face’s expression (which can therefore also be thought of as shape), and the texture (which refers to the colouring) [11]. In the seminal work by Blanz & Vetter (1999) [11], it was proposed that a unique face’s shape, expression, and texture can be represented as linearly applying three separate coefficient vectors (i. e. shape, expression, and texture) to those on a base face to represent and ultimately model the variations [11]. While such an approach was developed in the 90s, it still represents a simple

yet strong framework for face modelling. As compute-power has grown over these last few decades however, deep-learning architectures have been able to take advantage of the 3DMM foundation to continuously improve face and head reconstruction. The crux of the research is therefore based upon understanding how 3DMMs have been incorporated with deep-learning architectures such as AEs and GANs to achieve promising reconstruction results.

This background section is designed to chronologically guide one’s understanding of the leading research in head reconstruction and explain their theoretical underpinnings. To facilitate a more intuitive understanding, some fundamental knowledge is first established in the form of 3D output representations, and the fundamental AE and GAN architectures. The section will then detail the initial theory and literature in 3D Morphable Models, before exploring how they have been incorporated with the AE and GAN architectures. Finally, some useful deep-learning approaches to reconstruction are discussed that have not been properly applied to the domain of head modelling. This is because some of these approaches are later trialled in implementation, and may provide inspiration for future implementation.

2.1 Fundamentals in 3D Reconstruction

2.1.1 3D Output Representations

An area of initial importance is establishing the common output representations for a 3D reconstructed object. Broadly speaking, outputs can be categorised into one of two representations: volumetric representations and geometric/surface-based representations [26, 24]. Figure 2.1 presents a visual depiction, where (a) and (b) are deemed volumetric, and (c) and (d) deemed geometric. Intuitively, it can be seen that volumetric representations are cubic because they choose to parameterise the object using grids [24]. While this can be computationally expensive [24], they are also popular due to their simplicity of implementation. On the other hand, geometric representations are finer in detail, comprising a set of points that are either unconnected (as in (c)), or connected (d) [26]. While volumetric and geometric output representations comprise the majority of papers and will be the sole focus for outputs in this project, it must be noted that representations outside of these categories also exist such as implicit surface representations [26].

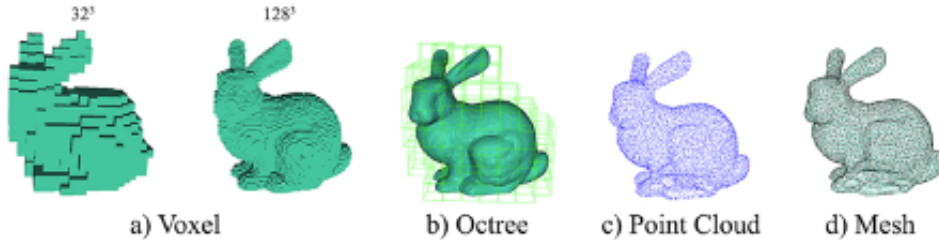


Figure 2.1: Common output representations for 3D reconstruction. Objects **a** (voxels) and **b** (octrees) are volumetric representations, whereas **c** (point clouds) and **d** (meshes) are geometric. Figure taken from ‘Single-View 3D reconstruction: A Survey of deep learning’ by Fahim et al. [26].

Volumetric Representations

In volumetric representations, voxels or octrees are most common. As seen in Figure 2.1, voxels use cubes to fill the expected 3D object, implementing an occupancy grid to determine the presence of a cube in a 3D net [24]. One potential representation is a binary occupancy grid returning 1 to represent the presence of a voxel for the object of interest and 0 representing an empty voxel [32]. This can also be probabilistic, assigning a probability for a voxel belonging to the object of interest [24]. As shown in Figure 2.1 (a and b), the resolution of the object can be modified [26], representing a trade-off between intricacy of surface representation, and computational efficiency. Higher resolutions are naturally preferred from an accuracy standpoint, resulting in a line of work aiming to increase the computational efficiency of deploying voxel representations [33, 34, 35].

An approach to help counter the disadvantage of poor space optimisation in voxels is the octree data structure [36]. Whereas voxels remain uniform and fixed, octrees are able to dynamically represent space by implementing nodes that can recursively divide into eight differently sized octants to fill the space [26]. Therefore, areas requiring little intricacy need not require heavy subdivision, but more detailed areas can be represented with more intricacy through greater subdivisions. As proposed in the *OctNet* by Riegler et al. [37], data can be sparse and not always dense in the way that is implied by voxel implementations. They propose a more adaptive strategy to limit the number of wasted computations, observing that points in a 3D object can be sparser when further away from the surface, thus requiring less subdivision and retaining a larger octree cell. On the other hand, cells closer to the surface often require finer and higher resolutions, to which computation can be focused towards those areas instead [26, 37]. This therefore improves the computational efficiency of volumetric reconstruction. Despite this however, octrees still follow a cubic approach, such that it can never truly represent surface curvature perfectly [26].

Geometric Representations

While volumetric approaches physically capture the volume of the 3D object, the geometric approach can be simpler, positing that reconstructing the surface is the priority. The first commonly used geometric method is a point cloud, a set of scattered points with 3D coordinates that represent the surface [4]. Meshes are slightly more complex, but are effectively point clouds represented as a series of connected edges between vertices (points) that combine to represent the surface [26]. These representations are more memory efficient [4], but can also capture more intricate details that volumetric approaches can neglect [24]. However, they are more difficult to represent in deep-learning models [24], and the lack of explicit ordering of points can render them difficult to implement [26]. There are also potential difficulties in extrapolating full information about the point clouds due to the sheer complexity of the target 3D object, the data available to perform construction, and object occlusion [4]. As will be shown in the technical experimentation and evaluation, geometric approaches meshes are more effective than volumetric approaches in the field of face and head modelling, with the task of capturing the intricacies of faces being too challenging for the latter. Even if the resolution were to be scaled up in a volumetric voxel approach, this would also introduce computational efficiency issues that does not serve the rendering requirements.

2.1.2 Important Deep Learning Architectures

Auto-Encoders

The auto-encoder (AE) is a fundamental architecture used in the reconstruction literature. Auto-encoders are comprised of two principal elements: the encoder and decoder. The basic functionality operates such that an encoder extracts the features of the input (e.g. a 2D image) into a lower-dimensional space called a latent (often represented with the symbol $\mathbf{z}$ [27]). This vector is then decoded by learning the decoding function to try to best represent the information captured in the latent as the output (e.g. a 3D object) [24]. An example is shown in the figure below, where it can be seen that the 2D image of the rabbit is passed into the encoder which derives a latent code, which is passed into the decoder to produce a voxelised 3D output. Convolutional Neural Networks (CNNs) are often used in AE architectures [38], and exciting new architectures have emerged from the basic AE such as the U-Net [39], while maintaining the capacity to incorporate other leading architectures such as VGGNet and ResNet [40, 41].

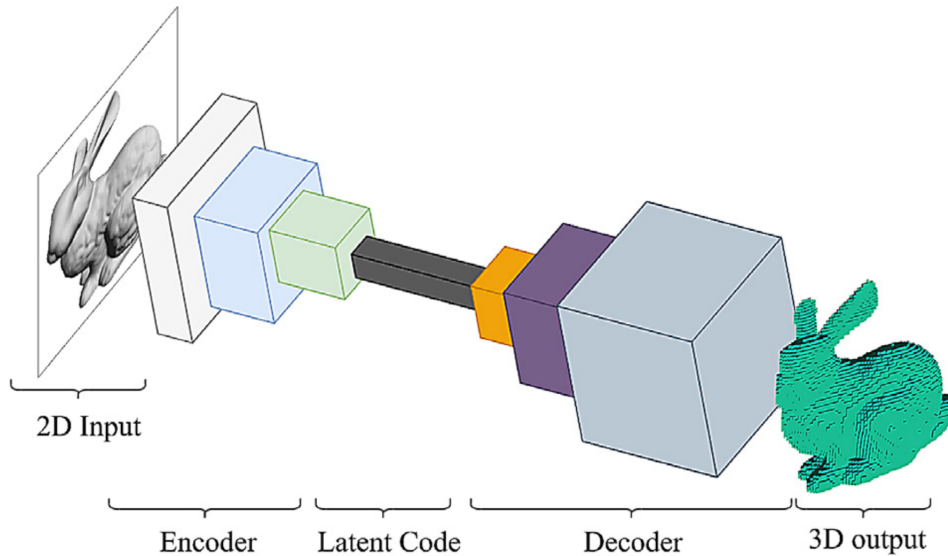


Figure 2.2: The classic Auto-Encoder pipeline. Figure taken from ‘Single-View 3D reconstruction: A Survey of deep learning’ by Fahim et al. [26].

Generative Adversarial Networks (GANs)

Generative Adversarial Networks are an alternative method for 3D reconstruction proposed by Goodfellow et al. [28]. The basis of GANs works by having two adversarial forces: generators and discriminators. Generators aim to generate data that captures the true data distribution as closely as possible, analogous to counterfeiters aiming to produce fake goods so realistic that they are indistinguishable to those that try to determine whether the goods are real or fake. The determination of real from fake is the role of the discriminator. The adversarial nature of GANs is thus derived from the competitive process of the discriminator aiming to maximise the probability of correctly labelling training from generator data, while the generator aims to minimise this probability (see Equation 2.1) [28]. Through back-propagation of the loss function and stochastic gradient descent usually requiring a Multi-Layer Perceptron network, the generator is continuously improved until the discriminator can no longer distinguish the

training from generator data [42].

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (2.1)$$

Equation 2.1 (taken from [28]) presents the most traditional GAN objective, representing a two-player min-max game [28]. Taking the perspective of the discriminator, the $\mathbb{E}_{x \sim p_{data}(x)} [\log D(x)]$ refers to the expected value of the log probability that a value x sampled from a probability distribution $p_{data}(x)$ will be correctly classified as real. $\mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$ represents the second component, which conversely refers to the expected value of the log probability of a noise vector z sampled from a probability distribution $p_z(z)$ being correctly classified as being fake (i.e. generated by $G(z)$). Combined together, this motivates the $\max_D$ goal of the equation in maximising the correct prediction of whether a generation was real or fake. This is wrapped inside the competing goal of $\min_G$, which has the aim of minimising the function denoting the success of the generator in fooling the discriminator. The crucial point of the equation is therefore in the $D(G(z))$, with values closer to 0 demonstrating the labelling of the generated output as more fake than real, and values closer to 1 denoting the labelling of the generated output as more real than fake.

Having established the fundamental theoretical knowledge required to understand the research here and in the implementation, this section delves into the field of study that has been most influential in face and head reconstruction - 3D Morphable Models.

2.2 An Introduction to 3D Morphable Models

The application of 3D Morphable Models in computer vision has been significant across a variety of domains [43], in particular 3D face modelling [11, 44, 18, 16]. The theoretical motivation behind the 3D Morphable Model (3DMM) lies in understanding how to capture the variation of a single class of objects in statistical space [14]. In the case of 3D face reconstruction, the class of objects are the human faces, and the goal is to mathematically model the variation between faces in a way that preserves as many of the facial details as possible, while constraining its complexity to allow for easier inference. Accounting for the exact differences between faces is clearly complex, if not for a crucial principle that traditional 3DMMs follow: dense point-to-point correspondence [14]. This refers to the concept of mapping a point on an object to the same point in another object, therefore allowing one to parameterise the differences. For example, the tip of the nose on one individual may be in a slightly different location on another individual's face, but because of dense point-to-point correspondence, the 3DMM labels that point as the tip of the nose across all objects in that class, which facilitates easier tracking and modelling of within-class differences.

Early research introduced the idea of representing faces in a vector space (termed Eigenfaces), using eigenvectors to model facial variation derived from Principal Component Analysis (PCA) [45, 46]. PCA is a form of dimensionality reduction which aims to compress a large number of variables into a smaller set while preserving as much useful information to the specific case as possible, ensuring that the smaller set of variables each provide unique and significant explanation of the variance [15]. This aligns with the motivations behind the 3DMM, and therefore represents its theoretical underpinnings [19, 11]. The pioneering work

of 3DMMs was proposed by Blanz and Vetter (1999) [11], adopting the linear PCA model to model the variation between faces. Explained intuitively, the idea behind Blanz and Vetter’s 3DMM is that if we compute the average face across the different 3D face samples, we can simply represent the differences between faces by applying some linear combination of weights to the different features derived from the PCA model. Given the linear nature of the model, it allows one to flexibly separate categories of variation, commonly into shape, expression, and texture/appearance models [14].

2.2.1 Mathematical Theory Behind the 3D Morphable Model

The mathematical foundations of the 3DMM are explained here, taken from the pioneering paper by Blanz & Vetter (1999) [11]. At a basic level, one can interpret the shape of the 3D head model as a set of vertices (or points) with corresponding X, Y, and Z coordinates, such that shape $S = (X_1, Y_1, Z_1, X_2, \dots, Y_n, Z_n)^T \in \mathbb{R}^{3n}$. The $3n$ is derived from n being the number of vertices, and each vertex having 3 vector values (X, Y, Z). To map texture, which essentially refers to the colour mapping of a 3D model, it is assumed that there is an equal number of texture values to number of points in the model, such that texture $T = (R_1, G_1, B_1, R_2, \dots, G_n, B_n)^T \in \mathbb{R}^{3n}$, such that each point n (represented in the subscript of the vector) has 3 values attached to it - the corresponding RGB values of that vertex. With knowledge of shape and texture vectors, one is able to model the new shape S_{mod} and texture T_{mod} models as shown below.

$$\mathbf{S}_{mod} = \sum_{i=1}^m a_i \mathbf{S}_i \quad (2.2)$$

$$\mathbf{T}_{mod} = \sum_{i=1}^m b_i \mathbf{T}_i \quad (2.3)$$

$$\sum_{i=1}^m a_i = \sum_{i=1}^m b_i = 1 \quad (2.4)$$

In equations 2.2 and 2.3, the shape S_{mod} and texture T_{mod} of the model are derived from applying multiple regression across the vertices. The shape represents a function of the model’s shape vector S and a parameterised set of coefficients in $a = (a_1, a_2 \dots a_m)^T$, while the texture is a function of the model’s texture vector T and another set of coefficients in $b = (b_1, b_2 \dots b_m)^T$. In Blanz and Vetter’s 3DMM [11], the elements in vectors a and b are optimised by fitting a multivariate Gaussian distribution to determine the probability of their appearance in the model. This is achieved by first calculating the average shape $\bar{S}$ and texture $\bar{T}$, along with their covariance matrices C_s and C_t which are derived from the differences between the shape model and the computed average $\Delta S_i = S_i - \bar{S}$ and $\Delta T_i = T_i - \bar{T}$. In equation 2.5, s_i and t_i are the eigenvectors resulting from C_s and C_t , and in equations 2.6, σ_i is the eigenvalue of C_s and C_t respectively.

$$S_{model} = \bar{S} + \sum_{i=1}^{m-1} \alpha_i s_i \quad T_{model} = \bar{T} + \sum_{i=1}^{m-1} \beta_i t_i \quad (2.5)$$

$$p(\vec{\alpha}) \rightarrow \exp[-\frac{1}{2}\sum_{i=1}^{m-1}(\alpha_i/\sigma_i)^2] \quad p(\vec{\beta}) \rightarrow \exp[-\frac{1}{2}\sum_{i=1}^{m-1}(\beta_i/\sigma_i)^2] \quad (2.6)$$

While the above approach by Blanz and Vetter models the shape, effort has also been made to separately capture the expression of the face, albeit in the same linear manner as previously mentioned. Specifically, parameterising the differences between expressions was seen simply as linearly capturing the differences between an expressive face model and a neutral face model of the same subject [11, 14]. This can be represented by the equation: $\delta := \mathbf{c}_{exp} - \mathbf{c}_{ne}$, more formally represented in equation 2.7, where $\bar{c}$ still represents the mean expression, but where E^s and E^e is the matrix basis for the model’s shape and expression, and w^s and w^e represent the modifiable coefficients.

$$c(w^s, w^e) = \bar{c} + E^s w^s + E^e w^e \quad (2.7)$$

2.2.2 Extending the Traditional 3DMM

As demonstrated in the mathematical formulation thus far, the entire foundation of the original 3DMM proposed by Blanz and Vetter is a system of linear combinations applied to represent the facial variation. This has made such a proposition versatile in its ability to separate different aspects of a face model. For example, appearance modelling has not yet been discussed, but which can also be structured in the same way to achieve the same effects, focusing upon capturing aspects such as albedo and illumination [14]. The impact of the 3DMM has been nothing short of significant, and has spurred on significant research to build and improve on its existing framework.

Given Blanz and Vetter’s research featured a small sample size of around 200 individuals with primarily neutral expressions, the scope for further research remained large not only in the proposition of novel approaches but also in the simple replication and introduction of more representation into the morphable model. Attempts have been made to separate PCA models for expression completely [47, 48], with some jointly modelling expression with albedo as in the Face2Face framework [49]. Furthermore, while Blanz and Vetter took a more global approach towards modelling the entire face, another proposed idea has been to take features at a more local level to achieve greater granularity and resolution for higher quality outputs [14]. This has resulted in a line of work carrying out PCA on faces that were manually segmented [50, 51], but also automatically segmented [52]. In the multi-resolution framework [43], Huber et al. provided a method of specifying the mesh reconstruction resolution, and produced C++ software to enhance the ease of using 3DMM Face Models.

A common issue in the literature regards many 3DMMs being unable to reconstruct the entire head, often opting to fit face masks. Research has therefore also been carried out to address these issues. The implicit 3D Morphable Model (i3DMM) provided the first approach to be able to completely model the entire head with hair [53], while the framework proposed by Ploumpis et al. found a method to combine 3DMMs of different classes into one. In other words, the researchers were able to identify a method to merge multiple 3DMMs derived from different datasets, holding different templates, and focusing on different areas of representation [54]. Therefore, this framework provides a means of fusing a face-based 3DMM

with a head-based 3DMM, allowing finer shape reconstruction on the face, while allowing the full 3D-head render [54]. These findings demonstrate the gravity of the achievements that 3DMMs have been able to exhibit.

2.2.3 3D Morphable Model Topology

Given the early success of the Basel Face Model as a 3DMM [18], a number of other 3DMM topologies for heads have been proposed as shown in table 2.1. The topology effectively refers to how the head mesh has been structured, differing by the number of scans used when calculating the PCA, the number of PCA bases used to denote the axes of variation, and the number of vertices n_v and faces n_f . The motivation behind these different frameworks have been to try to more accurately represent the face model perhaps based on different data sources or with slightly different techniques for training the 3DMM.

	BFM[18]	FWH[55]	FLAME [56]	LSFM [44]	LYHM [57]	HIFI3D [58]	HIFI3D++ [22]
# scans	200	140	3800	8402	1212	200	1957
n_v	53215	11510	5023	53215	11510	20481	20481
n_f	105840	22800	9976	105840	22800	40832	40832
# basis	199	50	300	158	100	200	526

Table 2.1: Table showing different 3DMM topologies, taken from the REALY website [22]. # scans refers to the number of head scans used to fit the 3DMM, n_v and n_f refers to the number of vertices and faces in the topology, and # basis refers to the number of PCA basis represented in the 3DMM topology.

While a number of topologies have been shown in the table, one particularly successful approach in the literature has been in the introduction of FLAME, which stands for Faces Learned with an Articulated Model and Expressions proposed by Li et al. (2017) [56]. In this paper, the researchers proposed a novel topology aimed at improving the capture of shape and expression. This is a problem they desired to solve particularly since existing 3DMM fittings such as the Basel Face Model [18] and the expression model of FaceWarehouse [55] lacked expressivity and cross-cultural representation [11]. Training on a far larger dataset, Li et al. proposed FLAME which separates identity, pose, and facial expression. The benefits of such a framework are that compared to other 3DMM topologies, it is more computationally efficient due to the low polygon count (see Table 2.1) [56]. As a consequence, such a topology maintains high expressivity while being easier to implement with graphics pipelines. FLAME also enables greater animation capabilities due to jointly modelling the head and neck to facilitate movements for pose deformation[56], as well as the separation of shape and expression allowing for independent control of the coefficients in each respective space. Later in the report, the results of FLAME-fitting will be shown, as well as reference to some of the other topologies shown in table 2.1.

2.3 Combining 3D Morphable Models with Deep-Learning

While traditional 3DMMs have been groundbreaking in the field of face modelling, deep-learning approaches over the last couple of decades have improved the reconstruction outputs

by either directly incorporating 3DMMs or extrapolating the theoretical underpinnings to apply to novel architectures. As will be seen, auto-encoders and generative adversarial networks are crucial in many of these frameworks.

2.3.1 Auto-Encoder 3DMMs

Linear Approaches

A popular approach in the literature has been in defining network architectures geared towards predicting the morphable model parameters of shape, expression and texture and combining them with external parameters such as lighting and pose. A couple of papers that have been particularly impressive are thus detailed below.

The Deep 3D Face Reconstruction model by Deng et al. (2019) [16] is a linear 3DMM-fitting framework which incorporates the Basel Face Model (BFM) [18]. This architecture has become quite popular in the field of research owing to its strong results. The paper offers various contributions to the literature, most notably in its definition of the loss functions that help to generate more accurate representations of the face. Specifically, they introduce an image-level loss responsible for detecting the finer-level details of the face, and a perception-level loss aimed at building a higher-level understanding of the entire face, thus counteracting any local minima issues that come with finer detection [16]. The model is weakly-supervised, taking only "facial landmarks, skin masks, and a pre-trained face recognition CNN" [16]. This is useful given the difficulty of obtaining sufficient and good-quality supervised pairings. The framework also incorporates multi-view image reconstruction input functionalities.

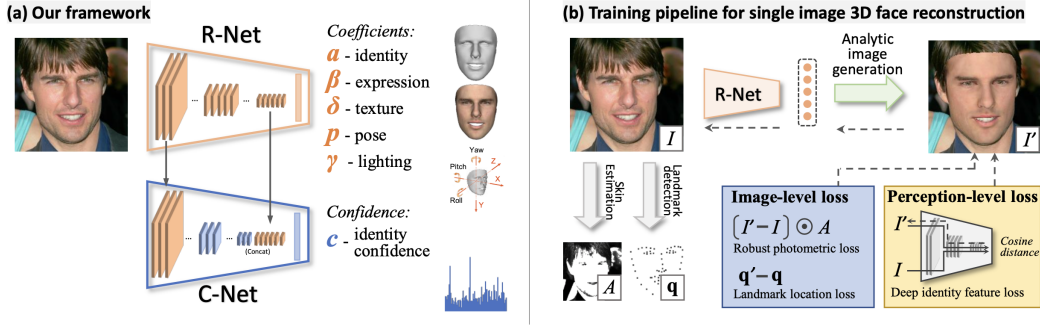


Figure 2.3: Diagram showing the Deep 3D Face Reconstruction architecture taken from Deng et al. (2019) [16]. (a) represents the notion of capturing the target coefficients of interesting using the ResNet-50 network [59]. In (b), the training flow is illustrated whereby the predicted coefficients by the network are rendered to produce a generated image, with calculation of image-level and perception-level losses between the source and target image being used to learn the 2D-image to 3D-face mask mapping.

Prior to discussing the modelling architecture and loss functions, it is important to establish the output of the CNN model and what it represents. Given that this framework extends from the 3DMM, it comes as no surprise that part of this output follows the goals of the traditional 3DMM [11]. The mathematical foundations of the 3DMM were discussed in part in the background review in section 2.2.1, but the following structure is explained according to the mathematical derivations of the paper by Deng et al. (2019) which features some

minor differences [16]. The identity (shape), expression and texture models of the 3DMM are preserved as crucial factors guiding the 3D face reconstruction. In equation 2.8, $\bar{S}$ and $\bar{T}$ represent the calculated average shape and texture of the face models in the collection, and the B_{id} , B_{exp} , and B_t are the identity (i.e. shape), expression, and texture bases respectively. Finally, the α , β , and δ are the corresponding coefficients that enable transformation from the 2D space to 3D space of the bases. The goal of this model is therefore to effectively estimate these vector coefficients. The identity and expression bases and averages are derived from the previously mentioned BFM [18], and the expression bases from FaceWarehouse [55]. To whittle down the face representation excluding the ear and neck, the vectors are represented with the dimensions $\alpha \in \mathbb{R}^{80}$, $\beta \in \mathbb{R}^{64}$, and $\delta \in \mathbb{R}^{80}$.

$$S = S(\alpha, \beta) = \bar{S} + B_{id}\alpha + B_{exp}\beta \quad T = T(\delta) = \bar{T} + B_t\delta \quad (2.8)$$

The model further considers the illumination and pose coefficients in determining the reconstruction outputs. The illumination coefficients after applying the spherical harmonics and Lambertian surface reflectance results in a vector $\gamma \in \mathbb{R}^9$. The pose consists of the rotation $\mathbf{R} \in SO(3)$ and translation $t \in \mathbb{R}^3$ vectors, overall forming 6 elements in the vector. Summing the identity, expression, texture, illumination and pose coefficients together, we obtain the target prediction vector $x = (\alpha, \beta, \delta, \gamma, p) \in \mathbb{R}^{239}$. To calculate these unknown coefficient parameters, a ResNet-50 network was established [59], but adapted such that the final fully-connected layer is instead a 239-element vector to fit x .

Perhaps the most significant contribution of this paper lies in its definition of the loss functions guiding the training of the model. Broadly speaking, these are categorised into three different types, image-level losses, perceptual-level losses, and regularisation losses. In the image-level loss, exists a sub-category called a photometric-loss, which essentially aims to capture the pixel-level differences (see equation 2.9). In the equation, i is the pixel index, M is the re-projected face region, $\|\cdot\|$ is the l_2 norm, and A is the skin attention mask [16], which is an algorithm designed to deal with issues such as occlusions and complex facial feature variations by assigning probabilistic values to skin-colour using a naive Bayes classifier and Gaussian Mixture Models [16].

$$L_{photo}(x) = \frac{\sum_{i \in M} A_i \cdot \|I_i - I'_i(x)\|_2}{\sum_{i \in M} A_i} \quad (2.9)$$

Another image-level loss is the landmark loss, with landmarks representing the detected points from face alignment techniques [60, 61]. In this paper, the 68-point landmark detection algorithm of Bulat and Tzimiropoulos (2017) is used [62]. These points are represented as $\{q_n\}$, and are compared in the loss function below in comparison to $\{q_n^i\}$ which represents the landmarks detected after projecting the 3D reconstruction onto a 2D image [16]. In the equation 2.10, w_n is the weight ascribed to the landmark features.

$$L_{lan}(x) = \frac{1}{N} \sum_{n=1}^N w_n \|q_n - q_n^i(x)\|^2 \quad (2.10)$$

Following the image-level losses, are the perception-level losses, which are important in mitigating the effects of solely relying on image-level losses, which in the paper were found to smooth the texture but reduce shape accuracy [16]. This also follows a weak supervision style, and essentially aims to extract the "deep features of the images" by computing cosine distance defined below:

$$L_{per}(x) = 1 - \frac{\langle f(I), f(I'(x)) \rangle}{\|f(I)\| \cdot \|f(I'(x))\|} \quad (2.11)$$

Finally, regularisation losses are employed that are defined to target the shape (L_{coef}) and texture (L_{tex}) components of the model [16]:

$$L_{coef}(x) = w_\alpha \|\alpha\|^2 + w_\beta \|\beta\|^2 + w_\gamma \|\delta\|^2 \quad (2.12)$$

$$L_{tex}(x) = \sum_{c \in \{r, g, b\}} var(T_{c,R}(x)) \quad (2.13)$$

A similar approach was proposed in MoFA by Tewari et al. [17], which trained an auto-encoder to encode a semantic vector storing information about "face pose, shape, expression, skin reflectance and scene illumination" and passing that through a decoder. MoFA is a completely unsupervised framework applied to single "in-the-wild" images, and is optimised by calculating the photometric loss E_{loss} between the decoded (synthesised image) and the original input image. The most important aspect of the model involves the encoding of the latent semantic vector $x \in \mathbb{R}^{257}$. The parameters of x can be summarised in equation 2.14, including shape $\alpha \in \mathbb{R}^{80}$, expression $\delta \in \mathbb{R}^{64}$, skin reflectance $\beta \in \mathbb{R}^{80}$, camera rotation $T \in SO(3)$, translation $t \in \mathbb{R}^3$, and scene illumination $\gamma \in \mathbb{R}^{27}$ [17], learned using pre-trained weights from AlexNet [63] or VGG-Face [64]. As demonstrated, the trainable coefficients of the model are in many ways similar to that proposed by [16].

$$x = (\alpha, \delta, \beta, T, t, \gamma) \quad (2.14)$$

The parameters in x are then passed into an image-formation decoder which attempts to render a synthesised image using a pinhole camera and spherical harmonics. To complete the framework, the photometric loss is calculated using the equation below. E_{land} refers to a sparse landmark alignment which can be optionally specified in a binary $w_{land} \in \{0, 1\}$. E_{photo} is a dense photometric alignment designed to calculate per-vertex differences, with w_{photo} set to 1.92. Finally, E_{reg} is a regularisation term designed to ensure that the face parameters α , δ and γ are standardised. $w_{reg} = 2.9 \times 10^{-5}$. The idea behind having both a dense and sparse loss metric is to facilitate faster convergence of the model [17].

$$E_{loss}(x) = w_{land} E_{land}(x) + w_{photo} E_{photo}(x) + w_{reg} E_{reg}(x) \quad (2.15)$$

Non-linear 3DMMs

While the approaches discussed thus far have taken a linear approach towards parameterising the face model, an interesting approach has been proposed in the realm of non-linear 3D Morphable Models. According to Tran et al. (2018) [19], facial expressions exhibit so much complexity of variation that the assumption that this can be represented in a non-linear perspective is tenable. They therefore proposed that instead of the previous statistical approach shown in traditional 3DMMs, that deep neural networks could be used to replace the PCA models entirely [19]. A crucial observation that they made is that the linear PCA model can simply be likened to a single linear layer in a neural network architecture, as one can simply flatten the entire vector space previously mentioned into a one-dimensional space. Furthermore, such a PCA model does not apply any activation function towards it. To introduce non-linearity into the model, Tran et al. adopt an auto-encoder architecture to replace PCA as shown in Figure 2.4.

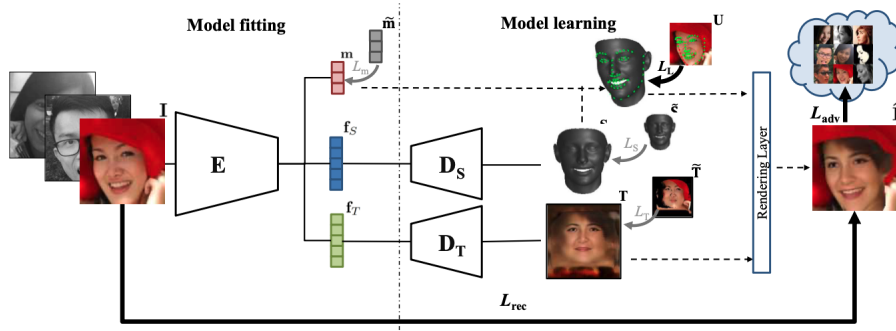


Figure 2.4: Diagram showing the Non-Linear 3DMM architecture taken from Tran et al. [19]. This architecture uses an auto-encoder to learn the shape, texture, and projection.

In the model by Tran et al. (2018) [19], the encoder attempts to learn the projection, shape, and texture parameters non-linearly taking a 2D image as input, with the shape being modelled using a Multi-Layer Perceptron (MLP) network, and the texture as a Convolutional Neural Network (CNN). To complement the shape and texture encoders, are their own respective decoders which essentially reconstruct the latent space encoding to return the 3D shape or texture output. To combine the shape, texture, and projection parameters, a rendering layer is created, which is notably differentiable to allow for backpropagation during learning. The model is trained to improve its reconstruction accuracy by combining the various parts of the architecture and optimising the model by minimising the loss between the reconstruction and input [19].

Another non-linear approach to modelling has been in the adoption of graph convolutional networks which has garnered increasing research [65, 66]. The application of mesh-based structures to Convolutional Neural Networks (CNNs) remains non-trivial, but can improve computational efficiency if implemented successfully. In a paper by Ranjan et al., a Convolutional Mesh Autoencoder (CoMA) was proposed to deal with 3D face generation [67] that learns "a non-linear representation of a face using spectral convolutions on a mesh surface". Through adopting a non-linear approach while preserving the mesh topology, CoMA [67] was able to outperform FLAME [56] on capturing the extremities of facial expression, and was also able to do so using less parameters. The CoMA architecture includes just a

single encoder and decoder in traditional CNN style, to which the former is responsible for down-sampling the mesh representation using Chebyshev convolutional filters, and the latter is used to upsample the mesh representation back to dimension of $\mathbb{R}^{5023 \times 3}$, which notably features the same number of vertices as FLAME [56].

CoMA is not the only approach that applies graph convolution however, and has also been implemented by Lin et al. (2020) [68]. In this paper, the primary focus is towards improving texture representation achieved using Graph Convolutional Networks (GCNs). In the first step of the model architecture, two pre-trained networks are provided. The first is the 3DMM-fitting framework as defined in Deng et al.’s work [16], capturing the shape, expression, and texture of the image, as well as the pose and lighting coefficients. The second is FaceNet [69], which operates like an encoder and is used to obtain a feature vector from the input image I . The second stage is the crucial stage of the approach by Lin et al. Firstly, the feature vector obtained from the image using FaceNet is passed into the GCN decoder with the aim of obtaining the RGB values of the vertices. The texture map obtained in the 3DMM-Fitting is passed into the GCN refiner network, which ”refines the vertex colors from the 3DMM model with spectral convolutional layers” [68]. Finally, the outputs of the GCN decoder and refiner are concatenated and fed into a graph convolutional layer with *tanh* activation to produce the final refined texture to be placed onto the unchanged shape [68].

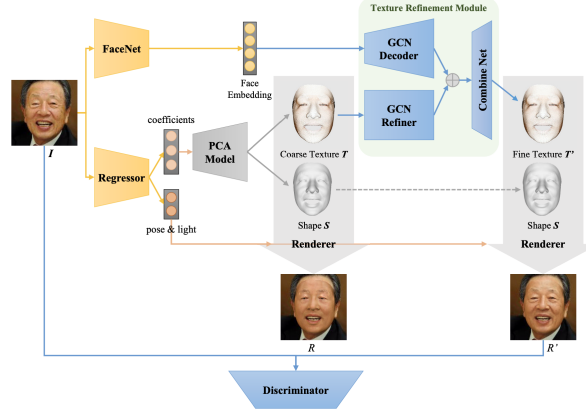


Figure 2.5: Diagram showing the architecture of the GCN by Lin et al. [68]

2.3.2 GAN-Based 3DMMs

While auto-encoder architectures have demonstrated some strong results in face and head reconstruction, GANs can help to further the accuracy and realism of reconstruction through the adversarial loss [28]. Such an advantage is especially beneficial for mapping textures, which remains a particularly challenging component of the overall reconstruction. While the per-vertex colouring in the original 3DMM by Blanz and Vetter is simpler to implement [11], the real high-quality texturing comes from overlaying UV-maps despite its challenging nature [20]. The FFHQ-UV approach provides a rigorous means of UV-unwrapping an entire head model, which becomes incredibly useful when combined with shape and expression topologies. The architecture posited by Bai et al. [20] adopted a GAN to generate textures

capable of integrating with 3D head models, and is described in detail below, later being implemented (see [methodology](#)).

FFHQ-UV

The faithful generation of UV-maps is a hot topic in the field of reconstruction. While high-quality renders can be carried out manually by 3D artists, this is also time-consuming and infeasible when fast test-time inference is required for certain applications. While point-cloud-based texture mappings are easier to understand, quality outputs are more conducive to texture-to-mesh mappings [20], to which two categories of approaches emerge: image translation-based and texture decoder-based. The former approach aims to work in a bottom-up sort of strategy by generating a map of lower quality and translating this into higher quality [58, 70, 71]. The latter is geared towards using 3DMM [11] to reconstruct the latent, but the decoder itself is non-linearly trained [72, 73, 74]. The texture-decoder approach is the opted framework used by FFHQ-UV [20].

Perhaps the most interesting contribution of this paper is how it cleverly deals with "in-the-wild" images, referring to pictures that have not been taken in any pre-specified setting or angle. As Bai et al. describe, "our pipeline utilizes the recent advances in StyleGAN-based facial image editing approaches to generate multi-view normalized face images from single-image inputs." Its ability to do so facilitates the reconstruction of the UV map used for the entire head model, which is interesting in its linkage to the UV-GAN proposed by [21] with regards to the multi-view methodology in trying to infer areas of a head model texture which cannot simply be inferred by the single-image. The following texture pipeline is thus described, and can be visualised below.

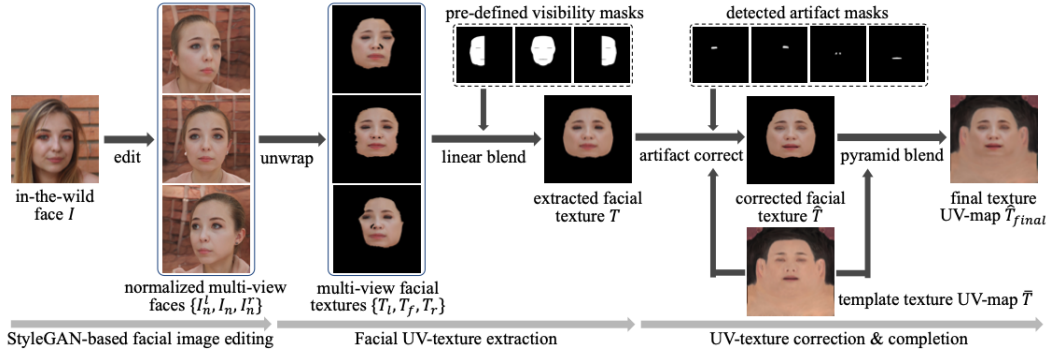


Figure 2.6: Diagram showing the FFHQ-UV texture pipeline [20].

The first step in the approach is to normalise the single-view photo in a way that produces even illumination, neutral expression, and pose [75, 76, 20]. This further includes consideration of occlusion-based issues such as glasses and hair covering the face in the "in-the-wild" image. In order to achieve this, the GAN-inversion method e4e is first applied to the input image I [77] which returns the latent w in $W+$ space [20]. The w latent is really important because it represents the editable code that allows one to deal with the modifiable attributes such as illumination, expression and pose. The latent is then passed through the generator G of the StyleGAN2 [78] to return the GAN-inverted image $I_{inv} = G(w)$. While I_{inv} should somewhat closely resemble input I , it is useful for the following steps. In the FFHQ-UV

approach, the attributes are detected using the [Microsoft Face API](#), but can indeed also be edited manually, covering bases on age, baldness, beard, expression, gender, glasses, pitch, and yaw. The lighting is predicted using the DPR model [79]. The normalisation occurs on the detected attributes, employing StyleFlow [80] and InterFaceGAN [81] to edit the latent code w to obtain a normalised latent $\tilde{w}$, which is finally passed through the StyleGAN2 generator to produce the new normalised images having edited the original latent such that $I_n = G(\tilde{w})$. To obtain the multi-view images, whereby the left and right angle of faces are generated, StyleFlow [80] is employed on the head pose attribute to obtain I_n^l and I_n^r respectively. As a consequence, it can be seen how useful StyleGAN can be in dealing with editing.

The next step of the process is to extract the UV-texture maps from the three normalised images I_f , I_r , and I_l . Firstly, the Deep3D Face Reconstruction model described previously [16] is trained using a 3D Morphable Model called HiFi3D++ (see Table 2.1) [22] to obtain the coefficients for each of the three input images, alongside head pose estimates [20]. The original input images are consequently unwrapped onto the 3D model derived from each morphable model fitting, further using a face parsing model (link [here](#)) to remove any aspects of the texture beyond the face. As shown in figure 2.6, the background, hair and ears are all removed. The output thus returns three texture maps corresponding to each original image: T_f , T_l , and T_r . The final component of the texture unwrap involves ensuring that the three texture maps are consistent in colour, which is achieved from " T_l and T_r to T_f for each channel in YUV color space" using equation 2.16, "where T_b , T_a , and T'_a are the target texture, source texture, and color-matched texture, respectively; μ and σ denote the mean and standard deviation; and w is a hyper-parameter (set to 1.5 empirically)" [20], which essentially controls colour contrast. The final component of the UV-texture extraction process is to fuse these texture maps together, done through linear blending using visibility masks to obtain the completed texture T .

$$T'_a = \frac{T_a - \mu(T_a)}{\sigma(T_a) \times w} \times \sigma(T_b) + \mu(T_b) \quad (2.16)$$

The final step in the texture map processing relates to the optimisation of the mapping to ensure consistency. As has been discussed in the two previous steps, various regions have been excluded due to being more error-prone such as the eyes and mouth. As a consequence, Bai et al. extrapolated an approach by excluding error-prone regions and filling them instead with a template texture map $\bar{T}$, followed by using Laplacian pyramid blending [82] to return the final texture $\bar{T}_{final}$. Overall, these three steps broadly represent the FFHQ-UV texture map pipeline, which can be applied to "in-the-wild" images to obtain the huge dataset as has been carried out in the original paper [20]. With the described pipeline, a large-scale robust dataset with over 50,000 UV maps was obtained comprising the FFHQ-UV dataset. FFHQ-UV was applied to train a GAN-based texture decoder capable of reconstructing a full 3D head texture map while maintaining high-quality outputs. The model used the architecture of StyleGAN2 [78], and is similar to GANFIT [74].

UV-GAN

The UV-GAN proposed by Deng et al. (2017) [21] provides a framework mirroring the motivation of the FFHQ-UV, recognising two crucial issues. The first involves the issue of

occlusion, which refers to objects or coverage of parts of the face preventing the model from operating correctly. This could include aspects such as glasses or facial hair. The second issue involves the pose, which is an ill-posed problem given that "in-the-wild" images may not always provide a clear-cut detection of the frontal profiles. This issue is further made difficult due to the lack of data capable of covering the poses [21]. Utilising the generative capabilities of the GAN, UV-GAN was proposed to deal with these two issues, and has been found to be more effective than a lot of face frontalisation methods [83, 84].

Using local and global Deep Convolutional Neural Networks (DCNNs) incorporated into an architecture involving a generator and two discriminators, the goal of UV-GAN is to learn the 3D embedding of an image to represent a 2D-texture map which can be fitted to mesh. Crucially, it aims to capture areas of the face from the image that are potentially occluded while preserving identity. It deploys a 3DMM to obtain the 3D face from the image. The generator works to construct the areas of the UV-textures that are missing using an embedded auto-encoder structure, the purpose of which provides an innate function to train the pixel-filling mechanism. It uses a pixel-wise l_1 norm as the reconstruction loss measured between the rendered estimate and the ground truth texture, and is learnt by filling the missing pixels in the texture with random noise. To obtain the other side of the face that cannot be seen, the face is mirrored and concatenated to achieve the purpose of rendering with views that cannot be seen. While it is perfectly legitimate to train a single auto-encoder in carrying out the task, the incorporation of the discriminator is also highly useful because it provides a way to ensure realism of output due to the distinguishing of real and fake UV maps as per the theory of GAN [28]. The model actually has two discriminators: local and global. This is done for the purpose of the global discriminator taking a more big-picture view of the face, but the local one focusing on preserving the detail of the face itself. While generating an entire representation of the face in the UV-map is the goal, it is important that one does not lose the original identity during training. As a solution to this, Deng et al. [21] proposed an Identity Preserving Module. To train the entire network, an objective function was proposed including the different losses: generator loss, global and local adversarial loss, and the identity preserving module loss with associated weights.

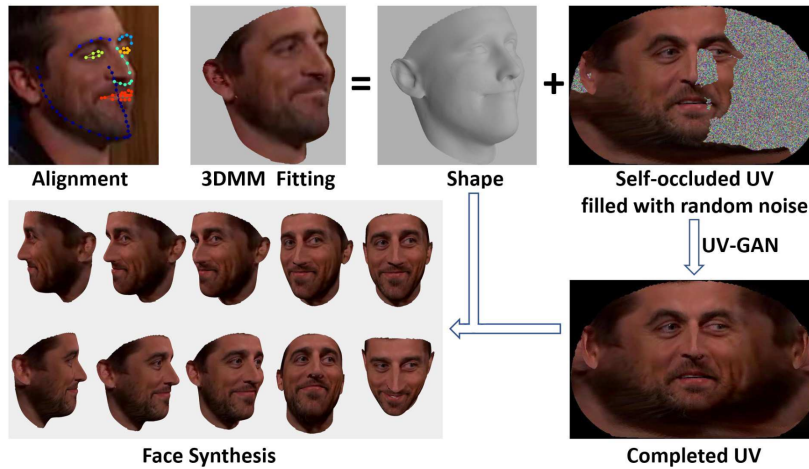


Figure 2.7: Diagram showing the UV-GAN workflow [21].

2.4 3D Deep-Learning Approaches for Reconstruction

In this section, a number of deep-learning models are detailed outside of the 3DMM framework which have not yet seen extensive integration with face or head data. Such approaches have been more successful when applied to the reconstruction of daily-life object datasets such as ShapeNet [85], prompting intrigue as to their applicability to head reconstruction. As will be seen in the [methodology](#), some of these approaches are experimented with the hope that non-domain-specific solutions to head reconstruction can also be sourced.

TL-Embedding

A seminary paper by Girdhar et al. [86] presented TL-embedding, a framework represented by two networks: a T-Network for training, and an L-Network for testing (named after their appearance to the letters T and L as seen in Figure 2.8). The T-Network takes a set of 2D-images with corresponding 3D voxel-images as input, and is composed of a ConvNet (middle vertical bar), and a classic auto-encoder (top horizontal bar). In the auto-encoder, the 3D voxel representation is taken as input and encoded into a 64D latent vector representing the compressed 3D representation. At the same time, the ConvNet in the middle part of the T-Network takes the 2D images as input and outputs a 64D vector representing the 2D compression. Using a simple Euclidean loss function comparing the two vectors, the ConvNet is optimised to ensure that the 2D-image and 3D-image representations are closely aligned. The auto-encoder also decodes the latent to generate a 3D representation, with its model simultaneously being improved using cross-entropy loss. Once optimised, the L-Network is able to take the trained ConvNet model taking a 2D-image as input, and send this straight to the decoder of the original auto-encoder (thus not requiring a 3D-voxel representation input anymore) to render a 3D reconstruction output [86]. Therefore, TL-embedding facilitates a supervised modelling framework, to which at test-time, only requires a single-view image to render a 3D object.

3D-VAE-GAN

An interesting approach adopting a volumetric output representation is the 3D-VAE-GAN [12]. As implied by the name, the 3D-VAE-GAN incorporates two architectural elements: a Variational Auto-Encoder (VAE) and a Generative Adversarial Network (GAN). The benefit of such an architecture is that it offers a generative capability that extends beyond the learnt capacity of a simple auto-encoder. It is not simply the GAN enforcing a degree of realism due to the discriminator holding the generator accountable for its outputs [28], but also the VAE itself [87], which differentiates itself from traditional AEs by its generative capability [38]. While encoders in an AE architecture typically output a fixed-sized latent representing the number of pixels, a VAE outputs two vectors: the mean and standard deviation of pixels, which is then fed into the decoder [38]. As a result, rather than deploying a learned function as in the simple auto-encoder, the decoder is able to output a 3D model that looks slightly different each time due to the implementation of a standard deviation. This has therefore been implemented by Wu et al. [12], who extended their original 3D-GAN into the 3D-VAE-GAN.

The previously mentioned 3D-GAN [12] attempts to generate a 3D voxel representation of an object by implementing the GAN architecture. The generator takes a probabilistic latent vector z as input, outputting a generated (fake) 3D representation to be used as input by the discriminator, which determines whether this generation is a real object or fake. The

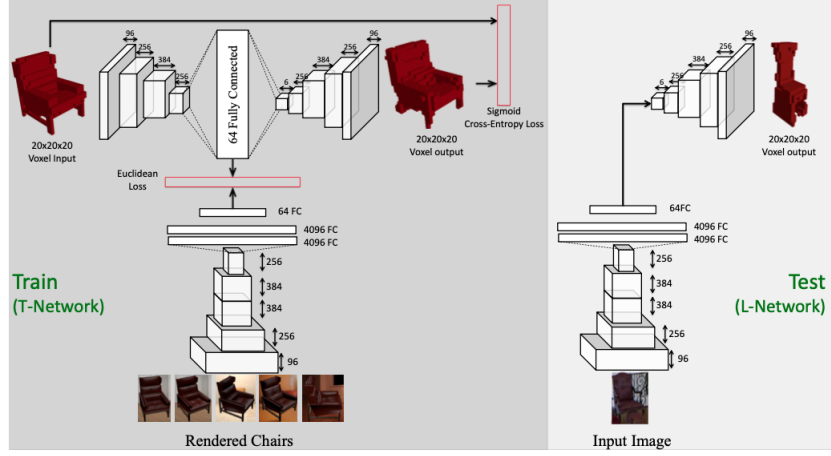


Figure 2.8: Diagram showing the TL-Embedding architecture. The T-Network is comprised of an auto-encoder in the top horizontal bar and a ConvNet in the middle vertical bar. The L-Network removes the encoder in the auto-encoder leaving the ConvNet and decoder. Figure taken from ‘Learning a Predictable and Generative Vector Representation for Objects’ by Girdhar et al. [86]

loss function used mirrors the principle seen in the GAN equation (see equation 2.1) as binary cross-entropy loss. An important question to ask is where this latent z originates from, which forms the interesting part of Wu et al.’s model [12] inspired by Larsen et al.’s VAE-GAN [88]. In essence, the VAE mentioned previously is incorporated in a manner whereby there is an encoder E taking a 2D image as input instead of a random vector z , and which encodes this into the probabilistic latent z . This z is decoded, which represents the original role of the generator mentioned in 3D-GAN. Finally, a discriminator D is involved which carries out the original functionality. Combined together, this extension is called the 3D-VAE-GAN [12]. Such a model was tested on the IKEA dataset [89], out-performing other state-of-the-art-models trained on the same dataset.

Point Completion Network

In certain cases, one may only have access to partial 3D object data. For example, while the goal of the project is to obtain a complete head, many algorithms (as will be discussed) only generate a 3D face mask, unable to extrapolate or estimate the rest of the head. To solve this issue, a potential solution was sourced in the literature surrounding Point Completion Networks [90]. The essence of this methodological field is to try to learn (primarily using supervised methods), the complete 3D representation of a single class of objects given a partial input. The original PCN architecture is shown in figure 2.9.

The PCN operates on a traditional auto-encoder architecture, where an input point cloud X is taken as input by the encoder, which learns the features in order to produce a latent vector of size k . This vector is then fed into the decoder network which produces two outputs, a coarse point-cloud output Y_{coarse} that provides a high-level reconstruction of the complete object, and a dense and detailed point-cloud output Y_{detail} which attempts

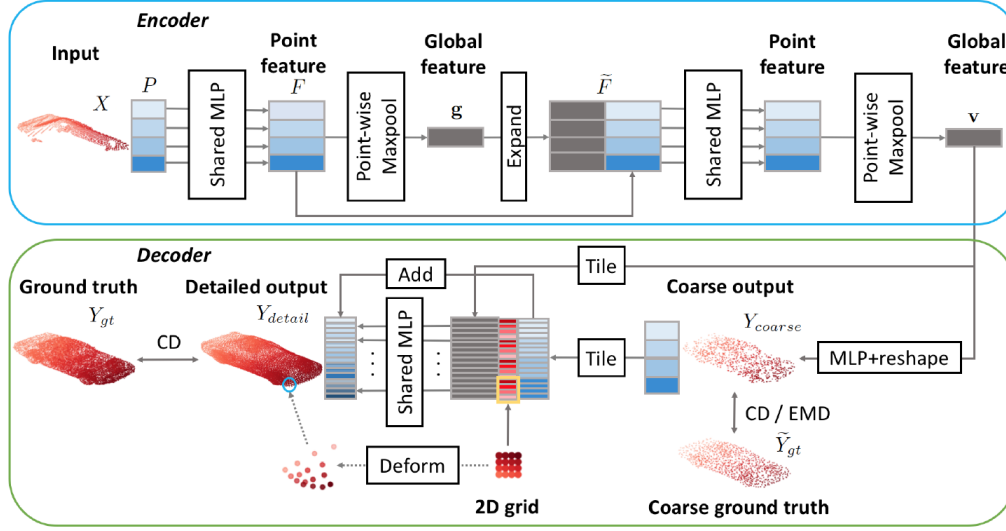


Figure 2.9: Diagram showing the PCN auto-encoder architecture [90].

to accurately capture the intricacies of the model. To facilitate learning of the model, a supervised framework involving the partial input point cloud and ground truth completed point cloud Y_{gt} are fed as data to the model, with the loss function L simply measuring the disparity in the reconstructed outputs Y_{coarse} and Y_{detail} , and the ground truth Y_{gt} and using backpropagation.

The encoder in the PCN is inspired by PointNet [91], taking advantage of its permutation-invariance, a critical factor for point cloud representations which is unordered unlike mesh representations. In the encoder of the PCN, the outputted latent feature vector $v \in \mathbb{R}^{1024}$ has a dimension of 1024, derived from stacking two PointNet layers. In the first of the layers, the input points of the partial input are passed through a Multi-Layer Perceptron (MLP) to obtain the feature matrix F composed of point feature vectors f_i . This is then passed through a point-wise max-pooling layer to obtain a global vector g . In the second PN layer, a concatenation of the point feature vectors and global vectors are first performed to obtain an "augmented point feature matrix $\tilde{F}$ " [90]. Using the same MLP, point-wise max-pooling is carried out to obtain global feature vector v which represents the outputted latent of size 1024 that the decoders use for reconstruction.

For the decoder part of the architecture, two decoders are used that hold different functions. The first is the fully connected decoder [92], which is useful for its ability to capture global shape features. The second is the folding-based decoder [93], which captures local geometry [90]. In PCN, Y_{coarse} is generated by passing the feature vector v through the fully-connected decoder, and patches for local features are generated by passing each point q_i in Y_{coarse} through the folding-based decoder which when combined, yields the output Y_{detail} . Finally, the Chamfer Distance (CD) (see equation 2.17) is used as the reconstruction loss function for model optimisation [94].

$$CD(S_1, S_2) = \frac{1}{|S_1|} \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2 + \frac{1}{|S_2|} \sum_{y \in S_2} \min_{x \in S_1} \|y - x\|_2 \quad (2.17)$$

The PCN offers clear advantages in its ability to successfully extrapolate missing parts of an object. Given the fact that so many reconstruction methods are only able to model the face mask and not the entire head, this approach provides serious potential for full 3D head reconstruction as long as a suitable dataset can be found. Building on the general architecture of the PCN, has been the *Cascaded Refinement Network for Point Cloud Completion* approach [13]. A crucial improvement of this approach compared to the original PCN is that it is designed to be able to retain the details in the input point cloud [13], while only attempting to identify the missing points. This is particularly useful for this project, since altering a perfectly well-represented face input would impact the reconstruction accuracy which would not be helpful. While the general logic of the architecture is very similar to the PCN, it defines a few crucial differences in the definitions of the decoder functionality. Furthermore, the architecture in the Cascaded Refinement Network (CRN) is actually a GAN, whereby the auto-encoder of the PCN is embedded as the generator, to which a separate discriminator is employed to facilitate realism [13]. Further details of this framework will be discussed in the later methodology and implementation.

In the point completion literature, alternate approaches have also been discussed with reference to introducing further processing steps to ensure realism. This includes the hybrid auto-completion approach by Stasinakis et al. (2022) [95], which applies the cascaded refinement network discussed prior to the field of archaeology, but introducing pre-processing and post-processing steps in-order to facilitate objects with mesh representations. It is also an approach that takes on a self-supervised approach by introducing data augmentation techniques, but is not discussed here since it is not relevant to the implementation of the project. An attempt at face data completion has also been trialled in the FinerPCN approach [96], but has still yet to have been attempted on full 3D head data. The researchers used 6144 points to complete the face, but was still deemed as insufficient in capturing full detail.

Chapter 3

Methodology

In this section of the report, the methodological process of the project is discussed. This is detailed in chronological order of experimentation, since each framework was tested in response to solving a different component of the head reconstruction problem. In the end, the 3D Face Reconstruction with FFHQ-UV approach by Bai et al. [20] was selected as the model which has the most potential for an interactive avatar application. However, the models that were also investigated included the 3D-VAE-GAN [12], the Deep 3D Face Reconstruction model [16], a novel experiment of the cascaded refinement network [13, 90, 95], and FLAME-fitting [56]. As will be shown in the evaluation, many of these approaches have quite significant limitations in their ability to model a 3D head. However, they are described here to motivate potential future research and to provide an account of the project’s learnings. While the 3D Face Reconstruction with FFHQ-UV [20] is the principal model selected for application, the other experiments are described first in favour of justifying why the FFHQ-UV framework was eventually selected. A subsection will be dedicated to each separate modelling framework in this chapter and in the following implementation chapter.

3.1 Design Considerations

Before the methodology behind the tested modelling frameworks is provided, it is important to establish some important considerations that will guide the choice of model selection. While deep-learning models are a powerful set of approaches to 3D reconstruction, they are also data-hungry, requiring high-quality data to sustain the efficacy of their outputs, which in the supervised case often involves a ground truth 3D object [26, 25]. As will be shown, full supervision does not end up leading to the more favourable outcomes in head reconstruction. Another crucial caveat to 3D deep-learning reconstruction, is the type of input data that is used. With the goal of being able to successfully generate a 3D avatar as used in a digital psychotherapy intervention, it can be assumed that the simpler the input to the application, the better. In this case, feeding single images at test-time is likely preferred to feeding multiple images from different views. Sourcing single-view 2D images is also easier than sourcing multi-view images. However, the performance of single-view 2D image to 3D reconstruction may be compromised with regards to the accuracy [38]. As posited by Fu et al. [27], challenges facing the single 2D-image to 3D object approach involve the ability of the

model to capture shapes of differing complexity, the inability for models to extrapolate 3D information that is not necessarily present (due to the single perspective), and the faithful representation of fine-grained details.

A crucial crossroad dictating the direction of this project regards the type of output representation of the 3D reconstruction. As outlined in the background review, models can generally take a volumetric or geometric approach [26], whereby the former represents the object as consisting of mini cubes packed together in a volume, whereas the latter takes a more surface-based approach to representation using vertex points in the case of point clouds, with the addition of edges in the case of meshes. While volumetric voxel-based representations can easily facilitate the 2D to 3D transition through convolutions and can flexibly be applied to complex shapes [97, 24], this comes at a considerable computational cost [98]. Resolution is especially important due to the intricacies required for facial reconstruction, and voxels ensure that increasing resolution involves cubic growth in computational complexity [97]. On the other hand, geometric representations such as point clouds and meshes offer a slightly different advantage, particularly in its improved memory-efficiency [24]. They are also explicit, making for efficient GPU-based rasterisation [31]. Despite this, they are harder to implement in deep-learning architectures [24] and in 3D-CNN which is crucial [27]. Furthermore, the irregular structure of their topologies may limit the extent to which they can capture geometric detail [24, 97]. As will be seen in the rest of the paper, these themes will consistently re-appear in model justification, design, and evaluation.

3.2 Model Experimentation

3.2.1 3D-VAE-GAN

Justification for Experimentation

As described in the background section, the 3D-VAE-GAN is an approach towards 3D reconstruction taking a 2D-image as input and outputting a 3D voxel object [12]. The initial decision to implement the 3D-VAE-GAN was motivated by the design consideration of voxel-based representations being easier to train with Convolutional Neural Networks (CNNs) compared to meshes and point clouds. Exploring whether a simple architecture could feasibly represent the complexity of a face was thus the primary objective.

Model Architecture

As defined in Wu et al. (2016) [12], the 3D-VAE-GAN adopts a modified GAN- architecture. While the discriminator remains the same in function and name, the generator is actually the decoder component of a VAE. In traditional GANs, an input latent z (typically derived from a probability distribution) is supplied to the generator as input, but in this architecture, is instead replaced by the output of an encoder E represented by a learned mean and standard deviation latent vector. This satisfies the inputs of a decoder in a VAE network. Overall, the modified GAN architecture provides a means of learning features from a 2D image and extrapolating this learning to obtain a 3D reconstruction. In summarising this architecture, the encoder takes image x as an input, applying convolution operations to return mean z_μ and standard deviation z_σ latent vectors, which are used as input to the generator (i.e. VAE decoder) to produce a 3D reconstructed object. The discriminator maintains the same functionality as in the original GAN [28], responsible for determining whether the generated

object is real or fake in comparison with the supplied ground-truth object.

Encoder						
Layer	Input Channels	Output Channels	Kernel Size	Stride	Padding	Activation
Layer 1	3	64	4	2	1	ReLU
Layer 2	64	128	4	2	1	ReLU
Layer 3	128	256	3	2	1	ReLU
Layer 4	256	512	3	2	1	ReLU
Layer 5	512	400	2	1	0	ReLU
Output	400	200	N/A	N/A	N/A	Linear (FC)

Table 3.1: Table showing the details of the 3D-VAE-GAN encoder

The network details of the encoder are explained in the table above. Each layer features a 2D convolution to downsample the image (initially starts with 3 channels (i.e. RGB), followed by 2d batch normalisation (apart from the final output layer), followed by the activation function. In the output layer, there are actually two outputs returned: a mean and standard deviation vector that are each 200-dimensional representations. This works accordingly with the structure of the VAE. In the below tables, the generator (i.e. decoder) and discriminator are outlined. As shown by the input channel of the generator, this takes in a latent vector of size 200, corresponding to the size of the mean and standard deviation latent vectors which are concatenated. In each layer of the generator, 3D transposed convolution is applied to upsample the latent to the 3D fake output, followed by 3D batch normalisation and activation. Note that in the final layer, the Sigmoid activation function is used since an output channel of 1 is associated with the 3D generation. The discriminator network takes the generated output, attempting to determine its legitimacy, with each layer adopting 3D Convolution followed by batch normalisation and LeakyReLU activation (apart from the final layer). In the final layer, transposed convolution is applied to return the number of output channels to 1, and with the Sigmoid activation function, attempts to determine whether the generated output is real or fake.

Generator (Decoder)						
Layer	Input Channels	Output Channels	Kernel Size	Stride	Padding	Activation
Layer 1	200	512	4	2	0	ReLU
Layer 2	512	256	4	2	1	ReLU
Layer 3	256	128	4	2	1	ReLU
Layer 4	128	64	4	2	1	ReLU
Layer 5	64	1	4	2	1	Sigmoid
Output	N/A	1	N/A	N/A	N/A	N/A

Table 3.2: Table showing the details of the 3D-VAE-GAN generator.

Data Preparation

A suitable dataset involving the supervised pairing of a 2D-image with a 3D-voxel head/face object could not be found, such that data processing steps were carried out to convert the mesh .obj files into voxel format. As an initial means of experimentation, the data used for training was a subset of the results folder in the 3DFace GitHub repository of [Juyong \[99\]](#), featuring images and their .obj file pairs. The dataset was initially selected simply as a means of trialling the generation of a 3D output from using single-view image. However, from the actual implementation of the data preparation (see next chapter), it was determined

Discriminator						
Layer	Input Channels	Output Channels	Kernel Size	Stride	Padding	Activation
Layer 1	1	64	4	2	1	LeakyReLU(0.2)
Layer 2	64	128	4	2	1	LeakyReLU(0.2)
Layer 3	128	256	3	2	1	LeakyReLU(0.2)
Layer 4	256	512	3	2	1	LeakyReLU(0.2)
Layer 5	512	1	4	2	0	Sigmoid
Output	N/A	1	N/A	N/A	N/A	N/A

Table 3.3: Table showing the details of the 3D-VAE-GAN discriminator

that this approach would not be feasible for the intended application.

Model Training and Testing

The loss functions used to train the 3D-VAE-GAN in Wu et al. (2016) are comprised of three principal components: a "reconstruction loss L_{recon} , a cross-entropy loss L_{3D-GAN} , and a Kullback-Leibler (KL) divergence loss L_{KL} " [12]. The objective loss function is therefore defined below, with a_1 and a_2 representing the weights that are applied to the KL divergence and reconstruction loss respectively.

$$L = L_{3D-GAN} + \alpha_1 L_{KL} + \alpha_2 L_{recon} \quad (3.1)$$

The L_{3D-GAN} loss is the same as the binary GAN loss function defined in equation 2.1. The KL divergence loss is commonly implemented in VAEs [88], and provides a means of ensuring that the generator taking the latent vector z (i.e. the variational distribution $q(z|y)$) does not stray too far from the prior multivariate Gaussian distribution $p(z)$ used to inform the encoder [12]. Finally, the reconstruction loss essentially aims to measure the loss between the generated 3D object $G(E(y))$ from 2D-image y and the ground-truth 3D object x .

$$L_{3D-GAN} = \log D(x) + \log(1 - D(G(z))) \quad (3.2)$$

$$L_{KL} = D_{KL}(q(z|y)||p(z)) \quad (3.3)$$

$$L_{recon} = ||G(E(y)) - x||_2 \quad (3.4)$$

While solely experimental, the model was trained on an epoch size = 100, batch size = 32, and learning rate = 3e-4, optimised using ADAM [100]. The encoder, decoder (i.e. generator), and discriminator performed backpropagation at each step, with the encoder employing a loss function calculated from summing the KL-divergence and reconstruction losses, the decoder loss involving the summation of the adversarial loss from L_{3D-GAN} and reconstruction loss, and the discriminator loss using the adversarial loss in L_{3D-GAN} . The test function involved stripping away the learning component of the train function, directly measuring the reconstruction loss of the 3D object generated from an input image with the corresponding ground truth 3D object.

3.2.2 Deep 3D Face Reconstruction

Justification for Experimentation

The Deep 3D Face Reconstruction model by Deng et al. [16] (referred to in the background review) was directly replicated with some scripts added to facilitate smoother functionality. The original GitHub repository can be sourced [here](#). Having initially trialled a non-domain-specific model for reconstruction as in the 3D-VAE-GAN [12], the motivation behind trialling this model was because it incorporates 3DMMs in its design, which had yet to have been explored. Furthermore, the project was yet to validate the efficacy of mesh output representations, with this implementation carried out to determine the quality of rendering.

Data Preparation

Since this is a direct replication of the original code repository with pre-trained models, the only data preparation involved ensuring that the custom images supplied for rendering are in readable format. This was not supplied in the original repository, and is thus defined additionally in this implementation. This includes removing EXIF metadata from images that would cause rotation issues affecting the face detection algorithm, and generating a basic 5-point landmark detection file for each image, which returns the 2D-coordinates representing the left eye, right eye, nose, left mouth edge, and right mouth edge. For implementation details, please refer to the next chapter.

Preparing the Pre-Trained Models

A number of pre-trained models needed to be downloaded prior to running the fitting on custom images. As has been described in the background literature, Principal Component Analysis (PCA) is a crucial part of defining how morphable models work. In this implementation, the 2009 Basel Face Model was used to capture shape and texture bases [18], while expression was captured in the `Exp_Pca.bin` file (shown [here](#)) [99]. The trained model from the original paper was also provided for running the 3DMM fitting [16].

Rendering the Custom Images

After the data was prepared and pre-trained models placed into the relevant folders (as described in the reproduction repository [here](#)), the test script can be run to start the fitting. An example of the mask fitting process is shown in figure 3.1, which removes any object occlusions and creates a face mask with shape optimised using the landmark detection algorithm. The fitting results include a `.obj` file of the face mask combining the shape and texture in one file with per-vertex RGB values, and a `.mat` file with the coefficient vectors for the morphable model.

3.2.3 A Pipeline for Head Model Reconstruction Using the Cascaded Refinement Network

Justification for Experimentation

While the Deep 3D Face Reconstruction model produced solid reconstruction outputs with regards to rendering a coherent 3D face mask with texture mapping, it was still unable to achieve a fully reconstructed head. As mentioned in the design considerations, this limitation is derived from the difficulty of using single-view images to extrapolate areas of the head that



Figure 3.1: The 3DMM Fitting in the Deep 3D Face Reconstruction Model [16]

cannot be seen in the image, requiring a suitable means of estimation. To solve this issue, a potential solution was sourced in the literature surrounding Point Completion Networks (PCNs) [90]. The essence behind the PCN approach is to try to learn (primarily using supervised methods), the complete 3D point cloud representation of a single class of objects given a partial input point cloud. Since the Deep 3D Face Reconstruction framework was able to generate 3D face masks, the motivation behind implementing this approach was to implement a supervised framework using the face mask .obj files as partial input. The supervised nature would be enabled by the Headspace dataset [57], providing subject images with a corresponding full 3D .obj head scan. By applying data processing steps on this dataset, it was hoped that the first point completion network specifically modelling an entire 3D head could be successfully implemented.

Model Architecture

The model architecture follows the *Cascaded Refinement Network for Point Cloud Completion* by Wang et al. [13], and implements the [original code](#) from scratch. The Cascaded Refinement Network (CRN) is directly inspired by the pioneering Point Completion Network (PCN) by Yuan et al. [90], which is described in the background review. The CRN architecture retains the auto-encoder defined in the PCN [90], but extends this architecture to a GAN-framework [28] by modelling the original auto-encoder as being part of the generator, and introducing a separate discriminator for enforcing further realism in reconstruction.

As shown in figure 3.2, the generator consists of three broad components: feature extraction, coarse reconstruction and dense reconstruction [13]. The input point cloud P is firstly passed into the feature extractor, which consists of stacking two PointNet layers [90] and applying max-pooling to return a global feature vector f . This essentially represents the encoder component of the generator. The subsequent steps are responsible for the decoding functionality of the generator (i.e. generating the reconstruction outputs). In the coarse reconstruction phase g_1 , the application of some fully-connected layers to f is used to obtain a rough representation of the completed object, returning a coarse point cloud P_{coarse} with size $N_c \times 3$. This is followed by the dense reconstruction module g_2 which adds considerably more complexity. In g_2 , the goal is two-fold: firstly to ensure that the identity of the original partial input is preserved as faithfully as possible, and secondly, to use the rough global representation obtained in g_1 to combine both elements in ensuring accuracy of reconstruction. A sub-sampled point set from the partial input P is calculated using the

farthest point sampling algorithm [101], and concatenated onto P_{coarse} in a fashion inspired by skip-connections in U-Net [39], resulting in the combined point set P_s with a size of $2N_c \times 3$ [13]. g_2 actually represents a "conditional iterative refinement sub-network" [13], since P_s is fed into a "lifting module", which includes a feature contraction-expansion unit aimed at iteratively refining the set of points and their positions and up-sampling them by factor 2 to improve the reconstruction accuracy [13]. The result of passing through the lifting module is the dense (fine) reconstruction output point cloud P_{fine} .

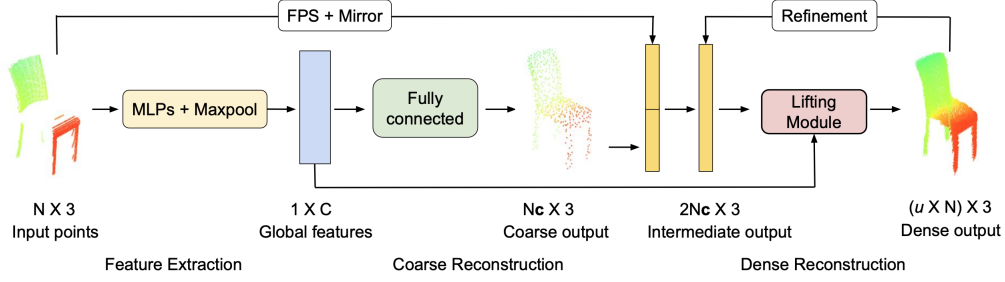


Figure 3.2: Diagram showing the generator network of the Cascaded Refinement Network by Wang et al. [13]

Figure 3.3 shows the components of the lifting module. Taking the combined point set P_s from the previous part of the generator network, the up-sampling is achieved by tiling P_s twice to obtain P'_s and applying a unique 2D grid vector to the point set, which essentially introduces some variation to P'_s [102]. While in the original paper, a mean shape prior f_m was also implemented, this calculation was not possible for the face data that was used and thus omitted. As shown in the architecture diagram, f_s is a resultant feature vector derived from concatenating P'_s , f_m , f , and the 2D grid vector (in this case omitting f_m). The aim of the contraction-expansion is "to predict per-vertex displacements $\{d_x, d_y, d_z\}$ for each point P'_s given the point feature f_s " [13]. The unit works by passing f_s into a contraction Multi-Layer Perceptron (MLP) network and reshaped, resulting in the feature vector f_e , which goes through an expansion MLP with reshaping to obtain feature vector f_e . Finally, element-wise addition is performed between the output of the contraction-expansion unit and an MLP applied to f_s earlier, which is then passed through another MLP with element-wise addition performed on that output and the original f_s . The refined point set can be summarised in equation 3.5, which comprises the addition of the vertex displacement in $F(\cdot)$ to the point set P'_S . This is a direct result of the lifting module predicting the residuals of the points [13].

$$P_i = F(P'_S) + P'_S \quad (3.5)$$

The discriminator network shown in figure 3.4 deviates from the traditional GAN [28], by considering the local features as opposed to one global feature [13]. To implement this design, the hierarchical feature learning method defined in PointNet++ [101] was used as the "patch discriminator", with random points being selected using FPS to obtain N_s points, and radii $r = \{0.1, 0.2, 0.4\}$ being used to adjust the size of the patch or local area that a specific point

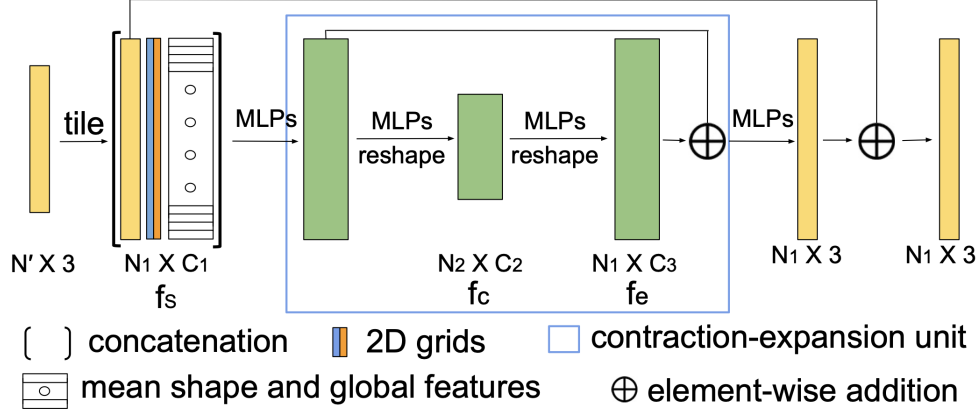


Figure 3.3: Diagram showing the lifting module in the decoder component of the Cascaded Refinement Network by Wang et al. [13]

was tested on. With the collection of different patch sizes, a MLP network was applied on each patch to learn and extract feature vectors, which were combined together to aggregate different levels of granularity in the model. Finally, these were passed through a final MLP in the value regression phase in order to obtain a vector of size $N_s \times 1$ dictating the realism of each patch [13].

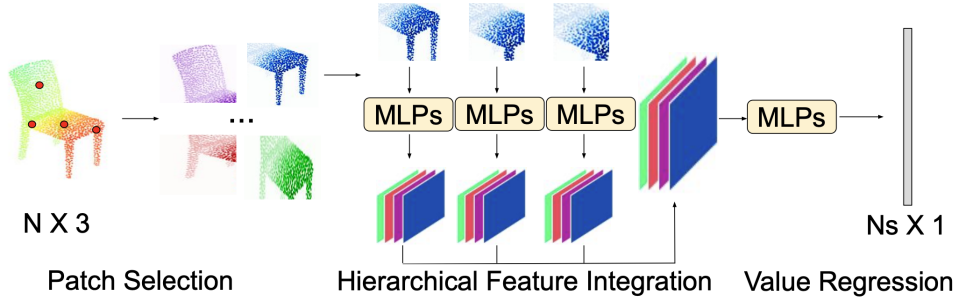


Figure 3.4: Diagram showing the patch discriminator network in the Cascaded Refinement Network by Wang et al. [13].

Data Preparation

While the literature on point completion has received some success on daily-life object datasets such as ShapeNet [85], it has not been applied to entire 3D heads despite having been applied to face masks in FinerPCN [96]. One reason explaining this lack of research may be because sourcing extensive 3D head point cloud data is difficult, meaning PCN architectures often cannot take on the full supervision required. However, in this approach, a method for facilitate this supervision is proposed by applying an intricate data preparation pipeline described below.

The data for the model is sourced from the Headspace dataset in the Liverpool-York Head Model (LYHM) project [57]. This dataset comprises two sub-datasets needed for analysis. The first is the complete 3D head model dataset featuring $n = 1519$ subjects with each subject having a folder comprising the full `.obj` head model, a `.bmp` texture file, a corresponding `.mtl` file for mapping the texture to the `.obj` mesh, and a `.txt` file detailing various qualitative details and demographics about the subject. This forms the ground truth for supervision. In the second dataset, the corresponding pictures and camera calibration files for each subject are provided with differing perspectives. To note, only $n = 1212$ subjects have full picture data that can be used. As will be described next, these images are crucial to obtaining the partial input face masks serving as correspondence to the ground truth. The Headspace dataset therefore satisfies the two things necessary for the model architecture: a 2D-image that can be converted to a partial `.obj` face mask, and a full 3D-head `.obj` file.

The reason these mesh files are sufficient, is because they can be converted into the point cloud representation needed for the model architecture. To convert the 2D-images from the Headspace images dataset to a 3D representation in point cloud format, a multi-step approach was implemented. Firstly, the images for the subjects were chosen, where it was found that the researchers had coded for image `2C` in each subject’s folder to represent a front-facing view with a slight angle [57]. Though this was not perfect, testing from the Deep 3D Face Reconstruction model [16] showed that straight-on face images were not strictly necessary to render the partial input face mask. As such, the model was deployed on each subject’s `2C` image to create a partial `.obj` face mesh and compiled into a partial inputs dataset. However, `.obj` is still not the desired output format for the modelling architecture, which requires point cloud representation.

The next step of this approach was therefore to convert the `.obj` file into a point cloud. This step was achieved by taking inspiration from Stasinakis et al. [95], first sampling 10000 points using Poisson Disk Sampling [103] since a training resolution of 2048 points was selected for this project as an initial trial. This was then followed by Farthest-Point Sampling to downsample to the 2048-point target resolution [104]. Finally, these points were normalised in the unit sphere [95]. As detailed in the implementation, this converts the mesh to the point cloud format `.pcd`. Since point clouds are unordered structures, these were then converted to numpy arrays as `.npy` files for easier use and adoption in the deep-learning architecture. Since the ground-truth (full 3D-Head) files from the Headspace dataset were already in `.obj` format, these also followed the same procedure to convert them into corresponding ground truth `.npy` files for training and testing. The final step was to load the partial input and ground truth `.npy` files into training, validation, and test sets.

Model Training and Testing

The model optimisation is carried out according to two priorities: to ensure accurate reconstruction, and to ensure realistic reconstruction. The former is handled through defining a reconstruction loss, which given the nature of point clouds as discussed in the PCN literature, invites the adoption of Chamfer Distance (CD) as the metric [90, 94]. In the CRN [13], the CD is square-rooted and divided by 2.

$$CD(X, Y) = L_{X,Y} + L_{Y,X}, \text{ where} \quad (3.6)$$

$$L_{X,Y} = \frac{1}{|X|} \sum_{x \in X} \min_{y \in Y} \|x - y\|_2 \quad (3.7)$$

$$L_{Y,X} = \frac{1}{|Y|} \sum_{y \in Y} \min_{x \in X} \|x - y\|_2 \quad (3.8)$$

The overall reconstruction loss is defined in equation 3.9, whereby the Chamfer Distance calculated on the coarse reconstruction to ground truth, and the fine reconstruction to ground truth is summed, with λ_f representing a reconstruction weight placed on P_{fine} , placing more importance on the fine reconstruction.

$$L_{rec} = CD(P_{coarse}, Q) + \lambda_f CD(P_{fine}, Q) \quad (3.9)$$

The second optimisation is based on rendering realism, achieved through the adversarial loss as characteristic of GAN architectures. The LS-GAN is adopted [105], defined in the below equations taken from [13], with x being the ground truth, and $\tilde{x}$ being the fake generation.

$$L_{GAN}(G) = \frac{1}{2} [D(\tilde{x}) - 1]^2 \quad (3.10)$$

$$L_{GAN}(D) = \frac{1}{2} [D(\tilde{x})^2 + (D(x) - 1)^2] \quad (3.11)$$

The reconstruction and adversarial loss are combined into one objective loss function in 3.12, where λ and β are weights applied to each loss component.

$$L = \lambda L_{GAN} + \beta L_{rec} \quad (3.12)$$

The model is trained using the parameters: epochs=1000, batch size=32, generator and discriminator learning rate=1e-4, step_ratio=2, and input and output number of points=2048. The results of training yields a `.pth` file representing the trained weights, which are used for testing purposes by being loaded into a generator to test the CD metric between the generated fine point cloud and its corresponding ground truth point cloud.

Reconstruction Rendering

With the trained model, renderings can be carried out on new point cloud samples. After inputting the target point cloud into the generator, the result will return the 2048-point generation as a 3d NumPy array. Adopting the methodology of Stasinakis et al. [95], the numpy array is returned to a `.pcd` file, which is returned to an `.obj` file for visualisation in MeshLab, using Poisson reconstruction [106]. The reason for conversion back to mesh as opposed to keeping the point cloud, is down to the fact that faces represent smooth surfaces and are highly detailed, which mesh representations capture better.

3.2.4 FLAME Reproduction

Justification for Experimentation

The source code for the FLAME head model has been published by the researchers from the original FLAME paper [56], which can be found [here](#). The motivation behind experimenting with FLAME was because of its ability to reconstruct an entire 3D head model from a single 2D image, which had not been successful when working with the other experimented approaches. FLAME demonstrates an ability to work with changing facial expressions, shown in an impressive interactive application [here](#). This approach was a direct replication of the original repository, aiming to determine the efficacy of the FLAME topology and its potential use in the digital psychotherapy intervention.

Data Preparation

Since the models required for FLAME have already been trained, the only data needing preparation are the custom images supplied to the reconstruction pipeline. This preparation comes in two forms: an image that allows easy face detection and removes noisy metadata (which happened to be a problem when taken from iPhone images), and a corresponding 51-point landmark file to go with the image. The latter issue is discussed in the pipeline section below and in the implementation chapter, requiring the download of the 68-point landmark detection algorithm.

Alongside the custom images, pre-trained models from [FLAME](#) need to be downloaded, with details of where to download the models described in the project’s GitHub repository [here](#). These include FLAME models trained specifically for male, female, or general cases to more accurately represent shape, pose, and expression intricacies, as well as embeddings to better map the vertices of the 3D mesh onto the calculated landmarks files. Texture mapping files of different resolutions are also provided to allow a user to specify the detail of texture applied to the 3D mesh, which in this case, involves overlaying the input test image onto the calculated FLAME head.

Pipeline

As an extension of the original code repository, a pipeline is defined to streamline the image to head reconstruction process. As mentioned, the input images first need to be processed, which involves first detecting a singular face from the image and resizing the image to include just the head itself. This is carried out to remove any potential occlusions in the image, and to enable easier landmark detection. The second part of the processing involves detecting the necessary landmarks from the image. From inspecting the sample landmarks, it was found that the FLAME repository requires a corresponding 51-point landmark representation (see figure 3.5), to which a script was not provided for this task. This was therefore implemented in this project, utilising a 68-point landmark file to detect 68 landmarks, and manually filtered to match the 51-point representation in the image below. This is not a difficult task, as the same ordering of indices are used for the landmarks in both the 51-point and 68-point detection. In the 68-point version, the first 17 indices are represented by the jaw, such that these were removed from the calculation to yield 51-points.

After the image is processed, the next stage of the pipeline is to fit the FLAME model to the corresponding 2D landmarks. The idea behind this stage is to optimise the parameters (shape, pose, expression, scale, translation, and rotation) that guide the 2D landmark to 3D FLAME head model reconstruction by minimising a landmark and objective loss function.

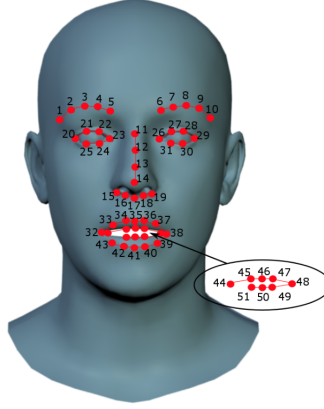


Figure 3.5: Diagram of the 51-point landmark representation taken from the [FLAME repository](#) [56]

In the first stage, the model is solely focused on minimising the landmark distance, which is calculated by comparing the distance between the 2D landmarks projected from the estimated 3D FLAME model using the parameters to the original ground truth 2D landmark. In the first stage, only the scale, translation and rotation parameters are updated. In the second stage, all parameters are updated, using an objective loss function involving the landmark distance, regularised neck pose, jaw pose, eyeballs pose, shape and expression with the values of 1.0, 100.0, 1e-3, 10.0, 1e-3, and 1e-3 respectively. These are trained using the L-BFGS-B optimisation algorithm [107]. The results of this step yield the `.obj` file and `.mtl` file renders for the original image.

The final step in the pipeline is to fit the outputted mesh to full FLAME topology, which provides flexibility for potential integration elsewhere such as in the FFHQ-UV repository discussed later. In this step, the optimisation process is actually really similar to that described in the previous step, except the comparison is no longer between 2D landmarks, but between 3D meshes. Specifically, the parameters to optimise remain the same just without the scale (e.g. shape, pose, expression, translation and rotation), while this time, weights are applied to data, regularised neck pose, jaw pose, eyeballs pose, shape and expression with the values of 1000, 1e-4, 1e-4, 1e-4, 1e-4, and 1e-4 respectively. In the first step of optimisation, the loss function is the mesh distance calculated between the mesh of the FLAME model (to be fit to the target mesh), and the target mesh which is the mesh that was obtained from the previous step. In this stage, only the translation and rotation vectors are optimised. In the second stage, all parameters are optimised, using the objective loss function which involves the application of the regularisation weights to the loss components. These are all optimised again using L-BFGS-B algorithm [107], returning a `.ply` file, with a corresponding implementation provided to obtain the `.obj` file.

3.3 3D Face Reconstruction with FFHQ-UV

3.3.1 Justification for Implementation

By building an understanding of the limitations behind the experimented models detailed thus far, it was the FFHQ-UV-based 3D reconstruction algorithm that achieved the greatest success in holistically dealing with the aforementioned problems [20]. As shown in the original paper, the greatest advantage of this model is in its ability to create a UV-texture map that wraps around the entire head with little distortion, and in a reliable manner. This is a particularly difficult problem, but can be achieved even just by supplying an "in-the-wild" single-view 2D image. Therefore, the complexity of image input is not a limiting factor to the quality of the render, which indicates high potential for the viability of this model for the intended application. Furthermore, it also utilises the 3DMM-framework and generates not simply the reconstructed model, but also the calculated 3DMM coefficient vectors, giving the model flexibility in modelling changes in shape and expression. This implementation was thus carried out to verify the reconstruction outputs of the original paper [20].

3.3.2 Reconstruction Algorithm

To facilitate the reconstruction, a GAN-Based Texture Decoder was trained on the texture map dataset generated from the FFHQ-UV pipeline using the StyleGAN2 architecture [78]. This is loaded as a pre-trained model, with the capacity to generate a texture map $\hat{T} = G_{tex}(z)$, whereby G_{tex} represents the texture decoder (a generator function), and z is the latent vector. In the reconstruction framework detailed, optimisation occurs to find the latent vector z^* that renders the best texture map $\hat{T}$.

To reconstruct the output, a fitting process consisting of three stages iteratively refines the quality of the shape model and texture. These are "linear 3DMM initialisation, texture latent code z optimisation, and joint parameter optimisation" [20]. In the first stage, the Deep 3D Face Reconstruction model by Deng et al. [16] is used, which returns the HiFi3D++ topology [22]. As described in the background literature, the predicted parameters of this model are $\{p_{id}, p_{exp}, p_{tex}, p_{pose}, p_{light}\}$, referring to the identity, expression, texture, head pose, and lighting conditions respectively. These parameters are crucial, and are optimised during the fitting process. In the second stage, the z code of the texture decoder is optimised by fixing $\{p_{id}, p_{exp}, p_{tex}, p_{pose}\}$ and minimising the function in equation 3.13 when comparing the input image I^{in} and the rendered estimate I^{re} . The function incorporates the LPIPS distance and the pixel-wise L_2 photometric loss between I^{in} and I^{re} . L_{id} is the "identity loss based on the final layer feature vector of Arcface" [20, 108]. Finally, L_{reg}^z is the regularisation of vector z . The motivation behind fixing the parameters is because this second stage is only really concerned with optimising the texture decoder latent.

$$L_{s2} = \lambda_{lpips} L_{lpips} + \lambda_{pix} L_{pix} + \lambda_{id} L_{id} + \lambda_{reg}^z L_{reg}^z \quad (3.13)$$

The final step of the training is to jointly optimise all parameters as shown in equation 3.14. Additional loss terms have been added in the landmark loss L_{lm} , and the identity and expression regularisation losses in L_{reg}^{id} and L_{reg}^{exp} respectively [20].

$$L_{s3} = \lambda_{pix} L_{pix} + \lambda_{id} L_{id} + \lambda_{reg}^z L_{reg}^z + \lambda_{lm} L_{lm} + \lambda_{reg}^{id} L_{reg}^{id} + \lambda_{reg}^{exp} L_{reg}^{exp} \quad (3.14)$$

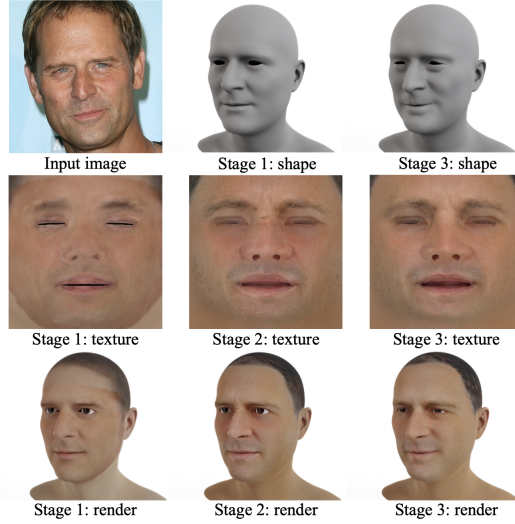


Figure 3.6: Diagram showing the fitting process in the 3D Face Reconstruction with FFHQ-UV approach. Image taken from the original paper [20].

3.3.3 Additions to the Rendering Pipeline

To facilitate greater ease of use of the pipeline, a few modifications and additions were made to the original code. The first addition regarded further image processing prior to runtime. While images supplied to the pipeline will in the general case work without issue, some images come with metadata that can apply rotations to the images thereby affecting the ability for the algorithm to detect faces. Furthermore, the size of the image can also affect face detection. Using a robust Haar Cascade Classifier [109], images can optionally be processed whereby the algorithm will detect a face and crop the image around the face to ensure greater robustness when run at test time.

Another modification to the pipeline was the direct integration of eyeball rendering. While this is supplied in the original code, it was not immediately obvious how to integrate these with the reconstructed heads. A small addition was thus made to the shell script for running the pipeline. Finally, for viewing the reconstruction in Blender, an automation script (described in the implementation chapter) was written to directly process the mesh files with textures together, allowing for greater flexibility when applied to different applications.

3.4 Extension: Trialling an Interactive Application for Dynamic Facial Expression Change

3.4.1 Justification

Taking inspiration from the FLAME interactive viewer (try [here](#)), the project desired to investigate if the rendered head model from the 3D Face Reconstruction with FFHQ-UV could also be applied to dynamically model expression change. This was hypothesised to work based on 3D Morphable Model theory (see 2.2.1), since the FLAME topology [56] and HIFI3D++

[22] used in the FFHQ-UV framework both originate from the same theoretical line of work using Principle Component Analysis (PCA). Specifically, this extension attempted to implement the same interactive application but developed specifically for the HIFI3D++ topology. Furthermore, a particular area of added value lies in this implementation being the first to assign the coefficient bases in HiFi3D++ to the six basic emotions [23]. This was therefore a novel addition to the research field.

3.4.2 Application Concept and Design

The overall concept of the application involves a head mesh without texture being able to change facial expressions according to the demands of the user. While in the figure above, this is achieved by using slider scales, it is not necessarily practical since the bases calculated in PCA are arbitrary, not assigning any specific semantic meaning to them. Having a way to assign emotions to these bases or understanding how different combinations of the bases can lead to certain expressions is exceedingly useful, particularly for a potential avatar-based application which needs to implement robust values for certain emotions. In this project, the application mirrors the format shown above, but instead of using arbitrary sliders, implements toggles for the six basic emotions after trial-and-error. This allows the user to change the emotions of the head mesh. Note that while a default head mesh is supplied in the current development of the application, this can also easily be replaced by the head mesh of a subject if desired.

The design of the application follows a basic frontend-backend architecture. The frontend utilises Unity for the Graphical User Interface (GUI), while the backend is developed in Python, and incorporates the logic from the FFHQ-UV reconstruction. This is managed through a series of requests from the user on the GUI to the Python server, which executes the logic in the backend and sends the information back to the frontend to update the changes on the GUI. While the frontend logic is fairly intuitive and is described in more detail in the implementation chapter, the backend logic simply takes the rendering code used to reconstruct the 3D model in FFHQ-UV [20] and applies changes to the expression coefficients in the model coefficients file and renders that into a new .obj head mesh to be implemented on the GUI.

Chapter 4

Implementation

In this section, the technical implementations behind the experimented frameworks are described. The code for the entire project is published in [this](#) centralised repository, organised with git sub-modules for each implementation. Since much of this project involves direct replication of existing repositories, if the code is taken from the original repository, it will be made clear at the top of the script. To better understand the motivations behind the technical implementation, please refer to the [methodology](#) in the previous chapter. The following chapter sections will be organised in the order with which they were described in the methodology.

4.1 3D-VAE-GAN

The source code for the 3D-VAE-GAN [\[12\]](#) developed in this project can be found [here](#), and was built from scratch according to the original research paper by Wu et al. (2016) in "Learning a Probabilistic Latent Space of Object Shapes via 3D Generative-Adversarial Modeling". Credit is also given to the reproduction of 3D-VAE-GAN by Bryon Kucharski, found [here](#).

Dependencies

The implementation of the 3D-VAE-GAN did not require a complex library to run, and was developed locally without GPU solely for experimentation purposes. The framework was developed in PyTorch 2.x, with additional dependencies in [NumPy](#), [SciPy](#) and [Matplotlib](#). As will be detailed in the subsequent data processing section, a couple of external software packages were required to facilitate the transfer of the mesh files to voxel representations.

Data Processing

Since voxels are used as the output representation in the 3D-VAE-GAN [\[12\]](#), a suitable method of conversion was required to translate the face `.obj` mesh in the [dataset](#) to a binary voxel grid, which reconstructs volumes by indicating whether a voxel exists or not in a grid spot. A number of Python packages were explored to carry out this conversion such as [Pyvista](#), [Trimesh](#), and [Open3d](#). However, after extensive research, a somewhat gold standard approach was sourced in the use of Patrick Min's work, which can be found [here](#):

<https://www.patrickmin.com/binvox/>. Min developed a command line tool to handle this conversion, supporting operations such as specifying the dimensions of the voxel grid and rotation of the voxelised representation. It also supports multiple 3D input file types including `.obj`, `.stl`, and `.ply`, writing the conversion output to the `.binvox` file format. Min also developed a package called `viewvox`, taking the `.binvox` file as input and producing a visualisation of the output. Both the `binvox` and `viewvox` command line tools were downloaded according to the instructions on the [website](#).

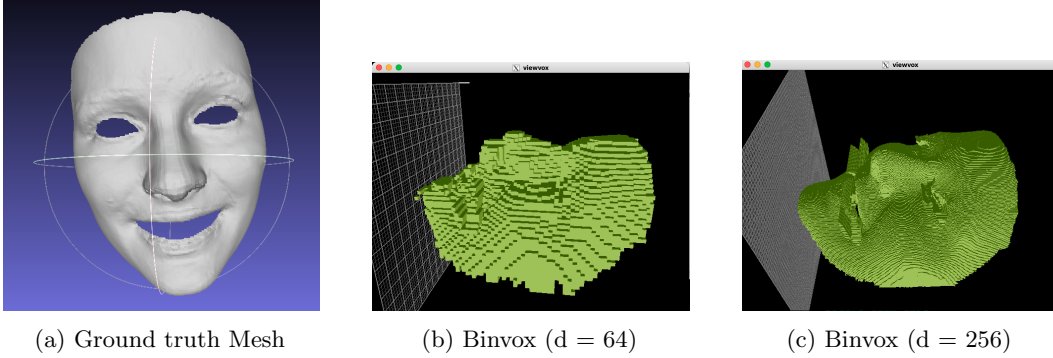


Figure 4.1: (a) represents the ground truth `.obj` mesh rendered in MeshLab. (b) is the binary voxel representation with dimensions of 64^3 viewed using `viewvox`, and (c) is the same, but with dimensions of 256^3 . Note that (a) and (b) are rotated for the purposes of demonstrating the achieved depth output.

To process the image and `.obj` mesh dataset, a `binvox.sh` shell script was written to convert the mesh files to `.binvox` file format with 64^3 dimensions using the `binvox` command line tool. Since the `.binvox` file format cannot be read by Python, the final step to processing this output for insertion into the 3D-VAE-GAN architecture was to convert the `.binvox` file into a 64^3 3D numpy array. This was achieved by using Daniel Maturana’s `binvox-rw-py` package, which can be found in this [link](#). After converting all `.binvox` files to numpy arrays in `binvox2NpArray.py`, the data for the image inputs and outputs were matched by name and split into training and test sets. The data loading function called during training and testing was then implemented by importing `Dataset` from `torch.utils.data` in the `DataLoader` class in `DataLoader.py`. In this script, the input images corresponding to the ground truth voxel numpy arrays are resized to the dimensions 28×28 and converted to numpy arrays. These are then converted to tensors to be used for batch processing during model runs, with `DataLoader` returning a float tensor for the image, and a boolean tensor for the binary 3D numpy array.

Model Implementation

After the data was prepared, the model architecture described in the methodology was implemented in `model.py`. Creating separate classes for the encoder, generator, and discriminator, `nn.Module` from `torch.nn` was used to facilitate the application of `nn.Sequential` in the forward function, and the various 2D and 3D convolution operations (e.g. `torch.nn.Conv2d` and `torch.nn.ConvTranspose3d`), batch normalisation, and activation functions.

For training, the dataset was loaded using `DataLoader` in `torch.utils.data`, and the

encoder, generator and discriminator constructors were called, with the Adam optimiser initialised using `optim.Adam`. The rest of the training function followed the training flow as described in the 3D-VAE-GAN methodology (see 3.2.1), with the discriminator using the binary cross entropy loss in `nn.BCELoss()`. The test function operates similarly, but was implemented to pass the test image through the generator only, calculating the reconstruction loss when compared to the ground truth voxel representation. Finally, the `SaveFloat_Voxels` function defined by Bryon Kucharski in his [implementation](#) was used for this project, responsible for saving the test render voxel output as a pickle file and visualising it.

4.2 Deep 3D Face Reconstruction

Since this is a direct replication of the [original repository](#), only a few scripts were added in this project's [code](#), with the purpose of simply exploring the application of the 3DMM. During the process of reproduction, a number of setup issues and steps were encountered that were not fully documented in the original repository, but are documented here. Note that this reproduction is only concerned with building the pipeline for testing custom images, and not re-training the model.

Dependencies

The code for loading the dependencies are taken from the original repository [here](#). To install the dependencies, the `environment.yml` file needed to be called in an [Anaconda](#) environment, which automatically created an environment called `deep3d_pytorch` using Python 3.6. Such an early version is used since the original repository used PyTorch and TensorFlow version 1.

```
1 conda env create -f environment.yml
2 conda activate deep3d_pytorch
```

Listing 4.1: Create Environment.

The [nvdiffrast](#) and [ArcFace](#) [108] library also required installation from source. The former is a library that is particularly useful in the 3D reconstruction literature for its differential rendering capabilities, and the latter is a powerful loss function designed to help distinguish one face from another, which is useful given the nature of 3DMMs discussed in the background literature (see chapter 2). Finally, the `dlib` and `MTCNN` packages required installation. These packages form the extended functionality added by this project to the original repository, as will be discussed in the next part.

```
1 # Install nvdiffrast
2 git clone https://github.com/NVlabs/nvdiffrast
3 cd nvdiffrast
4 pip install .
5 # Install ArcFace
6 cd .. # ./Deep3DFaceRecon_pytorch
7 git clone https://github.com/deepinsight/insightface.git
8 cp -r ./insightface/recognition/arcface_torch ./models/
```

Listing 4.2: Nvdiffrast and ArcFace Configuration.

Finally, CUDA compatibility was implemented using CUDA 10.0.130 with CUDNN 7.6.4.38, as this is a specific TensorFlow 1.x requirement. This was executed by modifying the paths in the `.bashrc` file.

```
1 export CUDA_HOME=/vol/cuda/10.0.130-cudnn7.6.4.38
2 export PATH=$CUDA_HOME/bin:$PATH
3 export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
```

Listing 4.3: Configure CUDA

During runtime, a compiler error was raised regarding GCC and G++ versions. This was solved by installing GCC and G++ versions 7.5.0 into the Conda environment. To activate the compiler, the paths to the package installations were exported as shown in listing 4.4.

```
1 conda install gcc_linux-64=7.5.0 gxx_linux-64=7.5.0
2 export CC=/vol/bitbucket/rqg23/anaconda/envs/deep3d_pytorch/bin/x86_64-conda-linux-
  gnu-gcc
3 export CXX=/vol/bitbucket/rqg23/anaconda/envs/deep3d_pytorch/bin/x86_64-conda-linux-
  gnu-g++
```

Listing 4.4: Configure GCC and G++

Preparing Requisite Models

The original implementation requires pre-trained models that must be installed and arranged carefully to ensure successful reproduction, and can be found [here](#). The `01_MorphableModel.mat` file from the [Basel Face Model](#) project was downloaded, which can be done [here](#) after agreeing to the conditions. An expression basis was also downloaded from [Guo et al.](#) called `Exp_Pca.bin` from this [link](#). Finally, the file for the original trained model called `epoch_20.pth` was installed (see [here](#)).

Data Preparation

To test with custom images, a folder was set up for the test images, with corresponding landmark `.txt` files named after the base name of the image inside a sub-folder titled `detections`. From attempting to test one's own face as taken from an iPhone, experimentation showed that the original implementation was unable to robustly detect faces. With careful inspection, it was identified that some photos come with EXIF metadata, which can apply rotations to images. A `process_exif` script was therefore written to process this metadata by using `ExifTags` from the `PIL` package to choose the orientation value with code 274 and correct the rotation. After this was carried out, the MTCNN package was used in `mtcnn_detector.py` to detect the 5 x 2 landmarks (two eyes, two mouth edges and nose) and saved to `.txt` file to be placed in the test folder's `detections` sub-folder. `mtcnn_detector.py` was taken from a script in the repository's issue thread (found [here](#)). The entire test image processing pipeline was automated in a shell script defined in this project as `process_images.sh`.

Running the Model

The 3DMM-fitting was executed by running the original test script, after configuring the environment and pre-trained models.

```
1 python test.py --name=<model_name> --epoch=20 --img_folder=<folder_to_test_images>
```

Listing 4.5: Test Script

4.3 A Pipeline for Head Model Reconstruction Using the Cascaded Refinement Network

This implementation was based on the [original repository](#) and paper for the Cascaded Refinement Network by Wang et al. [13], and the hybrid auto-completion paper by Stasinakis et al. [95]. However, the original implementation is somewhat confusing and uses the deprecated TensorFlow version 1, which conflicts with the PyTorch functionalities and CUDA requirements. Furthermore, the code in the auto-completion paper is not publicly available. The pipeline and replication of the original papers were therefore implemented from scratch here, primarily adapting the code from the original cascaded refinement network repository (found [here](#)). Instead of using TensorFlow 1 to build the architecture, the project implemented the functionality in PyTorch version 2, which enhances the intuitive understanding of the architecture and maintains GPU flexibility for future reproduction.

Dependencies

As mentioned, PyTorch version 2 was used to build the model allowing the use of more modern Python and CUDA versions. In this project, Python 3.9 was activated in a Conda environment.

```
1 conda create -n myenv python=3.9
2 conda activate myenv
```

Listing 4.6: Dependency Installation

While most packages should download without issue, a bottleneck was encountered in the installation of the [PyTorch3D](#) library. Prioritising building this package over others is therefore recommended, where the necessary installation documentation can be found [here](#). This will naturally include the `pytorch` and `torchvision` dependencies which should be compatible with the CUDA version discussed next. Outside of PyTorch3D, the rest of the dependencies should not be too difficult to install. The `requirements.txt` file is provided with the dependencies used in the project, such that one can either execute `pip install -r requirements.txt` or manually install the requirements on demand as required by the scripts.

As for the CUDA version, this script was run on CUDA version 11.7.1. For the installation of PyTorch3D, the environment variable `FORCE_CUDA=1` may need to be set for PyTorch3D to register the use of GPU. These were added to the `.bashrc` file, following by executing `source <path_to_bashrc>` to ensure that the environment variables were applied correctly.

```
1 export CUDA_HOME=/vol/cuda/11.7.1
2 export PATH=$CUDA_HOME/bin:$PATH
3 export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
4 export FORCE_CUDA=1
```

Listing 4.7: CUDA Requirements

Data Preparation

As part of the data preparation process, a number of shell scripts were written to execute the Python scripts. As detailed in the methodology, the Headspace datasets [57] were used,

but required significant pre-processing for the pipeline. The technical implementation details are discussed here.

The image data originated from the `headspacePngTka` dataset, with each subject comprised of multi-view images and camera calibration files. Since the image data needed to be inputted into the 3DMM-fitting framework described earlier [16], a suitable image needed to be chosen for each of the subjects that would be able to be detected and converted to an `.OBJ` mesh. This was achieved by choosing image 2C from each folder, which featured a side angle but front-facing view, which was enough to be detected by the Deep 3D Face Reconstruction model [16]. Given this, the shell script `prepare_data.sh` was executed, with lines inside the script needing to be edited based on the source and target location of the non-processed and processed data respectively.

To prepare the ground truth 3D head models for each respective subject, `prepare_gt.sh` was executed on the `headspaceOnline` dataset. For initial experimentation purposes, details other than the vertices and faces were removed from the `.obj` file such as the per-vertex RGB colour values, vertex normals, and vertex textures by executing `preprocess_obj.py`, which saves the cleaned `.obj` file to a pre-determined folder. For the final part of the ground truth 3D head data processing, the `mesh_to_cloud.py` script was executed, following the pre-model processing approach described in the methodology by Stasinakis et al. (refer to the data preparation section in 3.2.3) [95], using the `Open3D` package. This returns the 3D head object, which is used as the ground truth for model training.

Having processed the ground truth, the more complicated component comes from processing the image inputs. The issue is that the supervised model requires corresponding input cloud data, which can be tricky to achieve. However, one form of approximation is the use of the Deep 3D Face Reconstruction framework [here](#), which allows one to import custom images and returns a 3D Morphable Model representation face mask `.obj` file. We therefore used this framework as a means of generating the partial input data for the model since a mask is only part of the head model, obtained using the processed 2C images from the `headspacePngTka` dataset discussed previously. To obtain the full partial input dataset of `.OBJ` files, the extracted 2C folder was inputted into the Deep 3D Face Reconstruction framework in the repository [here](#), with modification to the `test_options.py` function in the `options` folder allowing one to specify the location of where to store all of the data files from the framework. Finally, the same `preprocess_obj.py` was executed on that folder to extract and clean the `.obj` files, returning the partial input meshes. Finally, `mesh_to_cloud.py` was executed to also obtain the `numpy.npy` and point cloud `.pcd` files for the partial input.

To complete the data preparation process with the partial input and ground truth files, the functions in `data_process.py` were executed. First, only the `numpy` files in the partial input and ground truth folders were extracted for use, ensuring that inputs had their corresponding ground-truths and vice versa. The second step was to organise the inputs and ground truths to obtain their respective pairings, before splitting them randomly into training, validation, and testing datasets with proportions of 80-10-10.

Model Implementation

The model architecture was implemented according to that of the Cascaded Refinement Network [13] described in the background review and methodology, and translating the TensorFlow `code` into PyTorch code using `nn.Module` and the operations in `torch.nn`. One thing to note that is different to the original repository however, is that the original code

created C++ and CUDA optimisations for the point-related operations in the model, which was not done in this project. These include Farthest Point Sampling (FPS), point gathering, point grouping, and ball-point querying. The project was unsuccessful in directly using their optimisations, such that other solutions were sourced to replace these functions. FPS was achieved by calling the `fps` function in `torch_geometric.nn`, point gathering was achieved using the PyTorch3D implementation detailed [here](#), and the ball-point querying and point grouping were achieved through direct translation of the corresponding code in the original repository. As a consequence, the slow speed of the model could potentially be attributed to this factor, thus requiring further investigation.

`train.py` and `test.py` were then defined following the original code and framework in the paper with simplification. For training, the input and ground truth `.npz` files were loaded in a custom `DatasetLoader.py` script using the `Dataset` from `torch.utils.data` and called with `DataLoader` in the same package. Using `optim.Adam`, and with custom loss functions apart from the `chamfer_distance` metric in `pytorch3d.loss`, the rest of the training script was executed accordingly. In `test.py`, gradient descent was not employed with `torch.no_grad()` and the loss was tested only on the generator function. Corresponding shell scripts `train.sh` and `test.sh` were written to be submitted to the GPU cluster and to facilitate the specification of training and testing parameters.

Visualising the Outputs

To apply test time inference on a numpy partial input, a function was written in `render.py` loading the trained model as a `.pth` file and applying the input to the generator to output an estimated full head numpy output. The final step of the implementation involved following the framework laid out by Stasinakis et al. [95], advocating for point cloud to mesh reconstruction in MeshLab using Poisson reconstruction [106]. Since the outputted result of the model is still a numpy array, further processing is required to convert the numpy array back into a point cloud, which was executed in `cloud_to_mesh.py` following the Open3D documentation [here](#). Since MeshLab did not accept the point cloud `.pcd` file, a further implementation incorporating the `trimesh` package with Open3D was sourced in [this](#) Stack Overflow thread to convert the `.pcd` to `.obj` file representation. However, when trying to visualise the results in MeshLab [95], the reconstruction was found to be unsuccessful, which will be discussed in the evaluation.

4.4 FLAME-Fitting Reproduction

This framework was also a direct replication utilising the code from the original TensorFlow implementation defined in Timo Bolkart’s GitHub repository (found [here](#)). However, some additions to the previous implementation are described here, focusing upon streamlining the testing of custom images. Originally, the idea was to integrate this with the texture map outputs in the FFHQ-UV pipeline, but the resulting topology was found to be incompatible with the mesh representation for eyes. The replication is nonetheless described in detail below for reproduction purposes.

Dependencies

Since the original repository used TensorFlow 1.x, the Python and CUDA versions were limited to older versions. A Conda environment using Python 3.7 was activated, with dependencies installed using `requirements.txt`.

```

1 conda create -n flame python=3.7
2 conda activate flame
3 pip install -r requirements.txt

```

Listing 4.8: Conda Activation

The final dependency was the `psbody-mesh` package which required external installation (found [here](#)). This is dependent on having the Boost C++ library, which was downloaded by executing: `conda install anaconda::boost`. To download the PS-Body mesh, the below command can be executed.

```

1 git clone https://github.com/MPI-IS/mesh.git
2 cd mesh
3 python setup.py install

```

Finally, CUDA 10.0.130 with CUDNN 7.6.4.38 was set in the `.bashrc` file:

```

1 export CUDA_HOME=/vol/cuda/10.0.130-cudnn7.6.4.38
2 export PATH=$CUDA_HOME/bin:$PATH
3 export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH

```

Data Preparation

To run FLAME with custom images, a corresponding 51-point landmark detection is required (see figure 3.5). However, the original repository did not provide a function enabling this detection, and did not provide image processing functions. Therefore, while the project directly replicates the original repository, a few added functionalities were implemented to facilitate easier fitting of FLAME head models.

Firstly, noticing a common issue with head detection and image sizing when applying custom images, `resize_image.py` was implemented, responsible for detecting a head and cropping the image to a pre-defined resolution of 1024 x 1024 in line with the dimensions of the CelebA-HQ face dataset [110] commonly used for face detection. The face detection was achieved using OpenCV's pre-trained Haar Cascade Classifier [109], followed by cropping the image around the head and leaving some space around the face from the image to ensure that the face does not cover the entire image. This was then saved in the output resolution using the Pillow library. The implementation code was taken from a GitHub thread found [here](#).

After this is completed, `get_51_landmarks` was called from `convert_51_landmark.py`, which simply uses the `dlib` face detection module with the 68-point landmark detector. This detector was chosen, because after some research, it was found that the detection of the 68-point landmark detector was no different to the 51-landmark point detector required, just that the landmarks in 68 included the jaw (which takes up 17 landmarks). As a result, the indices were just reshaped and calculated for each image to yield a numpy array of landmarks for 51-landmarks ready for processing. The entire functionality of head detection, image resizing, and landmark detection was all encapsulated inside `preprocess_test_image.py`, later called inside the shell scripting pipeline `imageToMesh.sh`.

Pipeline Implementation

With the requisite models installed and organised based on the specifications of the [repository](#), and the images placed inside the image testing folder, `imageToMesh.sh` can be called to automate the FLAME pipeline calling the `preprocess_test_image.py`, `fit_2D_landmarks.py` and `fit_3D_mesh.py` scripts. The result of the implemented pipeline yields the original `.obj` mesh with texture files (`.mtl` and `.png`), as well as a full FLAME-fitted topology in `.ply` and `.obj` files.

```
1 ./imageToMesh.sh ./test_images/pexels_tester.jpg ./test_images/pexels_tester.jpg ./
  models/shape_predictor_68_face_landmarks.dat female 512 ./test_results/
```

Listing 4.9: Example Execution for `imageToMesh.sh`

4.5 3D Face Reconstruction with FFHQ-UV

The implementation of the reconstruction was directly reproduced from the original FFHQ-UV repository on [GitHub](#), with only the necessary components of the repository extrapolated to the reproduced repository [here](#). This approach was found to be the most robust compared to the other frameworks explored, with further additions being made to facilitate the running of the model pipeline. As part of an extension, an application designed to dynamically model facial expression changes was also implemented.

Dependencies

Since the original code used TensorFlow 1.x, a Conda environment with Python 3.7 was established with the [PyTorch3D](#) and [nvdiffrast](#) dependencies. The below code for Conda activation was taken from ReadMe file of the [original repository](#).

```
1 conda create -n myenv python=3.7
2 conda activate myenv
3 mkdir thirdparty
4 cd thirdparty
5 git clone https://github.com/facebookresearch/iopath
6 git clone https://github.com/facebookresearch/fvcore
7 git clone https://github.com/facebookresearch/pytorch3d
8 git clone https://github.com/NVlabs/nvdiffrast
9 conda install -c bottler nvidia-cub
10 pip install -e iopath
11 pip install -e fvcore
12 pip install -e pytorch3d
13 pip install -e nvdiffrast
```

Listing 4.10: Conda activation. Code taken from <https://github.com/csbhr/FFHQ-UV>

It was found from different experiments that issues may arise in the installation of the third party directories. To fix this, a solution may be found by entering each third party directory separately, and executing `pip install -e ..`. [PyTorch3D](#) is a likely source of the issue due to it being highly dependent on CUDA versions. If one is operating on Linux using the Conda environment, a potential solution could be to execute `conda install pytorch3d -c pytorch3d`. Alternatively, one might try to build from source as shown in Listing 4.10. For more details on the installation of PyTorch3D, the link to the installation guide can be found [here](#).


```
1 pip install "git+https://github.com/facebookresearch/pytorch3d.git"
```

Listing 4.11: Building PyTorch3D from Source

To install the rest of the dependencies required for the project, a `requirements.txt` file was defined in the [reproduced repository](#), executed using `pip install -r requirements.txt`. To access the GPU functions in PyTorch and TensorFlow, CUDA 10.0.130 with CUDNN 7.6.4.38 was established. These environment variables were set in the `.bashrc` file shown below (note that `export FORCE_CUDA=1` is used to explicitly to specify CUDA support for PyTorch3D).

```
1 export CUDA_HOME=/vol/cuda/10.0.130-cudnn7.6.4.38
2 export PATH=$CUDA_HOME/bin:$PATH
3 export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
4 export FORCE_CUDA=1
```

Data Preparation

To prepare the data for testing, the images placed inside the data inputs folder were batch processed by running functions defined in `process_image.py`. This script carries out the removal of EXIF metadata if it exists in the photo provided, and detects, crops, and resizes the image around the detected face using the `OpenCV` Haar Cascade Classifier [109]. The scripts involved in this processing were the same as in the FLAME-fitting implementation. Finally, `clear_data.py` was written to simply clean the processed folders and images from the test images and outputs folder, useful for running data in batches.

Running the Reconstruction Pipeline

Prior to running the pipeline, some pre-trained models needed to be downloaded and organised as detailed in this [ReadMe](#) file. With all of the pre-trained files downloaded, the shell script can be run with `sh run_rgb_fitting.sh`. To note, in this implementation, the functionality for incorporating eyeballs was added to the shell script, since in the original repository it was not clear how to render eyeballs with the head mesh. While the original plan of the reconstruction was to combine the eyeballs with the FLAME topology defined in the previous section, this did not turn out to be possible. Nonetheless, doing so within this framework was simpler. Running the shell script creates an output folder for each input image including the `.obj` files, texture files (`.mtl` and `.png`), a `.pt` file with the specific model coefficients trained, and visualisation images of the fitting process.

Data Visualisation

To visualise the data, one can separately import the `.obj` files into MeshLab, which directly renders the `.obj` files with their corresponding textures. However, one additional functionality that was added in this implementation was the `blend.py` file, which builds an automated pipeline in Blender to render the image, which was proposed because it provides greater flexibility over MeshLab in a variety of applications. The technical implementation behind the `blend.py` file required having Blender and its command line tool set-up on the local device. In the implementation, a call to the Blender application was made in the command line, with environment variables provided to facilitate the running of the python script in Blender. This runs the processing in the background, which can be completed in around a

second for each subject. This involves loading all of the `.obj` files including the head, the two eyeballs, and their corresponding `.mtl` and `.png` files to render the texture.

The reason an automated pipeline was built in Blender was not because of the difficulty of importing the mesh, but because Blender does not automatically apply the texture mapping to the mesh, instead requiring manual configuration. Consequently a function within `blend.py` called `apply_tex2obj` was implemented, applying a number of operations. Firstly, the `.obj` file is smoothed out to ensure that the render does not appear with the triangular-looking mesh configuration. Secondly, the function makes a call to create a new material in the materials tab of the object’s properties and makes it the priority material. Thirdly, an image texture node in the shader functionality is created, which takes in the target texture image. Finally, the output colour of the image texture node is linked to the input base colour of the Principled BSDF node (the main node), which therefore permits full reconstruction. The entire render is stored in a `.blend` file in the subject’s original outputs folder, which facilitates opening a complete render just as in MeshLab.

4.5.1 Extension: Expression Change Application

While the project’s goals are primarily to identify viable approaches in the literature towards static head reconstruction, an extension was provided here to demonstrate the flexibility of the 3DMM approach in manipulating facial expression. This was identified as an important area of research, since the models previously were rendered with an ability to statically render changes in facial expressions, but not dynamically. This makes this implementation useful for the potential digital psychotherapy intervention which would require greater interactivity with the head. This implementation was inspired by the FLAME interactive viewer, with a solution provided from scratch (incorporating the rendering code from the FFHQ-UV fitting) based on 3DMM theory. While the application works, it has yet to have been extensively tested and is in ongoing development.

Dependencies

The front-end of the application was developed in Unity using C#, and did not require any external package downloads beyond base Unity. The back-end of the application was implemented in Python Flask. Since the backend only requires opening a `.pt` file storing the coefficients of a certain mesh, complex dependencies were not needed, with just the requirements of `torch`, `scipy`, `numpy`, `cv2` (OpenCV), `PIL` (Pillow), and `flask` packages. Furthermore, this application ran on CPU, thus not requiring CUDA specifications.

Implementation of the Frontend

Implementing the Unity application required using C# to manage the front-end logic of the interface. Figure 4.2 shows an early stage development example. Here, a random default `.obj` head model is loaded as an asset with neutral expression (taken from the outputs of the 3D Face Reconstruction with FFHQ-UV). This is the model that will change expressions according to the user’s defined commands with the GUI elements. In this early example, 45 sliders were created with a single compute button. The idea behind the 45 sliders is because each head model in the HIFI3D++ topology [22] is associated with 45 expression coefficients (as explained in the methodology). As such, the logic behind the front-end is that by sliding the values of the 45 sliders and clicking the *Compute* button, one can apply these coefficients to the `.pt` file associated with the `.obj` head, viewing those changes visually.

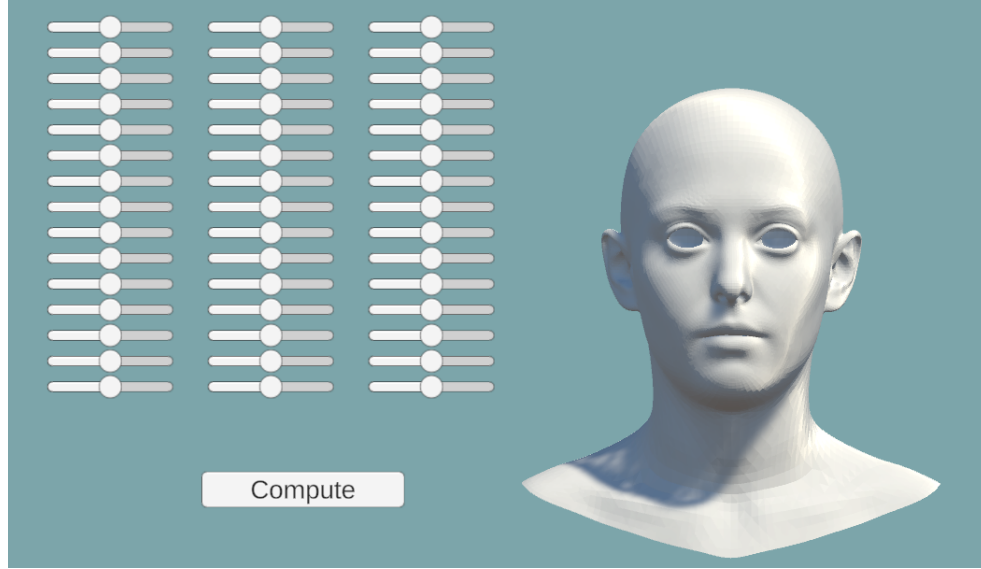


Figure 4.2: Diagram showing the user interface of the application with 45 sliders.

To enable this functionality, the front-end logic was contained inside `ButtonHandler.cs`, which referenced each of the 45 created slider objects, and the physical button object. The idea behind the code is that clicking the button triggers the event listener, which starts the function `OnSubmit` within the script. In `OnSubmit`, a list of float values of size 45 was created, and passed into the function `SendExpressionRequest` as a parameter. As part of this function, it first creates the data package that will be used to send a request to the Python backend. This includes the path of where the `.pt` coefficients file is stored, what path the output mesh file should be written to, and the float list of expression slider values. Stored as a JSON string, and passed as a HTTP POST request to the flask server, the backend logic is consequently executed, which after making the changes to the coefficients in the `.pt` file, renders them as a mesh and saves this render to the specified output location.

After a successful call and response to the Python back-end responsible for rendering a new mesh file with the changed expression, the front-end logic continues with downloading the `.obj` file rendered in the back-end. This was achieved with a GET request to download the mesh file to the front-end, before applying mesh processing logic in the frontend to clean the mesh file in preparation for data loading. As part of this cleaning, only the vertices and faces were extracted to simplify the mesh model, with normals calculated using `mesh.RecalculateNormals()`. Finally, the new mesh replaces the existing neutral mesh in the `ApplyMeshToModel` function.

Implementation of the Back-end

As described in the implementation of the front-end, two calls are made to the backend in the application: the first involves posting data about the change in expression, and the second is to download the outputted mesh file. In the first function called `generate_mesh`, `torch` was used to load the `.pt` file, before the coefficients were split to obtain the dictionary pointing towards the indices of the different bases respectively. Since we are only interested in the

expression basis, the values in the coefficients file were modified by adding the values specified by the sliders. Using a skeleton of the 3D-Face Reconstruction with FFHQ-UV repository only using the scripts used to render the .pt files, the mesh titled `emotion_exp.obj` was saved in an output folder specified in the frontend logic. The second part of the back-end involved downloading the model in `download_mesh`, and was achieved simply by executing the `send_from_directory` function in Flask.

Current Implementation

While the slider approach was used for initial experimentation, it is not the most practical for use in an interactive application, since the 45 sliders have arbitrary meaning. As such, the slider functionality was removed in favour of toggles for the 6 basic emotions [23] and neutral expression, which were obtained from trialling different linear combinations of the 45 sliders. The Python backend code remained the same, while the changes needed to only be applied in the frontend logic. Instead of calling slider objects, a `ToggleGroup` object was called instead allowing only one expression to be selected at one time, and with statically defined values for the 45 expression bases. For the neutral expression, all 45 values were set to 0. Assuming a default value of 0 for the basis if not mentioned, the six emotions are described here. For happiness, the 25th and 26th expression bases were each set to 1.38. For sadness, the 25th and 26th bases were set to -0.73. For fear, the 37th value was set to 0.3, and the 38th value set to -0.84. For anger, the 31st value was set to -0.11, 32nd value to -0.19, and 33rd value to 1.52. For surprise, the 10th value was set to 1.47. Finally, for disgust, the 14th value was set to 1. Based on the toggle that is selected when the button event listener is triggered, will be the fixed float list of values for that emotion which is passed as a JSON string to the Python backend to process in the same way as defined earlier. One further interactive capability of the model was to rotate the 3D head object, which was achieved by writing `MouseRotateHead.cs`, capturing the mouse input from holding the mouse scroll which rotates the head accordingly.

Chapter 5

Evaluation

In this chapter, the implemented models described in the methodology are evaluated based on their success in reconstructing the head avatar. The most successful model after experimentation was the 3D Face Reconstruction with FFHQ-UV approach [20], and will be the main focus of this evaluation since it provided the most promise holistically with regards to visual results and flexibility for integration within an interactive application. The evaluation will be structured to first briefly describe the results of the other experiment models, describing their limitations and offering explanations as to why these were dropped in favour of the current approach. This will then be followed by an extensive evaluation of the FFHQ-UV reconstruction framework, including an objective benchmark measure and a user feedback survey gauging opinions about the quality of reconstruction.

5.1 A Brief Evaluation of the Experimented Models

5.1.1 3D-VAE-GAN

While the 3D-VAE-GAN was easy to train with relatively simple processing steps and architecture, the experiment confirmed some critical issues leading to it being dropped early on. Firstly, the volumetric approach in this architecture was originally applied to more symmetrical daily-life objects requiring less precise resolution compared to faces and heads. The output representation in Figure 4.1 intuitively proves the difficulty of trying to represent a face accurately with binary voxels. Secondly, even if the resolution were to be increased, this would come at a significant computational cost. From using the `viewvox` function (mentioned in the implementation) to visualise a binary voxel representation at a resolution of 256^3 , it was immediately obvious that this would not be feasible for interactive app integration since it was fairly unresponsive even when viewed on a Mac M2 chip. Finally, the model only represents shape, without considering the issue of texture. While voxel colouring is an option, its combination with the cube-like nature of the output representation does not bode well for smooth face representation required for the application. These factors presented two key learnings - that volumetric approaches without significant computational power are not feasible or accurate, and that some domain-specific (i.e. directly modelled for the purpose of faces) research is likely required to guarantee some success in the visual

output.

5.1.2 Deep 3D Face Reconstruction

The Deep 3D Face Reconstruction architecture proposed by Deng et al. [16] provided a fantastic framework and reliable means of generating face masks, ultimately confirming the viability of using 3DMMs and mesh-based representations for reconstruction. The output of the fitting process is shown in Figure 5.1 below (the reference image is shown in Figure 3.1). However, while the ability to convert a 2D image into a 3D face mask is impressive, it does not directly solve one of the crucial objectives of the project - to build a full 3D-head model. Furthermore, the mesh file applies texture by using corresponding RGB vertex coordinates, which may not have the same consistency of capture as UV-map overlays [111]. From the side-profile in figure 5.1b, it can be seen that there is no robust way to deal with hair occlusions, although the ability to remove the glasses and preserve the facial identity is impressive. Nonetheless, the impressive results of this framework justifies its integration into other pipelines for full 3D head reconstruction as is the case in the proposed FFHQ-UV reconstruction pipeline [20]. If not for its inability to reconstruct the entire head, it may be argued that the quality of the texture mapping achieved in this framework is stronger than for the other models presented in this study.

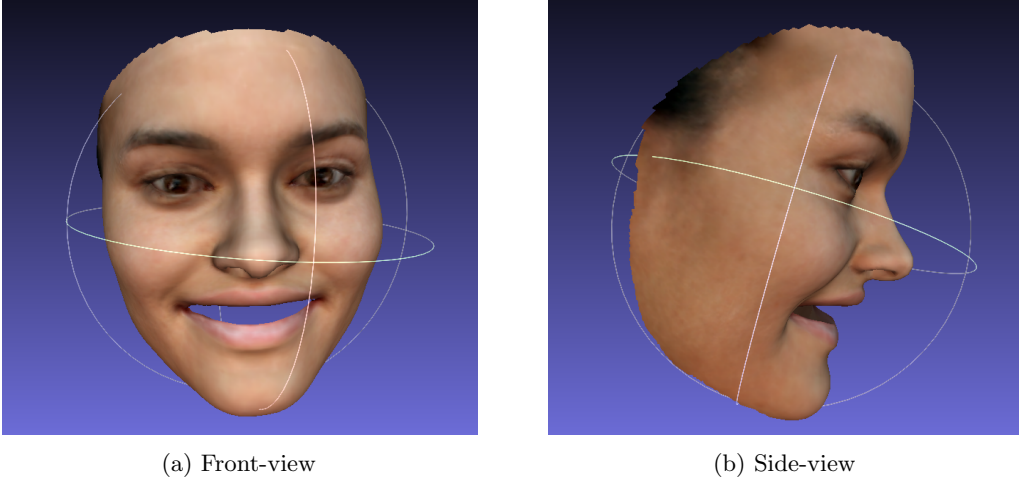


Figure 5.1: Face mask rendering from the Deep 3D Face Reconstruction Approach [16].

5.1.3 A Pipeline for Head Model Reconstruction Using the Cascaded Refinement Network

Training the model over 1000 epochs with mini-batch gradient-descent showed a training and validation loss plot in Figure 5.2, which from first glance appears to be promising, with steady learning progressing towards an error between 0.05 to 0.06. The model performed similarly on the test set from early experimentation. However, when attempting to apply the model to render a full 3D head after the output point-cloud for a chosen test-image was processed into mesh following the steps of Stasinakis et al. [95], MeshLab was unable to connect the point cloud vertices to create an output representation.

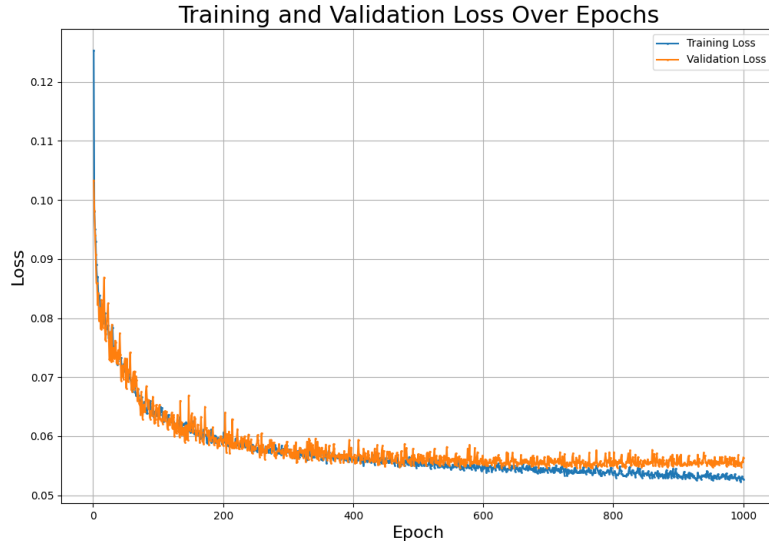


Figure 5.2: Plot showing the training and validation loss of the cascaded refinement network model over 1000 epochs.

A number of suggestions are made in hypothesising the issue behind this failure. Firstly, the extensive data conversion from mesh to point cloud to numpy array during model training, and reverse-conversion from numpy array to point cloud to mesh in the rendering may have caused a lot of information to be lost in translation. This was partially confirmed at the start of experimentation when trying to convert the face mesh from the Deep 3D Face Reconstruction framework [16]. Converting the mesh to point cloud using the functions defined in the implementation, and trying to render that point cloud back into mesh already led to slight inaccuracies in the shape of the facial mesh without any modifications. A second potential reason for the failure of the model may have been in selecting 2048 points as the target resolution, which may have been too little for properly reconstructing the entire head. As shown in Table 2.1, even the simplest topology in FLAME has far more vertices than 2048 [56], perhaps indicating that an output resolution at least double the current implementation may be needed to fully model the entire head. This hypothesis was also backed up by the results of the FinerPCN [96], which modelled the point completion of a face mask using 6144 points, which was still not enough to accurately represent the face. Therefore, trying to represent the entire head using 2048 points would have been optimistic. Furthermore, to work with a greater number of points would require greater GPU optimisation and hardware, which the project did not account for. A final issue lay in the fact that the approach still does not address the issue of texture. While the use of the Deep 3D Face Reconstruction framework may have helped to render front-view texture, a suitable method for estimating the texture not covered by the single-view image would have to have been found. This ultimately lead to the lack of success in this approach.

5.1.4 FLAME-Fitting

The advantages of the FLAME-fitting method cannot be understated owing to its capacity to reconstruct the entire 3D-head which could not be achieved in the project prior to this experiment [56]. As mentioned previously, FLAME can represent the shape, expression, and pose fairly convincingly, achieving this with less vertices compared to other topologies (see Table 2.1). A further advantage of the implemented FLAME framework is that it has the capacity to also manipulate expression in a dynamic manner (see [here](#)). However, the texture mapping capabilities in the original repository did not stack up to its shape and expression modelling capabilities. While the FLAME method can create texture overlays as UV maps instead of RGB vertices, the quality of the overlay is not so impressive. Using the same input image as in figure 3.1, the results of the FLAME-fitting are shown in figure 5.3. As can be seen, the rendered texture map (5.3a) does not deal with occlusions very well, instead attempting to keep as much of the input image as possible in texture mapping format. Such an approach is a double-edged sword. When applied correctly, it can provide a high-level of detail beyond the capabilities of other texture representations, but when applied incorrectly, can appear significantly distorted. For example, figure 5.3b shows the front-view of the mapping, which from visual inspection, appears to be somewhat impressive with regards to detail and realism. However, with its lack of standardisation, leads to the back of the head being unable to detect any texture mapping as in 5.3c. If a convincing methodology for fitting texture to the entire head can be found while preserving the same degree of front-facing texture mapping detail, the FLAME-fitting method could have serious potential for an interactive application.

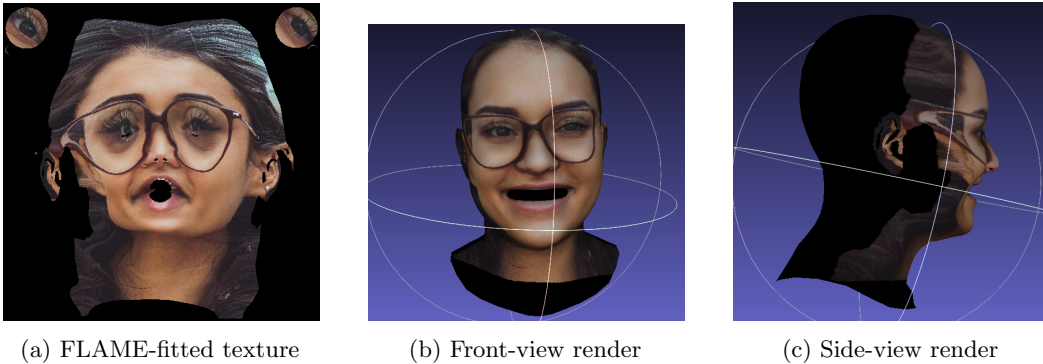


Figure 5.3: (a) represents the FLAME-fitted texture map. (b) is the front-view mesh render with texture map applied, and (c) is the same, but shown from the side view.

One interesting note is that in the original FFHQ-UV repository, a solution was proposed by the researchers to combine the FLAME model with the HIFI3D++ [22] UV map topology see [here](#). In figure 5.4, the exact same subject was rendered using this approach, which exhibits crucial differences. The FFHQ-UV texture map is capable of completing a full 3D-head render, and deals with object-occlusion definitively. However, this comes at the cost of texture mapping detail, despite the 3D shape being somewhat preserved. This approach also is not capable of eyeball rendering. Overall, FLAME offers a pretty versatile framework, but requires significantly more work in order to be used in an interactive application.

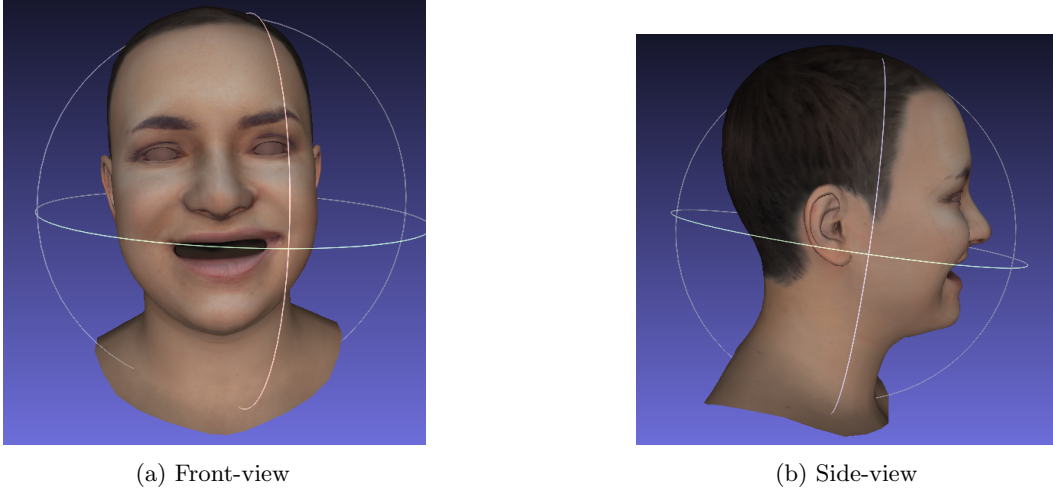


Figure 5.4: The front and side view renders of the reconstruction with FLAME topology and HIFI3D++ texture maps [56, 20]

5.2 3D Face Reconstruction with FFHQ-UV

Having discussed the other experiment models, we arrive at the approach with the most success in achieving the project goals - the 3D Face Reconstruction with FFHQ-UV approach by Bai et al. [20]. While the visual output is very similar to the hybrid FLAME and HIFI3D++ texture map representation shown in Figure 5.4, this framework generates the head model in HIFI3D++ topology which allows integration with eyeballs for more realistic renders (see figure 5.5). A rigorous method for evaluation is proposed for this model, as it represents a framework that has serious potential for further implementation in interactive applications. Firstly, the shape reconstruction of the head (more specifically the facial features) is tested against the REALY benchmark [22], allowing quantitative comparison with some leading face and head reconstruction research papers. Secondly, a user feedback survey (see Appendix A) was conducted to understand how subjects felt about the quality of the rendering. This is beneficial in providing real-world feedback as to the potential integration of this model in a digital psychotherapy intervention such as for the SAT [7, 10].

5.2.1 REALY Benchmark

REALY is a popular benchmark for measuring shape reconstruction accuracy featuring 100 face scans [22]. It works by testing the proposed model on the REALY dataset, returning 100 reconstructed .obj meshes. This is then followed by comparison with the standardised and aligned face mesh calculations from running the evaluation pipeline in the [REALY repository](#) by specifying the topology of comparison, which in this case is HIFI3D. While the topology of the FFHQ-UV framework is actually HIFI3D++, the basic topology structure with regards to the number of vertices and faces is exactly the same (see Table 2.1), allowing it to be used directly for comparison. The result of running the REALY benchmark yields the error differences for four key features: the nose, the mouth, the forehead, and the cheeks. This captures the mean, median, and standard deviation summarised in millimetres. Finally,

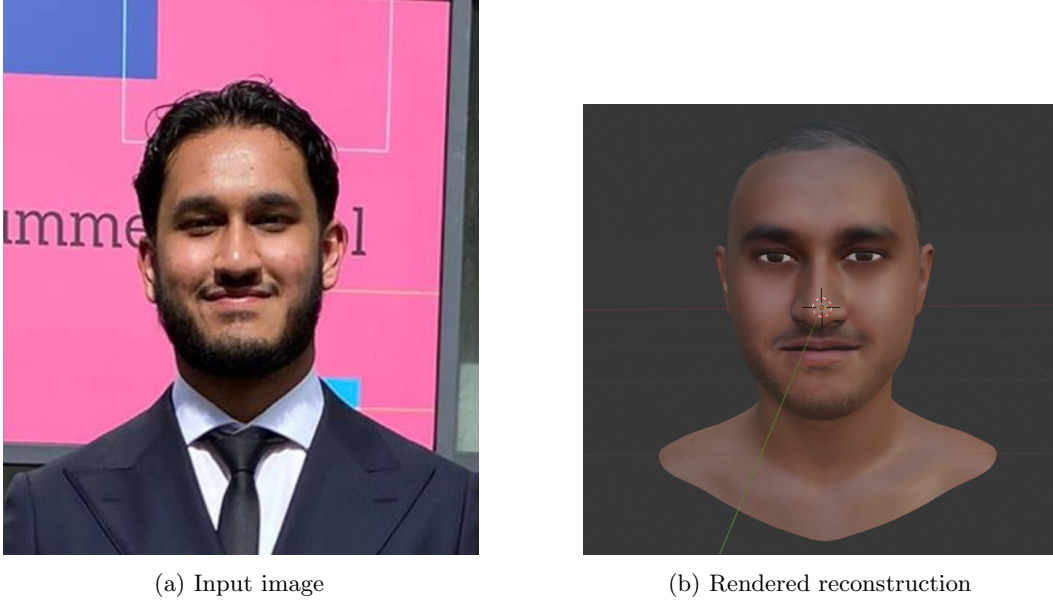


Figure 5.5: Example reconstruction using the 3D Face Reconstruction with FFHQ-UV approach [20]

an overall average is calculated to represent the holistic performance of the model.

A table showing the performance of different head reconstruction models in the REALY benchmark is shown [here](#), with the results of this reconstruction model shown in table 5.1. The average error in this model was 1.400mm, which on average performs the third best after HiFace-f and HiFace-c [112] with 1.275mm and 1.297mm respectively. When comparing each facial feature independently, the benchmark showed that that the modelling of the cheek was considerably stronger than for the other features, with a feature ranking of first compared to other papers in the literature. The mouth shape and forehead were also good, coming in second and third respectively. The nose did not perform so well comparatively, coming in seventh. One interesting point to note is that while the forehead did well in ranking, the higher value for the error metric may indicate that this is a particularly difficult feature to model compared to others. On the whole however, it can be concluded that the framework implemented here is robust from the perspective of shape-modelling.

	@nose (mm)	@mouth (mm)	@forehead (mm)	@cheek (mm)
avg.	1.682	1.273	1.678	0.968
med.	1.610	1.208	1.634	0.903
std.	0.410	0.430	0.390	0.316

Table 5.1: Table showing the result of the REALY benchmark [22] on the 3D Face Reconstruction with FFHQ-UV [20]. The average error across all metrics was 1.400 mm.

5.2.2 User Feedback Survey

A survey was conducted to ask the subjects some questions about renderings that were carried out for them and what they thought of the overall reconstruction. All questions (apart from the time of reconstruction) were measured on a 5-point Likert scale with the general idea of 1 representing very negative sentiments and 5 representing very positive sentiments towards the statement. These results are presented and evaluated. A total of $n = 31$ subjects provided feedback.

Table 5.2 shows feedback regarding how accurate the subjects perceived the reconstruction for the key features of the face including the eye, nose, mouth, and cheeks. From interpreting the table and box plots, it can be seen that the nose was deemed most accurate compared to other facial features, with an interquartile range of 0 at a rating of 4 suggesting that subjects were pleased with this feature. The subjects were also generally pleased with the accuracy of the eyes and mouth. However, there was mixed sentiment towards the accuracy for the cheeks, displaying a bell-shaped distribution as shown in figure 5.6. One interesting observation is that the results of the user feedback seem to contradict with the results of the REALY benchmark. This disparity may potentially be explained by the data supplied to the benchmark being more standardised compared to the images supplied in this study which were mostly in-the-wild captures. This may have affected how accurately the model was able to capture the features compared to the benchmark results.

	Eye Accuracy	Nose Accuracy	Mouth Accuracy	Cheek Accuracy
Mean	3.645161	3.870968	3.516129	3.225806
Std.	1.081616	0.921663	0.961629	1.334408
Min	1.000000	2.000000	1.000000	1.000000
25%	3.000000	4.000000	3.000000	2.000000
Median	4.000000	4.000000	4.000000	3.000000
75%	4.000000	4.000000	4.000000	4.000000
Max	5.000000	5.000000	5.000000	5.000000

Table 5.2: Descriptive statistics for facial feature accuracy metrics

From some of the other feedback provided in the survey, there were generally positive responses to the capturing of facial proportions and lighting. The capturing of skin tone and colour received mixed reactions, but the capturing of finer facial details was evidently not positive as shown in figure 5.7, directly indicating that the shortcomings of the implementation lies in the texture aspect of the model. The negative sentiment towards the finer facial details makes sense, since the model completely deals with object occlusions, and further generates a new estimated texture map from the model rather than attempting to preserve the complete details of the input image like it does in the FLAME-fitting experiment.

Attempting to detect further nuance in the mixed reaction regarding the skin colour and tone, the data captured on the subjects’ ethnicity were used to further distinguish the distribution of opinions. Figure 5.8 shows the range of ethnic backgrounds captured in the data, encoded directly using the ethnic backgrounds specified in the UK Census 2021. As is shown, certain ethnicities displayed more positive reception towards the reconstruction than others. This may be as a result of biases in the data used to train the original model, making the skin texture render less effective for some demographics over others. On the other

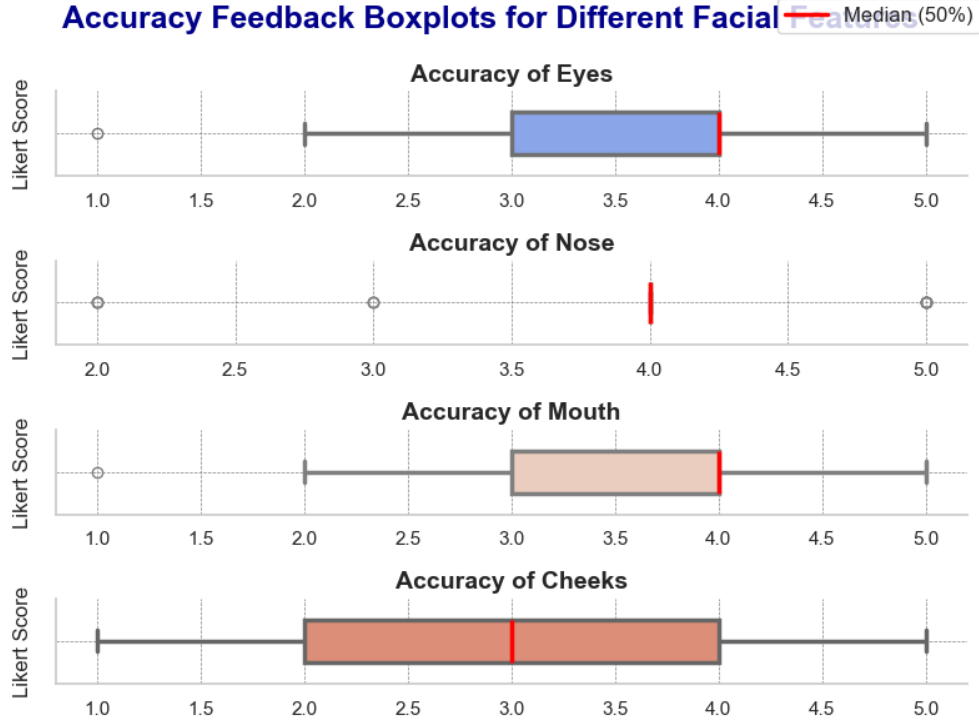


Figure 5.6: Box plots visualising the feedback for accuracy of facial features

	Facial Proportions	Skin Texture	Fine Facial Details	Lighting
Mean	3.612903	3.096774	2.612903	3.451613
Std.	0.954897	1.247794	1.054433	1.178663
Min	2.000000	1.000000	1.000000	1.000000
25%	3.000000	2.000000	2.000000	3.000000
Median	4.000000	3.000000	3.000000	4.000000
75%	4.000000	4.000000	3.000000	4.000000
Max	5.000000	5.000000	5.000000	5.000000

Table 5.3: Descriptive statistics for some of the other feedback. **Facial Proportions** refers to the size of the features in relation to each other, **skin texture** refers to the capturing of the skin tone and colour, **fine facial details** refers to the capturing of details such as acne or freckles, and **lighting** refers to how good the lighting of the reconstructed model is compared to the provided image.

hand, a potential confounding factor may be the quality of lighting in the images supplied, which was found to have quite a profound influence on the model reconstruction during experimentation. Despite this, the results shown in figure 5.8 need to be more rigorously tested on a much larger population, as the sample size is far too small to detect real nuances in the model’s skin capture abilities. Some of these thoughts will be summarised in my own

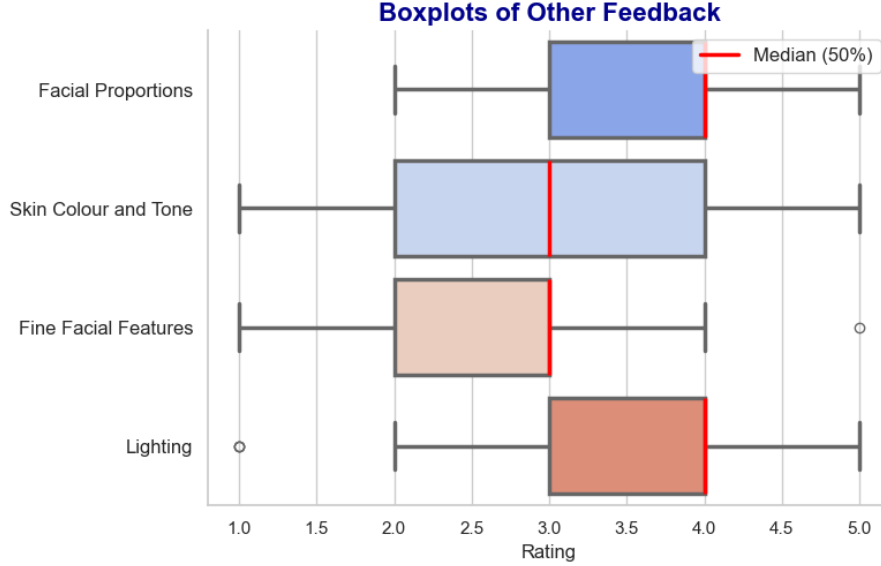


Figure 5.7: Box plots visualising the other feedback.

self-evaluation of the 3D Face Reconstruction with FFHQ-UV model.

Time for Reconstruction

Since some reconstruction times were incorrectly supplied in the survey, the wrong values were filtered out leaving $n = 28$ data points. The data showed an average reconstruction time of 138.15 seconds. As shown in the histogram in figure 5.9, there was quite a large distribution of times which is interesting, since the images were not all tested at the same time, but rather in batches. The disparities are therefore difficult to interpret, since some were reconstructed on an NVIDIA Titan Xp whereas others were reconstructed on the NVIDIA GeForce RTX 2080 Ti. Being trained on a shared network at different times of the day may also have led to differences in GPU utilisation, perhaps supporting the idea that lower GPU utilisation leads to faster renders and vice versa. One other potential explanation for longer times is the initial start-up time, whereby after various experiments, it was found that utilising and booting up CUDA after a period of inactivity led to the first render taking longer than subsequent renders. Finally, the complexity of the image supplied adds time to the image processing stage of the reconstruction. From the data, it was shown that the model took 91.88 seconds at its fastest, but 207.13 seconds at its slowest. According to opinion polls, subjects overall found this reconstruction to be fast, with an average of 4.14 out of 5 rating. The poll also indicated that subjects would prioritise rendering quality over rendering time with an overwhelming preference rating of 4.25 out of 5 with 3 representing equal importance.

5.2.3 Qualitative Evaluation of the Model Framework

Overall, the 3D Face Reconstruction with FFHQ-UV [20] displayed pleasing results from the perspective of achieving the basic aims of the project in generating a full 3D head avatar

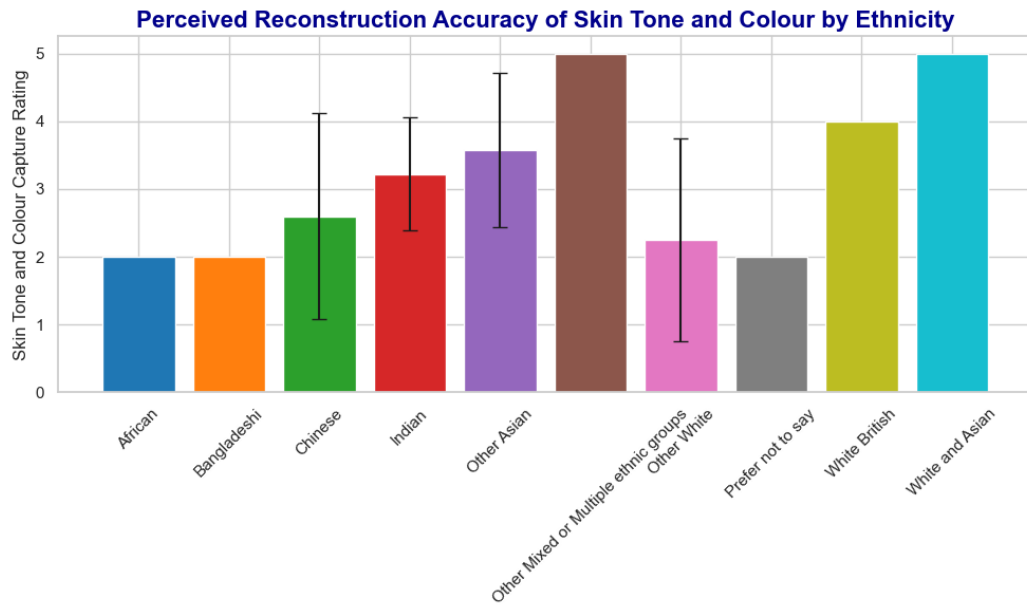


Figure 5.8: Bar plot with error bars describing what subjects from different ethnic backgrounds thought about the model's ability to capture skin tone and colour.

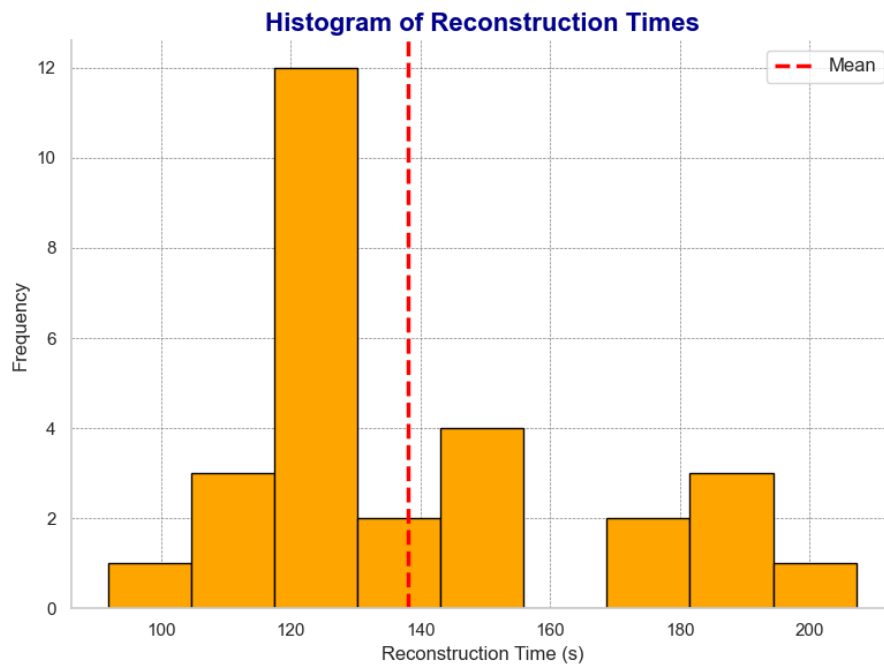


Figure 5.9: Histogram visualising the distribution of reconstruction times.

that could be used in an immersive application. The framework is fairly simple to implement when using the pre-trained models, and does not over-complicate aspects of the face and head that need not be modelled in too much detail. Furthermore, as has been shown in the expression-change application, the model can support more dynamic and interactive use cases, which gives credit to the field of 3D Morphable Models. The capturing of shape in the head model is also clearly quite strong as proven by the results of the benchmarking and from visual inspection.

Despite this, there are also criticisms of the model and areas requiring significant further exploration, particularly in identifying the edge cases that cause some renders to be very successful and others to not look like the input image whatsoever. Some speculative thoughts are therefore discussed here. The main issue involves the reliability of the skin colour reconstruction. From experimentation, it was found that darker skin tones and colours were often made lighter in reconstruction than they actually are in reality, and lighter skin tones can often be made darker than they actually are in reality. A likely cause of these effects may be in the data bias of the trained model or the quality of lighting in the input image. From various trials, it was also found that the texture mapping would sometimes cause the forehead area to have a darker skin colour compared to the rest of the face. This is hypothesised to potentially result from certain hairstyles such as fringes partially covering the forehead to impact the lighting factors around the forehead. Another issue is that from visual inspection, it appears as though facial hair such as beards may be influencing the shape of the face in increasing the proportions compared to that in reality. While this is less of an issue if used in the context of child avatars whereby facial hair does not exist, this is still an area that requires further investigation. The parts of the head reconstruction that are wholly unaccounted for include the eye colour, the modelling of the teeth, and more fine details such as acne or freckles. As such, these also require further investigation.



(a) Input image

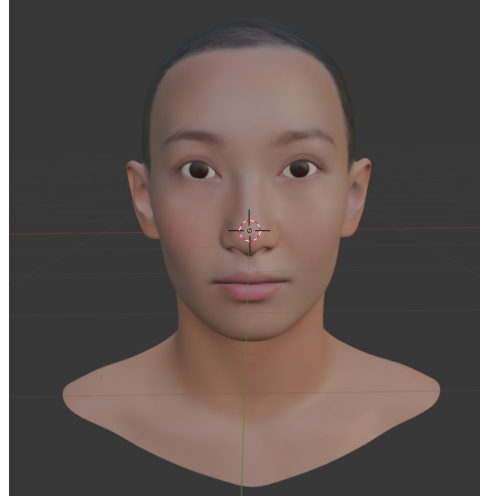


(b) Rendered reconstruction

Some of the qualitative feedback from the survey is also detailed. The majority of the feedback involved the desire to see hairstyle generation as part of the model, which although is not covered by the scope of this project, is certainly an area of future work. As one of the subjects mentioned, they believed that by incorporating the hairstyle, it would mask



(a) Input image



(b) Rendered reconstruction



(a) Input image



(b) Rendered reconstruction

some of the inaccuracies behind the model. Another comment included the difficulty in differentiating the gender of the subject from the reconstructed model, which provides an interesting path of potential research for future model versions. While this is not accounted for in the FFHQ-UV reconstruction pipeline [20], it is a capability of the FLAME-fitting

framework [56], which could therefore introduce a future area of incorporation. Additional comments were also made on the eye colour as previously mentioned, the ears, and the jaw, with one subject mentioning that the model displayed an overbite for them. Further testing on more subjects would therefore certainly help determine the areas of issue.

5.2.4 A Brief Word on the Expression Change Application

The head models for the six basic emotions are shown in figure 5.13 [23]. Initial experimentation of the application produced reliable and quick expression changes when called upon. This demonstrates that the proposed method for applying expression changes works, such that the next step is to test the application and get feedback about whether the fixed-coefficients for the emotions have been captured accurately. Another important next step is to try to apply the texture to the head model for use in an interactive application. Overall, the initial results from the application using the proposed 3D Face Reconstruction with FFHQ-UV framework is promising both from a static and dynamic perspective.

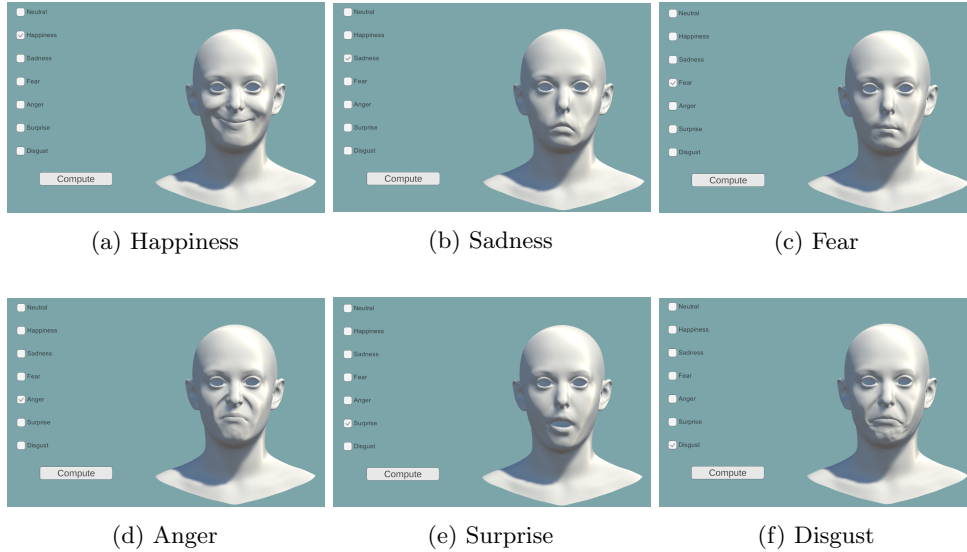


Figure 5.13: The 6 basic emotions captured by the expression change application

5.3 Ethical Considerations

The project considers ethical issues from two principal fronts: the handling of data collection in the survey, and the applications that 3D head reconstruction can have in wider contexts if misused. The project collected primary data in the form of images for feedback regarding the efficacy of reconstruction. Full consent was obtained from each participant to use the images that they chose for the reconstruction, and they were notified prior to taking the survey of their right to withdraw their data (including the images they provided and the survey responses) at any time of their pleasing unless published. For the images and reconstruction outputs published in this report, consent was also obtained, since many participants may be uncomfortable with how their faces have been reconstructed, particularly if generative models

are used. Participants were further informed of their data anonymity and confidentiality throughout the entire process. Further ethical considerations made during data collection involved capturing the demographic data, which was made optional in the survey. The demographic questions and options given to the participants were designed with sensitivity, respecting the freedom of choice in the gender question, and maintaining cultural sensitivity in the ethnicity question by using the ethnicity codes established in the UK Census 2021.

As for the second ethical concern, as Ho et al. suggest [29], generative techniques can cause unfavourable fake representations of individuals that can have social, political and legal consequences if misused. While this application does not intend to encourage misuse in this regard, it is important to note that these issues exist and further safeguards are installed to prevent this from occurring if used in a potential digital psychotherapy intervention. Generative models can reinforce negative biases based on the data inputs [29], which is one issue that must be acknowledged. This is indeed reflected in how models choose to render the subject, with concern for the unintended reinforcement of certain ethnic stereotypes that could offend the subjects. The request for ethnicity data in the survey was therefore motivated by a desire to evaluate the ethnicity-based nuances in the model to help gauge where sources of contention may lie.

Chapter 6

Conclusion

A variety of approaches to 3D head reconstruction were discussed in the present study, many of which emerge from the intersection between the field of 3D Morphable Models and modern deep-learning. The most effective approach from exploration was the 3D Face Reconstruction with FFHQ-UV [20], which was able to model the entire 3D head from a single 2D-image, and with a consistent UV-mapping texture. Results from a shape reconstruction accuracy benchmark (REALY) showed that the model outperformed a number of state-of-the-art 3D face and head reconstruction papers in the literature, and user feedback showed generally positive reviews about the reconstruction with regards to the accuracy and the time taken to model the head. A small application developed to dynamically apply changes to the expression of the 3D-heads generated by the FFHQ-UV framework also proved to be successful in capturing the 6-basic emotions [23]. Overall, the adoption of the FFHQ-UV framework was able to satisfy the core goals of the project in generating a full 3D head from a single 2D-image, while also showing serious potential for integration in an interactive application such as virtual reality or gaming. It is hoped that with this implementation, further work is carried out to improve the model and bolster its reliability, as its potential use in a digital psychotherapy intervention such as SAT [7] could bring serious positive engagement with self-administered therapy and thereby improve one's mental well-being.

Limitations and Future Work

While the 3D Face Reconstruction with FFHQ-UV framework successfully renders a 3D head with decent 3D shape reconstruction, texture is still the crucial limiting factor as to the perceived accuracy of the entire head model. As described in the evaluation, issues can arise in a variety of manners including the inadvertent lightening of the skin colour, occlusions such as facial hair and glasses impacting the accuracy of the texture, and the lack of finer facial details reducing the realism of the reconstruction. A number of potential solutions and future areas of research are therefore proposed. Firstly, the skin-colouring issue could have resulted from data biases in the original FFHQ-UV dataset used to train the texture decoder component of the reconstruction model. One potential solution to this problem is therefore to re-train the texture decoder using a more representative "in-the-wild" dataset featuring more equal representation across races and backgrounds. This is a particularly important aspect of the reconstruction model to get right, since the intended use case of deploying this to a psychotherapy intervention requires great sensitivity to the subjects involved, as has

been outlined in the ethical considerations. This is a feasible goal, since the source code from the original FFHQ-UV repository is openly available and capable of processing "in-the-wild" images in batches. While collating and rendering the images may take significant time and effort, the resultant dataset can be used to re-train the texture decoder to produce better results. Another potential solution to the issue of texture mapping could be to continue sourcing inspiration from other research in the literature. For example, in the FLAME-based fitting in figure 5.3, a UV-map overlay was created to fit the image directly to the head model. While it was shown that the texture was unable to extrapolate to the entire head, the actual front-facing texture map showed significant promise with regards to actual realism of render. Future research might therefore benefit from closely investigating how this approach could be improved to fill the missing parts of the texture file to reliably fit the head model. With regards to texture, one final area requiring further research is how to dynamically apply the texture mapping in line with expression changes as required in the interactive application. A pointer towards where this solution could be found is in further investigating the texture coefficients of the HIFI3D++ topology [22].

Another limitation of the framework also regards the representation of the eyes, ears, teeth, and hair. In the proposed framework, these are either broad generalisations that could do with greater detail or features that are completely missing. The eyeballs are rendered with the same colour in every head model, such that a robust method for detecting the colour from the image and applying that to the eyeball texture map is needed. While in many applications this may not be entirely noticeable, it is certainly an area that could be pursued in the future to raise the realism of the render. In the same vein, the shape of the ears also appear to remain the same for each render, which could be a further area of investigation. On the other hand, the modelling for teeth is completely missing, which can be seen in non-neutral expressions whereby the subject is opening their mouth such as in the surprise emotion (see figure 5.13). While the teeth could technically be approximated, further work is required to intricately investigate the vertices and faces in the HIFI3D++ topology to precisely map the location on the head mesh for where the teeth can be inserted. Finally, the ability to integrate hairstyles with the head is crucial in ensuring realism and satisfaction from the perspective of users obtaining renders. One future area of important contention will lie in how to align both representations together reliably and automatically for any prospective subject.

The final area of limitation in this study relates to the lack of rigorous testing applied to both the reconstruction model and the expression change application. To prepare both of these for potential integration in the digital psychotherapy intervention, a more detailed understanding of the edge cases that affect where the product is failing or producing non-optimal results is required to ensure that a fail-proof pipeline is established for the future. Once these have been explored in detail, it is believed that with the proper development of a frontend application, the 3D Face Reconstruction with FFHQ-UV model can be used effectively and to a positive reception within a digital psychotherapy intervention.

Bibliography

- [1] Ferdani D, Fanini B, Piccioli MC, Carboni F, Vigliarolo P. 3D reconstruction and validation of historical background for immersive VR applications and games: The case study of the Forum of Augustus in Rome. *Journal of Cultural Heritage*. 2020;43:129-43. Available from: <https://api.semanticscholar.org/CorpusID:213800823>.
- [2] Dawn S, Biswas P. Technologies and Methods for 3D Reconstruction in Archaeology. In: Thampi SM, Marques O, Krishnan S, Li KC, Ciuonzo D, Kolekar MH, editors. *Advances in Signal Processing and Intelligent Recognition Systems*. Singapore: Springer Singapore; 2019. p. 443-53.
- [3] Khan U, Yasin A, Abid M, Shafi I, Khan SA. A Methodological Review of 3D Reconstruction Techniques in Tomographic Imaging. *Journal of Medical Systems*. 2018;42(10):190. Available from: <https://doi.org/10.1007/s10916-018-1042-2>.
- [4] Samavati T, Soryani M. Deep learning-based 3D reconstruction: a survey. *Artif Intell Rev*. 2023 jan;56(9):9175–9219. Available from: <https://doi.org/10.1007/s10462-023-10399-2>.
- [5] Weightman M. Digital psychotherapy as an effective and timely treatment option for depression and anxiety disorders: Implications for rural and remote practice. *Journal of International Medical Research*. 2020 06;48:030006052092868.
- [6] Krupnick J, Melnikoff S. Psychotherapy with Low-Income Patients: Lessons Learned from Treatment Studies. *Journal of Contemporary Psychotherapy*. 2012 03;42:7-15.
- [7] Edalat A. Introduction to self-attachment and its neural basis. In: 2015 International Joint Conference on Neural Networks (IJCNN); 2015. p. 1-8.
- [8] Bowlby J. The Bowlby-Ainsworth attachment theory. *Behavioral and Brain Sciences*. 1979;2(4):637–638.
- [9] Law AJ, Hu R, Alazraki L, Gopalan A, Polydorou N, Edalat A. A Multilingual Virtual Guide for Self-Attachment Technique. 2022 IEEE 4th International Conference on Cognitive Machine Intelligence (CogMI). 2022:107-16. Available from: <https://api.semanticscholar.org/CorpusID:257526052>.
- [10] Edalat A, Polydorou N, Ryan F, Gilbert BJ, Nicholls D. Affectional interactions with your animated child avatar can enhance wellbeing. *Frontiers in Virtual Reality*. 2024;5.

- [11] Blanz V, Vetter T. In: *A Morphable Model For The Synthesis Of 3D Faces*. 1st ed. New York, NY, USA: Association for Computing Machinery; 2023. Available from: <https://doi.org/10.1145/3596711.3596730>.
- [12] Wu J, Zhang C, Xue T, Freeman WT, Tenenbaum JB. Learning a Probabilistic Latent Space of Object Shapes via 3D Generative-Adversarial Modeling. *CoRR*. 2016;abs/1610.07584. Available from: <http://arxiv.org/abs/1610.07584>.
- [13] Wang X, au2 MHAI, Lee GH. Cascaded Refinement Network for Point Cloud Completion; 2020. Available from: <https://arxiv.org/abs/2004.03327>.
- [14] Egger B, Smith WAP, Tewari A, Wuhler S, Zollhoefer M, Beeler T, et al. 3D Morphable Face Models—Past, Present, and Future. *ACM Trans Graph*. 2020 jun;39(5). Available from: <https://doi.org/10.1145/3395208>.
- [15] Shlens J. A Tutorial on Principal Component Analysis; 2014. Available from: <https://arxiv.org/abs/1404.1100>.
- [16] Deng Y, Yang J, Xu S, Chen D, Jia Y, Tong X. Accurate 3D Face Reconstruction With Weakly-Supervised Learning: From Single Image to Image Set. In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*; 2019. p. 285-95.
- [17] Tewari A, Zollhöfer M, Kim H, Garrido P, Bernard F, Pérez P, et al.. MoFA: Model-based Deep Convolutional Face Autoencoder for Unsupervised Monocular Reconstruction; 2017. Available from: <https://arxiv.org/abs/1703.10580>.
- [18] Paysan P, Knothe R, Amberg B, Romdhani S, Vetter T. A 3D Face Model for Pose and Illumination Invariant Face Recognition. In: *2009 Sixth IEEE International Conference on Advanced Video and Signal Based Surveillance*; 2009. p. 296-301.
- [19] Tran L, Liu X. Nonlinear 3D Face Morphable Model; 2018. Available from: <https://arxiv.org/abs/1804.03786>.
- [20] Bai H, Kang D, Zhang H, Pan J, Bao L. FFHQ-UV: Normalized Facial UV-Texture Dataset for 3D Face Reconstruction; 2023. Available from: <https://arxiv.org/abs/2211.13874>.
- [21] Deng J, Cheng S, Xue N, Zhou Y, Zafeiriou S. UV-GAN: Adversarial Facial UV Map Completion for Pose-invariant Face Recognition; 2017. Available from: <https://arxiv.org/abs/1712.04695>.
- [22] Chai Z, Zhang H, Ren J, Kang D, Xu Z, Zhe X, et al. REALY: Rethinking the Evaluation of 3D Face Reconstruction. In: *Proceedings of the European Conference on Computer Vision (ECCV)*; 2022. .
- [23] Ekman P. Are There Basic Emotions? *Psychological review*. 1992 07;99:550-3.
- [24] Han X, Laga H, Bennamoun M. Image-Based 3D Object Reconstruction: State-of-the-Art and Trends in the Deep Learning Era. *IEEE Transactions on Pattern Analysis & Machine Intelligence*. 2021 may;43(05):1578-604.
- [25] Wang L, Fang Y. Unsupervised 3D Reconstruction from a Single Image via Adversarial Learning. *ArXiv*. 2017;abs/1711.09312. Available from: <https://api.semanticscholar.org/CorpusID:12529163>.

- [26] Fahim G, Amin K, Zarif S. Single-View 3D reconstruction: A Survey of deep learning methods. *Comput Graph*. 2021;94:164-90. Available from: <https://api.semanticscholar.org/CorpusID:232272326>.
- [27] Fu K, Peng J, He Q, Zhang H. Single image 3D object reconstruction based on deep learning: A review. *Multimedia Tools and Applications*. 2020;80:463-498. Available from: <https://api.semanticscholar.org/CorpusID:225273174>.
- [28] Goodfellow IJ, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, et al. Generative Adversarial Nets. In: *Neural Information Processing Systems*; 2014. Available from: <https://api.semanticscholar.org/CorpusID:261560300>.
- [29] Ho J, Jain A, Abbeel P. Denoising Diffusion Probabilistic Models. *CoRR*. 2020;abs/2006.11239. Available from: <https://arxiv.org/abs/2006.11239>.
- [30] Mildenhall B, Srinivasan PP, Tancik M, Barron JT, Ramamoorthi R, Ng R. NeRF: representing scenes as neural radiance fields for view synthesis. *Commun ACM*. 2021 dec;65(1):99-106. Available from: <https://doi.org/10.1145/3503250>.
- [31] Kerbl B, Kopanas G, Leimkuehler T, Drettakis G. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Trans Graph*. 2023 jul;42(4). Available from: <https://doi.org/10.1145/3592433>.
- [32] Wu Z, Song S, Khosla A, Yu F, Zhang L, Tang X, et al. 3D ShapeNets: A deep representation for volumetric shapes. In: *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Los Alamitos, CA, USA: IEEE Computer Society; 2015. p. 1912-20. Available from: <https://doi.ieeecomputersociety.org/10.1109/CVPR.2015.7298801>.
- [33] Yang B, Rosa S, Markham A, Trigoni N, Wen H. 3D Object Dense Reconstruction from a Single Depth View. *CoRR*. 2018;abs/1802.00411. Available from: <http://arxiv.org/abs/1802.00411>.
- [34] Wang W, Huang Q, You S, Yang C, Neumann U. Shape Inpainting Using 3D Generative Adversarial Network and Recurrent Convolutional Networks. In: *2017 IEEE International Conference on Computer Vision (ICCV)*. Los Alamitos, CA, USA: IEEE Computer Society; 2017. p. 2317-25. Available from: <https://doi.ieeecomputersociety.org/10.1109/ICCV.2017.252>.
- [35] Dai A, Qi CR, Nießner M. Shape Completion using 3D-Encoder-Predictor CNNs and Shape Synthesis. *CoRR*. 2016;abs/1612.00101. Available from: <http://arxiv.org/abs/1612.00101>.
- [36] Meagher DJ. Geometric modeling using octree encoding. *Comput Graph Image Process*. 1982;19:85. Available from: <https://api.semanticscholar.org/CorpusID:205045003>.
- [37] Riegler G, Ulusoy AO, Geiger A. OctNet: Learning Deep 3D Representations at High Resolutions. In: *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*; 2017. p. 6620-9.
- [38] Tahir R, Sargano AB, Habib Z. Voxel-Based 3D Object Reconstruction from Single 2D Image Using Variational Autoencoders. *Mathematics*. 2021. Available from: <https://api.semanticscholar.org/CorpusID:240537138>.

- [39] Ronneberger O, Fischer P, Brox T. U-Net: Convolutional Networks for Biomedical Image Segmentation. CoRR. 2015;abs/1505.04597. Available from: <http://arxiv.org/abs/1505.04597>.
- [40] Simonyan K, Zisserman A. Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv 14091556. 2014 09.
- [41] He K, Zhang X, Ren S, Sun J. Deep Residual Learning for Image Recognition. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR); 2016. p. 770-8.
- [42] Koh A, Tan SY, Nasrudin M. A systematic literature review of generative adversarial networks (GANs) in 3D avatar reconstruction from 2D images. Multimedia Tools and Applications. 2024 03:1-41.
- [43] Huber P, Hu G, Tena JR, Mortazavian P, Koppen WP, Christmas WJ, et al. A Multiresolution 3D Morphable Face Model and Fitting Framework. In: VISIGRAPP; 2016. Available from: <https://api.semanticscholar.org/CorpusID:11525295>.
- [44] Booth J, Roussos A, Zafeiriou S, Ponniah A, Dunaway D. A 3D Morphable Model Learnt from 10,000 Faces. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR); 2016. p. 5543-52.
- [45] Sirovich L, Kirby M. Low-dimensional procedure for the characterization of human faces. Journal of the Optical Society of America A, Optics and image science. 1987;4 3:519-24. Available from: <https://api.semanticscholar.org/CorpusID:982857>.
- [46] Turk MA, Pentland A. Eigenfaces for Recognition. Journal of Cognitive Neuroscience. 1991;3:71-86. Available from: <https://api.semanticscholar.org/CorpusID:262317668>.
- [47] Amberg B, Knothe R, Vetter T. Expression invariant 3D face recognition with a Morphable Model. 2008 8th IEEE International Conference on Automatic Face & Gesture Recognition. 2008:1-6. Available from: <https://api.semanticscholar.org/CorpusID:5651453>.
- [48] Yang F, Wang J, Shechtman E, Bourdev L, Metaxas D. Expression flow for 3D-aware face component transfer. ACM Trans Graph. 2011 jul;30(4). Available from: <https://doi.org/10.1145/2010324.1964955>.
- [49] Thies J, Zollhöfer M, Nießner M, Valgaerts L, Stamminger M, Theobalt C. Real-time expression transfer for facial reenactment. ACM Trans Graph. 2015 nov;34(6). Available from: <https://doi.org/10.1145/2816795.2818056>.
- [50] Basso C, Verri A. FITTING 3D MORPHABLE MODELS USING IMPLICIT REPRESENTATIONS. In: Proceedings of the Second International Conference on Computer Graphics Theory and Applications - Volume 1: GRAPP, INSTICC. SciTePress; 2007. p. 45-52.
- [51] ter Haar FB, Veltkamp RC. 3D Face Model Fitting for Recognition. In: Computer Vision – ECCV 2008: 10th European Conference on Computer Vision, Marseille, France, October 12-18, 2008, Proceedings, Part IV. Berlin, Heidelberg: Springer-Verlag; 2003. p. 652–664. Available from: https://doi.org/10.1007/978-3-540-88693-8_48.

- [52] Tena JR, De la Torre F, Matthews I. Interactive region-based linear 3D face models. In: ACM SIGGRAPH 2011 Papers. SIGGRAPH '11. New York, NY, USA: Association for Computing Machinery; 2011. Available from: <https://doi.org/10.1145/1964921.1964971>.
- [53] Yenamandra T, Tewari A, Bernard F, Seidel HP, Elgharib M, Cremers D, et al.. i3DMM: Deep Implicit 3D Morphable Model of Human Heads; 2020. Available from: <https://arxiv.org/abs/2011.14143>.
- [54] Ploumpis S, Wang H, Pears N, Smith WAP, Zafeiriou S. Combining 3D Morphable Models: A Large scale Face-and-Head Model; 2019. Available from: <https://arxiv.org/abs/1903.03785>.
- [55] Cao C, Weng Y, Zhou S, Tong Y, Zhou K. FaceWarehouse: A 3D Facial Expression Database for Visual Computing. IEEE Transactions on Visualization and Computer Graphics. 2014;20(3):413-25.
- [56] Li T, Bolkart T, Black MJ, Li H, Romero J. Learning a model of facial shape and expression from 4D scans. ACM Trans Graph. 2017 nov;36(6). Available from: <https://doi.org/10.1145/3130800.3130813>.
- [57] Dai H, Pears N, Smith W, Duncan C. Statistical Modeling of Craniofacial Shape and Texture. Int J Comput Vision. 2020 feb;128(2):547–571. Available from: <https://doi.org/10.1007/s11263-019-01260-7>.
- [58] Bao L, Lin X, Chen Y, Zhang H, Wang S, Zhe X, et al.. High-Fidelity 3D Digital Human Head Creation from RGB-D Selfies; 2021. Available from: <https://arxiv.org/abs/2010.05562>.
- [59] He K, Zhang X, Ren S, Sun J. Deep Residual Learning for Image Recognition; 2015. Available from: <https://arxiv.org/abs/1512.03385>.
- [60] Blanz V, Mehl A, Vetter T, Seidel HP. A Statistical Method for Robust 3D Surface Reconstruction from Sparse Data. In: Proceedings of the 3D Data Processing, Visualization, and Transmission, 2nd International Symposium. 3DPVT '04. USA: IEEE Computer Society; 2004. p. 293–300.
- [61] Hassner T, Harel S, Paz E, Enbar R. Effective Face Frontalization in Unconstrained Images; 2014. Available from: <https://arxiv.org/abs/1411.7964>.
- [62] Bulat A, Tzimiropoulos G. How Far are We from Solving the 2D & 3D Face Alignment Problem? (and a Dataset of 230,000 3D Facial Landmarks). In: 2017 IEEE International Conference on Computer Vision (ICCV). IEEE; 2017. Available from: <http://dx.doi.org/10.1109/ICCV.2017.116>.
- [63] Krizhevsky A, Sutskever I, Hinton GE. ImageNet classification with deep convolutional neural networks. Commun ACM. 2017 may;60(6):84–90. Available from: <https://doi.org/10.1145/3065386>.
- [64] Parkhi OM, Vedaldi A, Zisserman A. Deep Face Recognition. In: British Machine Vision Conference; 2015. Available from: <https://api.semanticscholar.org/CorpusID:4637184>.

- [65] Bruna J, Zaremba W, Szlam A, LeCun Y. Spectral Networks and Locally Connected Networks on Graphs; 2014. Available from: <https://arxiv.org/abs/1312.6203>.
- [66] Defferrard M, Bresson X, Vandergheynst P. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering; 2017. Available from: <https://arxiv.org/abs/1606.09375>.
- [67] Ranjan A, Bolkart T, Sanyal S, Black MJ. Generating 3D faces using Convolutional Mesh Autoencoders; 2018. Available from: <https://arxiv.org/abs/1807.10267>.
- [68] Lin J, Yuan Y, Shao T, Zhou K. Towards High-Fidelity 3D Face Reconstruction from In-the-Wild Images Using Graph Convolutional Networks; 2020. Available from: <https://arxiv.org/abs/2003.05653>.
- [69] Schroff F, Kalenichenko D, Philbin J. FaceNet: A unified embedding for face recognition and clustering. In: 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). IEEE; 2015. Available from: <http://dx.doi.org/10.1109/CVPR.2015.7298682>.
- [70] Deng Q, Le BH, Jin A, Deng Z. End-to-End 3D Face Reconstruction with Expressions and Specular Albedos from Single In-the-wild Images. In: Proceedings of the 30th ACM International Conference on Multimedia. MM '22. New York, NY, USA: Association for Computing Machinery; 2022. p. 4694–4703. Available from: <https://doi.org/10.1145/3503161.3547800>.
- [71] Yamaguchi S, Saito S, Nagano K, Zhao Y, Chen W, Olszewski K, et al. High-fidelity facial reflectance and geometry inference from an unconstrained image. ACM Trans Graph. 2018 jul;37(4). Available from: <https://doi.org/10.1145/3197517.3201364>.
- [72] Lee GH, Lee SW. Uncertainty-Aware Mesh Decoder for High Fidelity 3D Face Reconstruction. 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2020:6099-108. Available from: <https://api.semanticscholar.org/CorpusID:219629860>.
- [73] Luo H, Nagano K, Kung HW, Goldwhite M, Xu Q, Wang Z, et al.. Normalized Avatar Synthesis Using StyleGAN and Perceptual Refinement; 2021. Available from: <https://arxiv.org/abs/2106.11423>.
- [74] Gecer B, Ploumpis S, Kotsia I, Zafeiriou S. GANFIT: Generative Adversarial Network Fitting for High Fidelity 3D Face Reconstruction. In: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). IEEE; 2019. Available from: <http://dx.doi.org/10.1109/CVPR.2019.00125>.
- [75] Cole F, Belanger D, Krishnan D, Sarna A, Mosseri I, Freeman WT. Synthesizing Normalized Faces from Facial Identity Features; 2017. Available from: <https://arxiv.org/abs/1701.04851>.
- [76] Nagano K, Luo H, Wang Z, Seo J, Xing J, Hu L, et al. Deep face normalization. ACM Transactions on Graphics (TOG). 2019;38:1 16. Available from: <https://api.semanticscholar.org/CorpusID:207998138>.
- [77] Tov O, Alaluf Y, Nitzan Y, Patashnik O, Cohen-Or D. Designing an Encoder for StyleGAN Image Manipulation; 2021. Available from: <https://arxiv.org/abs/2102.02766>.

- [78] Karras T, Laine S, Aittala M, Hellsten J, Lehtinen J, Aila T. Analyzing and Improving the Image Quality of StyleGAN; 2020. Available from: <https://arxiv.org/abs/1912.04958>.
- [79] Zhou H, Hadap S, Sunkavalli K, Jacobs D. Deep Single-Image Portrait Relighting. In: 2019 IEEE/CVF International Conference on Computer Vision (ICCV); 2019. p. 7193-201.
- [80] Abdal R, Zhu P, Mitra NJ, Wonka P. StyleFlow: Attribute-conditioned Exploration of StyleGAN-Generated Images using Conditional Continuous Normalizing Flows. *ACM Trans Graph*. 2021 may;40(3). Available from: <https://doi.org/10.1145/3447648>.
- [81] Shen Y, Gu J, Tang X, Zhou B. Interpreting the Latent Space of GANs for Semantic Face Editing; 2020. Available from: <https://arxiv.org/abs/1907.10786>.
- [82] Burt PJ, Adelson EH. A multiresolution spline with application to image mosaics. *ACM Trans Graph*. 1983 oct;2(4):217–236. Available from: <https://doi.org/10.1145/245.247>.
- [83] Yin X, Yu X, Sohn K, Liu X, Chandraker M. Towards Large-Pose Face Frontalization in the Wild. In: 2017 IEEE International Conference on Computer Vision (ICCV). Los Alamitos, CA, USA: IEEE Computer Society; 2017. p. 4010-9. Available from: <https://doi.ieeecomputersociety.org/10.1109/ICCV.2017.430>.
- [84] Huang R, Zhang S, Li T, He R. Beyond Face Rotation: Global and Local Perception GAN for Photorealistic and Identity Preserving Frontal View Synthesis; 2017. Available from: <https://arxiv.org/abs/1704.04086>.
- [85] Chang AX, Funkhouser T, Guibas L, Hanrahan P, Huang Q, Li Z, et al.. ShapeNet: An Information-Rich 3D Model Repository; 2015. Available from: <https://arxiv.org/abs/1512.03012>.
- [86] Girdhar R, Fouhey DF, Rodriguez M, Gupta A. Learning a Predictable and Generative Vector Representation for Objects. *CoRR*. 2016;abs/1603.08637. Available from: <http://arxiv.org/abs/1603.08637>.
- [87] Kingma DP, Welling M. An Introduction to Variational Autoencoders. *CoRR*. 2019;abs/1906.02691. Available from: <http://arxiv.org/abs/1906.02691>.
- [88] Larsen ABL, Sønderby SK, Winther O. Autoencoding beyond pixels using a learned similarity metric. *CoRR*. 2015;abs/1512.09300. Available from: <http://arxiv.org/abs/1512.09300>.
- [89] Lim JJ, Pirsiavash H, Torralba A. Parsing IKEA Objects: Fine Pose Estimation. In: Proceedings of the 2013 IEEE International Conference on Computer Vision. ICCV '13. USA: IEEE Computer Society; 2013. p. 2992–2999. Available from: <https://doi.org/10.1109/ICCV.2013.372>.
- [90] Yuan W, Khot T, Held D, Mertz C, Hebert M. PCN: Point Completion Network; 2019. Available from: <https://arxiv.org/abs/1808.00671>.
- [91] Qi CR, Su H, Mo K, Guibas LJ. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation; 2017. Available from: <https://arxiv.org/abs/1612.00593>.

- [92] Achlioptas P, Diamanti O, Mitliagkas I, Guibas L. Learning Representations and Generative Models for 3D Point Clouds; 2018. Available from: <https://arxiv.org/abs/1707.02392>.
- [93] Yang Y, Feng C, Shen Y, Tian D. FoldingNet: Interpretable Unsupervised Learning on 3D Point Clouds. ArXiv. 2017;abs/1712.07262. Available from: <https://api.semanticscholar.org/CorpusID:21221734>.
- [94] Fan H, Su H, Guibas L. A Point Set Generation Network for 3D Object Reconstruction from a Single Image; 2016. Available from: <https://arxiv.org/abs/1612.00603>.
- [95] Stasinakis A, Chatzilari E, Nikolopoulos S, Kompatsiaris I, Karolidis D, Touloumtzidou A, et al. A hybrid 3D object auto-completion approach with self-supervised data augmentation for fragments of archaeological objects. *Journal of Cultural Heritage*. 2022 07;56:138-48.
- [96] Chang Y, Jung C, Xu Y. FinerPCN: High fidelity point cloud completion network using pointwise convolution. *Neurocomput*. 2021 oct;460(C):266–276. Available from: <https://doi.org/10.1016/j.neucom.2021.06.080>.
- [97] Yuan Y, Li X, Huang Y, Mello SD, Nagano K, Kautz J, et al. GAvatar: Animatable 3D Gaussian Avatars with Implicit Mesh Learning. ArXiv. 2023;abs/2312.11461. Available from: <https://api.semanticscholar.org/CorpusID:266359217>.
- [98] Anciukevicius T, Xu Z, Fisher M, Henderson P, Bilen H, Mitra NJ, et al. RenderDiffusion: Image Diffusion for 3D Reconstruction, Inpainting and Generation. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). Los Alamitos, CA, USA: IEEE Computer Society; 2023. p. 12608-18. Available from: <https://doi.ieeecomputersociety.org/10.1109/CVPR52729.2023.01213>.
- [99] Guo Y, Zhang J, Cai J, Jiang B, Zheng J. CNN-based Real-time Dense Face Reconstruction with Inverse-rendered Photo-realistic Face Images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2019;41(6):1294-307.
- [100] Kingma DP, Ba J. Adam: A Method for Stochastic Optimization; 2017. Available from: <https://arxiv.org/abs/1412.6980>.
- [101] Qi C, Yi L, Su H, Guibas LJ. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. In: *Neural Information Processing Systems*; 2017. Available from: <https://api.semanticscholar.org/CorpusID:1745976>.
- [102] Yang Y, Feng C, Shen Y, Tian D. FoldingNet: Point Cloud Auto-encoder via Deep Grid Deformation; 2018. Available from: <https://arxiv.org/abs/1712.07262>.
- [103] Yuksel C. Sample Elimination for Generating Poisson Disk Sample Sets. *Comput Graph Forum*. 2015 may;34(2):25–32. Available from: <https://doi.org/10.1111/cgf.12538>.
- [104] Moenning C, Dodgson NA. Fast Marching farthest point sampling. In: *Eurographics*; 2003. Available from: <https://api.semanticscholar.org/CorpusID:11826820>.
- [105] Mao X, Li Q, Xie H, Lau RYK, Wang Z, Smolley SP. Least Squares Generative Adversarial Networks; 2017. Available from: <https://arxiv.org/abs/1611.04076>.

- [106] Kazhdan M, Hoppe H. Screened poisson surface reconstruction. *ACM Trans Graph.* 2013 jul;32(3). Available from: <https://doi.org/10.1145/2487228.2487237>.
- [107] Zhu C, Byrd RH, Lu P, Nocedal J. Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization. *ACM Trans Math Softw.* 1997 dec;23(4):550–560. Available from: <https://doi.org/10.1145/279232.279236>.
- [108] Deng J, Guo J, Xue N, Zafeiriou S. ArcFace: Additive Angular Margin Loss for Deep Face Recognition. In: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR); 2019. p. 4685-94.
- [109] Viola P, Jones M. Rapid object detection using a boosted cascade of simple features. In: Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001. vol. 1; 2001. p. I-I.
- [110] Karras T, Aila T, Laine S, Lehtinen J. Progressive Growing of GANs for Improved Quality, Stability, and Variation; 2018. Available from: <https://arxiv.org/abs/1710.10196>.
- [111] Chen Z, Yin K, Fidler S. AUV-Net: Learning Aligned UV Maps for Texture Transfer and Synthesis; 2022. Available from: <https://arxiv.org/abs/2204.03105>.
- [112] Chai Z, Zhang T, He T, Tan X, Baltrušaitis T, Wu H, et al.. HiFace: High-Fidelity 3D Face Reconstruction by Learning Static and Dynamic Details; 2023. Available from: <https://arxiv.org/abs/2303.11225>.

Appendix

Appendix A

Survey for the 3D Face Reconstruction with FFHQ-UV

Evaluation of the 2D-Image to 3D-Head Avatar Reconstruction

Start of Block: Default Question Block



Q1 Thank you for considering participation in this survey, which is being conducted as part of an MSc Computing Individual Project. The purpose of this survey is to gather feedback on the effectiveness of a 3D-head avatar machine-learning reconstruction pipeline rendered from a single 2D-image to facilitate a digital psychotherapy intervention. This intervention is called the Self-Attachment Techniques (SAT) developed by the Algorithmic Human Development Group at Imperial College London, with the aim of introducing 3D avatars that can be applied to a VR-platform to improve engagement with the therapeutic process. Your response would be greatly appreciated and will help towards improving the design of the reconstruction pipeline and quality of model.

Your participation is voluntary, and you are completely free to withdraw your response from the survey at any time, without giving any reason and without being penalised or disadvantaged in any way. The data that you provide will only be used for analysis purposes, and will be deleted once the project is completed. It will be stored securely, kept anonymous and made strictly confidential. The data that you provide here and the image you provided for reconstruction are used only for research purposes, and will not be shared with third-parties. If you choose to withdraw, any data we hold on you including these responses will be deleted.

This survey is designed to be taken after receiving your reconstruction results, which will be provided by the researcher in the form of a short video showing the reconstruction output and a time in seconds indicating how long it took to reconstruct the head model. In the survey, you will first be directed to answer a few demographic questions regarding your age, gender and ethnicity. If you do not want to answer these questions, you may optionally leave them out or withdraw from the study entirely. Following this section, are some basic Likert scale questions designed to evaluate the effectiveness of the reconstruction. In the question requiring inputting the time for reconstruction, please use the time that the researcher provided to you. At the end of the study, we ask for some qualitative feedback on the model, but this is completely optional.

If you have any questions or concerns about the survey, please contact the researcher Raymond Guo by email: raymond.guo23@imperial.ac.uk. Alternatively, if you have any concerns or complaints regarding the way the study has been carried out you may also reach out to Professor Abbas Edalat, the supervisor of this study at a.edalat@imperial.ac.uk.

By clicking "I agree to participate in this survey" below, you acknowledge that you have read and understood this information and consent to participating in this survey.

☐ I agree to participate in this survey

Page Break

Q2 Before the survey begins, we would like to ask a few questions about you. Your answers will remain anonymous and strictly confidential.

Q3 Please enter your age.

Q4 Please select your gender.

☐ Male

☐ Female

☐ Prefer to self-describe

☐ Prefer not to say

Q5 Please select your ethnicity (UK Census 2021)

▼ Asian, Asian British or Asian Welsh: Bangladeshi ... Prefer not to say

Q5.1 If "Other ethnic group: Any other ethnic group" was selected, please specify:

Page Break

Q6 Overall, how satisfied are you with the quality of the head reconstruction?

- ☐ Very dissatisfied
- ☐ Somewhat dissatisfied
- ☐ Neither satisfied nor dissatisfied
- ☐ Somewhat satisfied
- ☐ Very satisfied
-

Q7 How accurately do you feel the reconstruction captures these features?

	Not accurately at all	Not too accurately	Neither accurately nor inaccurately	Quite accurately	Very accurately
Your eyes	☐	☐	☐	☐	☐
Your nose	☐	☐	☐	☐	☐
Your mouth	☐	☐	☐	☐	☐
Your cheeks	☐	☐	☐	☐	☐

Q8 How well does the reconstruction capture the proportions of your face? This refers to the size and space of facial features relative to each other.

- ☐ Not well at all
 - ☐ Not too well
 - ☐ Neither well nor unwell
 - ☐ Quite well
 - ☐ Very well
-

JS

Q9 How well does the reconstruction capture your skin tone and colour?

- ☐ Not well at all
 - ☐ Not too well
 - ☐ Neither well nor unwell
 - ☐ Quite well
 - ☐ Very well
-

JS

Q10 How well does the reconstruction capture the finer features of your face (e.g. wrinkles, pores, freckles)?

- ☐ Not well at all
 - ☐ Not too well
 - ☐ Neither well nor unwell
 - ☐ Quite well
 - ☐ Very well
-



Q11 How well does the lighting in the head model reflect the lighting in your provided image?

- ☐ Not well at all
 - ☐ Not too well
 - ☐ Neither well nor unwell
 - ☐ Quite well
 - ☐ Very well
-

Page Break

Q12 Please indicate the time it took to reconstruct the model in seconds.

JS

Q13 What do you think about the speed of this reconstruction?

- ☐ Very slow
- ☐ Quite slow
- ☐ Neither fast nor slow
- ☐ Quite fast
- ☐ Very fast

JS

Q14 In an application, would you generally prefer to prioritise the time it takes to reconstruct the model or the rendering quality of the model?

- ☐ Definitely time
- ☐ Time is a bit more important than quality
- ☐ Both are equally important
- ☐ Quality is a bit more important than time
- ☐ Definitely quality

JS

Q15 How likely are you to approve of this reconstruction for use in other applications (e.g. digital psychotherapy, gaming, virtual reality, etc.)?

- ☐ Very unlikely
- ☐ Quite unlikely
- ☐ Neither likely nor unlikely
- ☐ Quite likely
- ☐ Very likely

Page Break

Q16 Please provide any feedback on where you think the model could be improved or if there are any features that you would like to see added in the future.

Page Break

Q17 Finally, would you be happy for the images taken from the short reconstruction video sent to you to be included in the published report?

- ☐ Yes, that is fine
- ☐ I would prefer not to

End of Block: Default Question Block
